Facultad de Ingeniería
Universidad Nacional de Mar del Plata

Universidad Nacional de Mar del Plata

Facultad de Ingeniería

Trabajo final

Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia

Victoria Carla Torres



Ingeniería Electrónica
Departamento de Ingeniería Electrónica y Computación
Septiembre 2026

Agradecimientos

Índice

Agradecimientos **1**

Siglas y abreviaturas **4**

Resumen **4**

1 Introducción **6**
1.1 Contexto y motivación . . . 6
1.2 Partes interesadas . . . 7
1.3 Objetivos . . . 7
1.4 Organización del informe . . . 7

2 Marco teórico **8**
2.1 Analizador Vectorial de Redes . . . 8
2.2 Parámetros de dispersión (S) . . . 8
2.2.1 Parámetro S11 . . . 9
2.2.2 Calibración S11 . . . 9
2.2.2.1 Errores . . . 9
2.2.2.2 Método SOL . . . 9
2.3 Acoplador Direccional . . . 10
2.3.1 Puente resistivo . . . 11
2.4 Buenas prácticas de diseño en RF . . . 12
2.5 Heterodinación de señales . . . 12
2.6 Procesamiento digital de señales . . . 13
2.6.1 Filtros digitales . . . 13
2.6.1.1 Diseño de filtros FIR mediante el método del enventanado . . . 14
2.6.2 Transformada Rápida de Fourier . . . 15

3 Proyecto **16**
3.1 Descripción general del producto . . . 16
3.2 Hardware . . . 16
3.2.1 Generadores de RF . . . 17
3.2.1.1 Filtro externo de lazo de control . . . 18
3.2.2 Oscilador . . . 19
3.2.3 Mezclador . . . 19
3.2.4 Acoplador direccional . . . 19
3.2.5 Amplificadores diferenciales . . . 20
3.2.6 Alimentación y conectores . . . 21
3.2.7 Microcontrolador . . . 21
3.2.8 Implementación en PCB . . . 21
3.3 Firmware . . . 23
3.3.1 Cálculo de registros . . . 23

3.3.2 Transmisión SPI y secuencia de sincronización . . . 24
3.4 Software . . . 24
3.4.1 Flujo de barrido . . . 25
3.4.2 Estimación del S11 . . . 25
3.4.3 Calibración . . . 26
3.5 Interfaz de usuario . . . 26

4 Resultados **28**
4.1 Ensamblado de la placa final . . . 28
4.2 Ficha técnica . . . 30
4.3 Pruebas de integración . . . 30
4.4 Mediciones de sistemas bajo prueba . . . 34
4.4.1 Cortocircuitos . . . 34
4.4.2 Circuito abierto . . . 35
4.4.3 Adaptación (50 Ω) . . . 35
4.5 Análisis de resultados . . . 35
4.5.1 Sensor resonante . . . 36
4.6 Interfaz de usuario . . . 37
4.7 Comparación con dispositivo comercial . . . 37

5 Conclusiones y trabajos futuros **39**
5.1 Trabajos futuros . . . 40

6 Esquemáticos y código **41**
6.1 Esquemáticos Altium . . . 41
6.1.1 Generador de RF incidente (ADF4351) . . . 42
6.1.2 Generador de RF Oscilador Local (ADF4351) . . . 43
6.1.3 Mezclador . . . 44
6.1.4 Alimentación y CLK . . . 45
6.1.5 Acoplador Direccional . . . 46
6.1.6 Amplificadores de FI . . . 47
6.1.7 Atenuadores . . . 48
6.2 Firmware . . . 49
6.3 Software . . . 53
6.4 Interfaz de usuario . . . 62
6.4.1 Core . . . 62
6.4.2 Graphical User Interface . . . 68

Apéndices **81**
A Estudio Preliminar . . . 83
B Plan de Trabajo . . . 87
C Especificación de Requerimientos . . . 91
D Especificación Funcional . . . 97
E Especificación Técnica . . . 105
F Plan de Pruebas . . . 117

Bibliografía **123**

Siglas y abreviaturas

Nombre	Descripción
VNA	Analizador Vectorial de Redes
UHF	Ultra High Frequency
CGM	Continous Glucose Monitoring
CONICET	Consejo Nacional de Investigaciones Científicas y Técnicas
LAC	Laboratorio de Comunicaciones
ICYTE	Instituto de Investigaciones Científicas y Tecnológicas en Electrónica
UNMDP	Universidad Nacional de Mar del Plata
FI	Frecuencia Intermedia
DUT	Device Under Test
RF	Radiofrecuencia
SOL	Short, Open, Load
PCB	Printed Circuit Board
SMD	Surface-Mount Device
USB	Universal Serial Bus
CPU	Central Processing Unit
CLK	Clock (señal de reloj)
OL	Oscilador Local
FIR	Finite Impulse Response
FFT	Fast Fourier Transform
IC	Integrated Circuits
VCO	Voltage Controlled Oscillator
PFD	Phase Frequency Detector
SMA	SubMiniature version A
SPI	Serial Peripheral Interface
CSV	Comma-Separated Values
IA	Inteligencia Artificial
GUI	Graphical User Interface
QFN	Quad Flat No-Lead
ADC	Analog-to-Digital Converter

Resumen

El proyecto propone el diseño, implementación, calibración y ensayo de un Analizador Vectorial de Redes (VNA) de un puerto, que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 hasta los 2,2 GHz, adaptado para operar en conjunto con un sensor de glucemia no invasivo.

El principio de medición se basa en la existencia de una correlación entre la concentración de glucosa en sangre y sus propiedades eléctricas (permitividad relativa y conductividad) [1], por lo que es razonable suponer que las variaciones en la composición dieléctrica de una muestra de tejido biológico pueden alterar la distribución de los campos electromagnéticos presentes en un sensor de microondas.

La implementación se realiza en una placa de circuito impreso, con circuitos integrados especializados y aplicando técnicas de reflectometría, diseño en radiofrecuencia y manejo de emisiones electromagnéticas.

Se obtienen señales analógicas en la banda de audio para su adquisición en la placa de sonido de una PC. Luego, el análisis de las señales es puramente digital. Se aplica un procesamiento basado en la Transformada Rápida de Fourier (FFT) para obtener el coeficiente de reflexión del sistema bajo prueba en función de la frecuencia.

Capítulo 1

Introducción

1.1 Contexto y motivación

La diabetes es una afección crónica que se desencadena cuando el organismo pierde su capacidad de producir suficiente insulina o de utilizarla con eficacia para mantener los niveles de glucosa en la sangre dentro de los márgenes normales. La cantidad de adultos que la padecen se ha casi cuadruplicado entre 1980 y 2016, con un aumento en la prevalencia más acelerado en países de ingresos bajos y medios que en países de ingresos altos [2]. Para 2021 fue la causa directa de 1,6 millones de muertes [3]. Existen numerosos dispositivos que permiten realizar un control de la glucosa en sangre pero requieren, en general, de una punción en alguna parte del cuerpo para extraer una muestra de sangre. El mecanismo tradicional requiere depositar la muestra en una tira reactiva dentro de un medidor para obtener el valor de glucemia. Se recomienda, en promedio, que los pacientes realicen 4 mediciones al día (dependiendo de la estabilidad de los niveles o del régimen de tratamiento). Dado que la extracción suele ser dolorosa, además de que tiene un costo asociado al gasto de cada tira reactiva, la mayoría de los pacientes no logran tener un buen monitoreo de su glucemia. Por este motivo se ha incursionado en la glucometría semi-invasiva como los sistemas CGM (Continuous Glucose Monitoring) [4] que obtienen mediciones del líquido intersticial y requieren la colocación de un sensor debajo de la piel que se debe reemplazar periódicamente, por lo que siguen siendo invasivos, incómodos de usar y descartables. El estudio de propuestas para la medición no invasiva ha avanzado en los últimos años, pero aún no existe una técnica ampliamente adoptada ni dispositivos comerciales fiables. Se ha observado que existe una correlación entre la concentración de glucosa en sangre y sus propiedades eléctricas (permitividad relativa y conductividad) [1], por lo que es razonable suponer que las variaciones en la composición dieléctrica de una muestra de tejido biológico pueden alterar la distribución de los campos electromagnéticos presentes en un sensor de microondas.

Por esta razón el Laboratorio de Comunicaciones (LAC), perteneciente al Instituto de Investigaciones Científicas y Tecnológicas en Electrónica (ICYTE), que depende de la Universidad Nacional de Mar del Plata (UNMDP) y del Consejo Nacional de Investigaciones Científicas y Técnicas (CONICET) tiene una línea de investigación cuyo objetivo es diseñar e implementar un prototipo que permita realizar una medición de glucemia, basado en un VNA y un sensor de microondas.

Un Analizador Vectorial de Redes es un instrumento de medición que permite caracterizar sistemas en alta frecuencia. La forma en que lo hace es haciendo incidir una señal de microondas en los puertos del sistema, y midiendo las señales que salen del resto de los puertos.

El Laboratorio busca optimizar, compactar y mejorar el rendimiento del VNA actual, por lo que requiere el diseño y desarrollo de un VNA de un puerto que esté adaptado a un sensor de microondas que resuena alrededor de los 2 GHz.

1.2 Partes interesadas

El Grupo de Investigación del LAC es quien solicita el desarrollo. Por esta razón, el equipo se utilizará bajo su supervisión, y sus posibles usuarios son:

- Ingenieros, técnicos y/o radioaficionados;
- Estudiantes de ingeniería.

1.3 Objetivos

El objetivo principal del proyecto es el diseño, desarrollo, calibración y ensayo de un VNA de bajo costo y tamaño compacto, que opere en la banda de frecuencias de UHF que abarca desde 1,8 a 2,2 GHz, orientado a la medición no invasiva de glucemia.

Los objetivos particulares son:

- Implementar en una placa de circuito impreso el VNA;
- Generar un barrido en la banda de frecuencias de interés;
- Implementar un sistema heterodino que permita trabajar con señales de frecuencia intermedia (FI) en la banda de audio;
- Digitalizar las señales obtenidas para aplicar un tratamiento digital que permita calcular el coeficiente de reflexión del sistema bajo prueba (DUT - *Device Under Test*);
- Implementar una rutina de calibración para eliminar errores sistemáticos del sistema;

1.4 Organización del informe

El informe se organiza de la siguiente manera: en el capítulo primero se introduce el proyecto y se brinda un punteo de los objetivos más importantes. El capítulo 2 se centra en el marco teórico, donde se explican las bases para entender el funcionamiento del dispositivo. El capítulo 3 describe el proyecto con su implementación. En el capítulo 4 se muestran los resultados y mediciones. El capítulo 5 se dedica a conclusiones y trabajos futuros, y el 6 exhibe los esquemáticos y códigos utilizados. Por último se anexan los apéndices en donde se encuentran: el estudio preliminar, el plan de trabajo, las especificaciones de requerimientos, funcional y técnica y el plan de pruebas.

Capítulo 2

Marco teórico

2.1 Analizador Vectorial de Redes

El Analizador Vectorial de Redes fue definido en sus inicios como un instrumento que permite medir los parámetros de redes eléctricas [5]. En su esquema más básico posee una fuente de radiofrecuencias (RF), que genera una señal de estímulo para el dispositivo bajo prueba, y un banco de receptores que miden las señales que salen del DUT y las comparan con la onda incidente. Los resultados se procesan y muestran al usuario por medio de alguna interfaz.

Dependiendo de los dispositivos que se buscan caracterizar, se determina la cantidad de puertos que debe tener el VNA. Un puerto permite medir el parámetro de reflexión. Con dos puertos, se puede medir además el de transmisión. Si posee un solo canal, se puede medir la transmisión inversa y la reflexión en la salida invirtiendo físicamente el dispositivo bajo prueba. Este paso se puede evitar agregando un canal, lo que incrementa el costo del VNA.

2.2 Parámetros de dispersión (S)

Los parámetros de dispersión, también llamados S por su nombre en inglés (*Scattering*) permiten caracterizar redes de dos o más puertos en función de sus ondas entrantes y salientes. Se especifican para una impedancia de referencia y a una frecuencia específica, y son útiles en altas frecuencias porque otros tipos de parámetros (como los Z o los Y) no pueden ser medidos directamente.

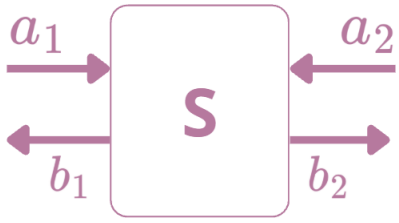


Figura 2.1: Parámetros S en una red de dos puertos

Según las ondas incidentes a_1 y a_2 y las reflejadas b_1 y b_2, los parámetros S se definen como:

$$S_{11} = \frac{b_1}{a_1}\Big|_{a_2=0};\ S_{12} = \frac{b_1}{a_2}\Big|_{a_1=0};\ S_{21} = \frac{b_2}{a_1}\Big|_{a_2=0};\ S_{22} = \frac{b_2}{a_2}\Big|_{a_1=0}$$

Para lograr que las a sean iguales a 0, el puerto al que pertenecen debe estar adaptado, garantizando que no haya ninguna señal incidente.

2.2.1 Parámetro S11

El parámetro S_{11} se conoce como el coeficiente de reflexión. Se puede definir en función de las ondas incidente y reflejada, o en función de la impedancia característica del sistema Z_0 y la impedancia de carga (es decir, del sistema bajo prueba):

$$S_{11}(DUT) = \Gamma = \frac{Z_L - Z_0}{Z_L + Z_0} \tag{2.2.1}$$

Expresado en dB, se expresa:

$$S_{11}(DUT)[dB] = 20.log(|\Gamma|) \tag{2.2.2}$$

2.2.2 Calibración S11

2.2.2.1 Errores

Los errores que pueden aparecer en una medición se pueden dividir en dos grandes clases: los errores sistemáticos y los aleatorios. Los primeros se pueden corregir, mientras que los segundos se encuentran en toda medición y están fuera del control del usuario.

En el caso de dispositivos electrónicos algunos errores sistemáticos son los producidos por variaciones en la potencia de salida de los generadores, pérdidas de inserción en materiales conductores y conectores, desfasajes ocasionados por diferencias en longitudes eléctricas o retardos en los circuitos integrados, reflexiones generadas por desadaptación de impedancias y deriva térmica. Algunos errores aleatorios pueden ser ruido térmico, de cuantización e interferencia electromagnética.

La calibración en un VNA permite corregir los errores sistemáticos, excluyendo: comportamientos alineales; repetibilidad en conectores; errores en las cargas de calibración; y deriva térmica. Este último se puede corregir solo si la calibración se realiza inmediatamente antes de la medición.

2.2.2.2 Método SOL

En VNAs de un puerto se pueden simplificar los errores a un modelo de tres términos que describen la directividad, adaptación del puerto y errores de seguimiento (variaciones de amplitud y fase en función de la frecuencia)[6]. Usando tres cargas distintas de valores conocidos, se puede formar un sistema de tres ecuaciones con tres incógnitas y resolver para obtener los términos del error:

$$e_{00} + \Gamma_1\Gamma_{\mathrm{M}1}e_{11} - \Gamma_1\Delta_e = \Gamma_{\mathrm{M}1} \tag{2.2.3a}$$
$$e_{00} + \Gamma_2\Gamma_{\mathrm{M}2}e_{11} - \Gamma_2\Delta_e = \Gamma_{\mathrm{M}2} \tag{2.2.3b}$$
$$e_{00} + \Gamma_3\Gamma_{\mathrm{M}3}e_{11} - \Gamma_3\Delta_e = \Gamma_{\mathrm{M}3} \tag{2.2.3c}$$

Donde Γ_{M} es el parámetro de reflexión medido para cada carga, e_{00} es la directividad, e_{11} la adaptación del puerto, $e_{10}e_{01}$ el error de seguimiento (*tracking*) y $\Delta_e = e_{00}e_{11} - (e_{10}e_{01})$. Los términos del error se pueden observar en la figura 2.2.

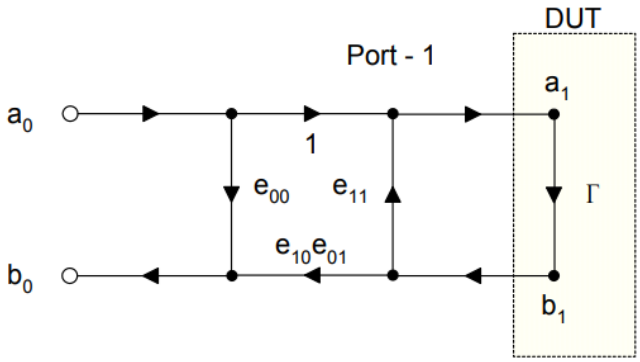


Figura 2.2: Términos de error para un VNA de un puerto

Si las cargas independientes son un circuito abierto, cortocircuito y carga de 50Ω entonces el método de calibración se denomina SOL (Short, Open, Load) y las ecuaciones (2.2.3) se simplifican a:

$$e_{00} - \Gamma_{\mathrm{MS}} e_{11} + \Delta_e = \Gamma_{\mathrm{MS}} \tag{2.2.4a}$$
$$e_{00} + \Gamma_{\mathrm{MO}} e_{11} - \Delta_e = \Gamma_{\mathrm{MO}} \tag{2.2.4b}$$
$$e_{00} = \Gamma_{\mathrm{ML}} \tag{2.2.4c}$$

Luego el Γ real se calcula como:

$$\Gamma = \frac{\Gamma_M - e_{00}}{\Gamma_M e_{11} - \Delta_e} \tag{2.2.5}$$

2.3 Acoplador Direccional

En la mayoría de diseños en electrónica, el acoplamiento electromagnético entre líneas de transmisión se considera indeseable porque interfiere con la señal que se busca transmitir. Una excepción es el caso de acopladores, donde el acoplamiento es un parámetro de diseño que permite medir la señal de una línea con un alto aislamiento, es decir, sin interrumpir su paso. Además se pueden diseñar un par de líneas acopladas que verifiquen que en uno de sus puertos la señal incidente se anula, generando un punto que permite medir la onda reflejada.

Las principales propiedades que caracterizan un acoplador son la directividad, el aislamiento, el acoplamiento y la pérdida de inserción [7]. La directividad mide la capacidad de separar las ondas que se propagan en distintas direcciones. El aislamiento, la potencia que llega al puerto desacoplado. El acoplamiento indica la proporción de potencia que se entrega al puerto acoplado. La pérdida de inserción muestra cuánta potencia (que se debería transmitir) se pierde por ser entregada a los puertos acoplados y aislados, y las pérdidas en los materiales (dieléctrico y resistividad de las pistas).

Un acoplador ideal es un dispositivo pasivo, que posee directividad y aislamiento infinitos.

Existen acopladores direccionales como el HP 778d [8] que posee una directividad mayor a 36dB y pérdidas por inserción menores a 0,6dB, pero tiene un ancho de banda que permite hacer mediciones entre 100 MHz y 2 GHz, y sus dimensiones físicas y peso hacen que su utilidad se limite a mediciones en laboratorio. Para soluciones integradas hay alternativas comerciales como el ZABDC20-322 de Minicircuits [9] pero su costo hace que el diseño no sea escalable.

2.3.1 Puente resistivo

Una solución compacta, económica y con un ancho de banda teórico infinito es el puente resistivo tipo Wheatstone. Como se puede observar en la figura 2.3 es una red simple de comparación de impedancias, formada por cuatro resistencias sobre las cuales se inyecta una señal de referencia en la parte superior. Una de las resistencias es variable (Zx), mientras que el resto son iguales entre sí. Se utiliza un voltímetro para medir la diferencia de potencial entre las ramas. Si la resistencia variable es de 50Ω, el puente queda balanceado y el voltímetro indica un cero. Si es menor a 50Ω, la rama izquierda presenta un voltaje menor a la derecha, mientras que para valores mayores ocurre lo opuesto. La tensión en el nodo de referencia V_A resulta:

$$V_A = V_{inc} \cdot \frac{Z_0}{Z_0 + Z_0} = \frac{V_{inc}}{2} \tag{2.3.1}$$

Mientras que la tensión en el nodo de medición V_B es:

$$V_B = V_{inc} \cdot \frac{Z_x}{Z_0 + Z_x} \tag{2.3.2}$$

Por lo tanto, la señal desbalanceada o tensión diferencial reflejada $v_{diff}(t)$ entre ambos nodos se define como:

$$v_{diff}(t) = v_B(t) - v_A(t) v_{diff}(t) = v_{inc}(t) \left[\frac{Z_x}{Z_0 + Z_x} - \frac{1}{2} \right] \tag{2.3.3a}$$

Reescribiendo la resta interna con común denominador $\frac{Z_x}{Z_x+Z_0} - \frac{1}{2} = \frac{1}{2}\left[\frac{Z_x - Z_0}{Z_x + Z_0}\right]$ entonces:

$$v_{diff}(t) = \frac{v_{inc}(t)}{2} \underbrace{\left[\frac{Z_x - Z_0}{Z_x + Z_0} \right]}_{\text{Coeficiente de reflexión } (\Gamma)} = \frac{v_{inc}(t)}{2} \cdot \Gamma \tag{2.3.4}$$

Si la señal incidente es sinusoidal es posible además caracterizar cargas reactivas midiendo el desfasaje de la señal Zx respecto de la incidente. Este es el principio que permite al puente comportarse como un acoplador direccional: la diferencia de potencial entre las ramas es función de la onda reflejada equivalente.

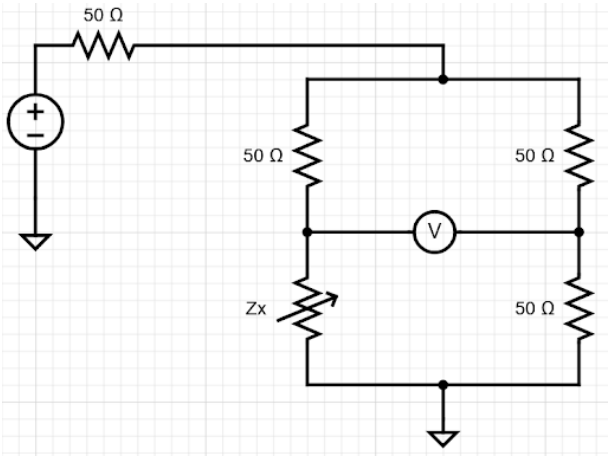


Figura 2.3: Esquema básico de un puente de Wheatstone

Para una aplicación de reflectometría, la magnitud de la impedancia de entrada al DUT haría las veces del Zx a caracterizar en este esquema simplificado. Esto, además de la capacidad de lograr directividad y ancho de banda elevados, lo vuelve una alternativa muy conveniente a la hora de implementar un VNA que funcione con buena sensibilidad en un rango amplio de frecuencias.

2.4 Buenas prácticas de diseño en RF

Al momento de rutear un PCB (*Printed Circuit Board* - Placa de Circuito Impreso) se deben tener en consideración algunas buenas prácticas que ayudan a mantener la integridad de las señales, la impedancia de las líneas y evitar la interferencia electromagnética, sobre todo cuando se trabaja en RF.

Se debe garantizar que las pistas cumplan con la impedancia requerida de 50Ω. Esto implica calcular el ancho de la línea usando las ecuaciones de diseño según el tipo de línea y las características de los materiales a utilizar (generalmente brindadas por el fabricante de PCB). Se pueden encontrar calculadoras online o integradas en los software de diseño con este fin. Se deben evitar ángulos rectos, que causan discontinuidades y aumentan la capacitancia parásita.

El plano de masa que se encuentra debajo de las pistas de RF debe ser continuo, para que la corriente de retorno pueda seguir el camino de menor inductancia. Por esta razón no se recomienda rutear en el plano inferior al de las rutas de RF, pero si se necesita hacerlo no se deben atravesar las regiones donde en la capa superior haya trazos de RF.

Se deben usar las técnicas *vía stitching* y *vía shielding*. La segunda se usa para reducir la interferencia electromagnética y el crosstalk [10], mientras que la primera "ata" los planos de masa de distintas capas, ayudando a mantener baja impedancia y caminos de retorno cortos. Además evita resonancias dentro de la placa.

Los capacitores de desacople se deben ubicar lo más cerca posible de los integrados, de forma de poder suministrar rápidamente los picos de corriente que demandan los integrados que la fuente de alimentación no puede entregar [11]. También deben utilizarse cuando se suministra alimentación a un trazo a través de vías.

Los trazos deben ser lo más cortos posible, para minimizar la atenuación de las señales y reflexiones indeseadas.

2.5 Heterodinación de señales

Las señales que se intentan medir en un VNA son de naturaleza analógica, por lo que para realizar el cálculo del parámetro de reflexión o S11 primero se debe poder medir su amplitud y fase. Existen integrados que pueden obtener la magnitud de una onda de RF como el HMC601 [12] pero se pierde información de la fase. Por eso se busca una forma de bajar la frecuencia de la señal para poder adquirirla de forma digital.

Un esquema heterodino se basa en la combinación de una señal de RF de interés con otra señal generada localmente por un oscilador local (OL o LO por sus siglas en inglés) desplazada en frecuencia un valor constante, llamado frecuencia intermedia (FI).

Cuando se multiplican idealmente en el dominio del tiempo dos señales sinusoidales $v_{RF}(t) = A_{RF}\cos(2\pi f_{RF}t)$ y $v_{OL}(t) = A_{LO}\cos(2\pi(f_{RF}+f_{IF})t)$ (suponiendo que tienen la misma fase inicial), a la salida existen componentes espectrales en la suma y diferencia de las frecuencias originales:

$$v_{mix}(t) = k \cdot v_{RF}(t) \cdot v_{LO}(t) \tag{2.5.1a}$$
$$v_{mix}(t) = kA_{RF}A_{LO}\cos(2\pi f_{RF}t)\cos(2\pi(f_{RF}+f_{IF})t) \tag{2.5.1b}$$
$$\tag{2.5.1c}$$

Reescribiendo con la identidad trigonométrica $\cos(\alpha)\cos(\beta) = \frac{1}{2}\left[\cos(\alpha-\beta)+\cos(\alpha+\beta)\right]$ entonces:

$$v_{mix}(t) = \frac{kA_{RF}A_{LO}}{2}\left[\underbrace{\cos\left(2\pi f_{FI}t\right)}_{\text{Componente Resta (FI)}} + \underbrace{\cos\left(2\pi(2f_{RF}+f_{FI})t\right)}_{\text{Componente Suma (Alta Frecuencia)}}\right] \tag{2.5.2a}$$

Mediante un filtrado se puede aislar la componente diferencia, logrando así trasladar la información de la señal de RF hacia la FI:

$$v_{\mathrm{IF}}(t) = \frac{kA_{\mathrm{RF}}A_{LO}}{2}\cos\left(2\pi f_{\mathrm{IF}}t\right) \tag{2.5.3}$$

Utilizando un mezclador doble se pueden heterodinar dos señales con un solo oscilador local, como muestra la figura 2.4 . De esta forma obtienen las señales incidente y reflejada, conservando su amplitud y fase, en una frecuencia de audio que facilite su adquisición y procesamiento.

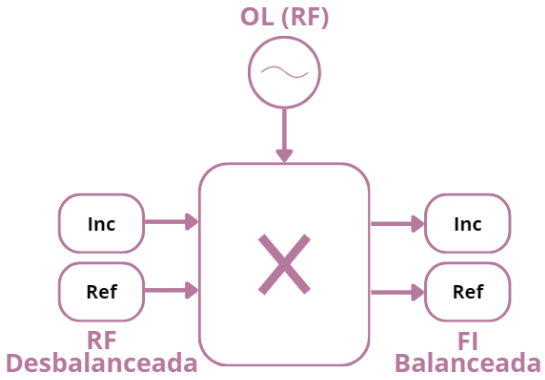


Figura 2.4: Heterodinación de señales

2.6 Procesamiento digital de señales

Para poder aplicar un procesamiento digital cualquier señal analógica continua en el tiempo debe ser muestreada, es decir convertirla en una secuencia de números de precisión finita. El proceso implica una pérdida de información de los intervalos intermedios, pero según el teorema de Nyquist-Shannon una señal de banda limitada puede reconstruirse inequívocamente a partir de sus muestras si la frecuencia de muestreo f_s cumple la condición $f_s \geq 2f_{\max}$, donde $f_{\max}$ representa la componente frecuencial de mayor valor de la señal [13].

Una vez que la señal está digitalizada, para poder extraer su amplitud y fase, sabiendo que se encuentra a una frecuencia de valor FI, se debe aplicar un filtrado digital y realizar la Transformada Rápida de Fourier (FFT - *Fast Fourier Transform*)

2.6.1 Filtros digitales

El objetivo principal de la etapa de filtrado es aislar la señal de FI eliminando el ruido de banda ancha, las interferencias de baja frecuencia y armónicos de orden superior.

La señal muestreada $x[n]$ pasa por un filtro pasabanda digital caracterizado por una respuesta al impulso $h[n]$. La operación se define mediante la convolución discreta:

$$y[n] = x[n] * h[n] = \sum_{k=0}^{M} h[k]\, x[n-k] \tag{2.6.1}$$

donde M representa el orden del filtro. Los coeficientes $h[k]$ se diseñan para configurar una banda de paso en torno a la frecuencia de FI.

Los filtros digitales se pueden clasificar en dos grandes familias según la naturaleza de su respuesta al impulso: filtros de respuesta finita (*Finite Impulse Response*, FIR) y filtros de respuesta infinita (*Infinite Impulse Response*, IIR).

Un filtro IIR se caracteriza por una respuesta al impulso de duración infinita, que se logra mediante una estructura recursiva con realimentación de las salidas anteriores. Su ecuación en diferencias se expresa como:

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k] + \sum_{k=1}^{N} a_k \, y[n-k] \tag{2.6.2}$$

La presencia de los coeficientes a_k introduce polos en el plano z fuera del origen. Si bien permiten lograr bandas de transición abruptas con órdenes bajos, presentan dos desventajas fundamentales: no garantizan fase lineal en toda la banda de paso y pueden ser inestables si sus polos migran fuera del círculo unitario debido a cuantización de precisión finita.

Por otro lado, un filtro FIR presenta una respuesta al impulso de duración finita N_{taps} (comúnmente denominada número de coeficientes o *taps*), siendo $h[n] = 0$ para $n < 0$ y $n \geq N_{\text{taps}}$. Su ecuación en diferencias es puramente no recursiva:

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k] = \sum_{k=0}^{N_{\text{taps}}-1} h[k] \, x[n-k] \tag{2.6.3}$$

donde $M = N_{\text{taps}} - 1$ representa el orden del filtro. Su transferencia $H(z)$ es causal, estable y realizable. Todos sus polos se ubican de manera trivial en el origen ($z = 0$), lo que garantiza estabilidad incondicional.

Si la secuencia de coeficientes $h[n]$ cumple la condición de simetría par alrededor de su centro ($h[n] = h[N_{\text{taps}} - 1 - n]$), la respuesta en frecuencia presenta la forma:

$$H(e^{j\omega}) = |H(e^{j\omega})| \, e^{-j\omega\tau_g} \tag{2.6.4}$$

donde el retardo de grupo τ_g es estrictamente constante para todas las frecuencias:

$$\tau_g = \frac{N_{\text{taps}} - 1}{2} \quad \text{[muestras]} \tag{2.6.5}$$

Un retardo de grupo constante asegura que el filtro no introduce distorsión de fase no lineal, preservando la diferencia de fase relativa entre el canal de referencia (V_{ref}) y el de medición (V_{med}).

2.6.1.1 Diseño de filtros FIR mediante el método del enventanado

La forma más simple de diseño de filtros FIR es el llamado método del enventanado (*window design method*) [14]. Este método consiste en truncar y desplazar la respuesta al impulso ideal de duración infinita, $h_d[n]$ (correspondiente a un filtro pasabanda ideal centrado en f_{FI}), multiplicándola punto a punto por una función ventana finita $w[n]$ de longitud N_{taps}:

$$h[n] = h_d[n] \cdot w[n], \quad 0 \leq n \leq N_{\text{taps}} - 1 \tag{2.6.6}$$

Una de las posibles ventanas a utilizar es la Hamming, definida como:

$$w[n] = 0{,}54 - 0{,}46 \cos\left(\frac{2\pi n}{N_{\text{taps}} - 1}\right), \quad 0 \leq n \leq N_{\text{taps}} - 1 \tag{2.6.7}$$

La multiplicación en el dominio del tiempo equivale a la convolución periódica de la respuesta en frecuencia ideal con el espectro de la ventana, $W(e^{j\omega})$. La ventana de Hamming tiene como características principales:

- Atenuación en banda de rechazo: Proporciona un nivel de lóbulo secundario pico de aproximadamente -41 dB y una atenuación mínima en la banda de rechazo de -53 dB.
- Suavizado del fenómeno de Gibbs: Elimina las oscilaciones de sobrepico en los bordes de la banda de paso típicas del truncamiento rectangular estricto, a cambio de un lóbulo central más ancho.

2.6.2 Transformada Rápida de Fourier

Para determinar la amplitud y fase de las señales de FI, las secuencias filtradas en el dominio del tiempo se transforman al dominio de la frecuencia mediante la Transformada Discreta de Fourier (DFT), calculada a través del algoritmo de la Transformada Rápida de Fourier (FFT). Para una secuencia de N muestras, la expresión de análisis viene dada por [13]:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, w[n]\, e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \ldots, N-1 \tag{2.6.8}$$

donde $w[n]$ representa la función de la ventana aplicada a la secuencia temporal.

Dado que la captura se realiza sobre una ventana de observación finita, si la duración del intervalo de adquisición no contiene un número entero exacto de ciclos de la señal, se produce el fenómeno de fuga espectral (*spectral leakage*). Este fenómeno dispersa la energía de la frecuencia fundamental hacia los bins frecuenciales adyacentes, degradando la precisión en la estimación de amplitud y fase. Para mitigar este efecto, se aplica una función de enventanado $w[n]$. La ventana *Flattop* se caracteriza por presentar un lóbulo principal ancho y plano; esta propiedad minimiza el error de estimación de amplitud, mejorando la exactitud en la medición de niveles de tensión.

Capítulo 3

Proyecto

3.1 Descripción general del producto

Como describe la sección 2 de la Especificación de Requerimientos (C) el sistema debe generar un barrido de frecuencias en el rango de 1,8 a 2,2 GHz. La señal se debe inyectar en el DUT y se debe medir la onda reflejada. Ambas ondas (incidente y reflejada) deben bajarse a una frecuencia intermedia para luego adquirirlas y digitalizarlas con el fin de realizar el procesamiento correspondiente. Un diagrama en bloques detallado del sistema se puede observar en la figura 3.1. Los datos a asumir, requerimientos y pautas de diseño se pueden encontrar en la Especificación de Requerimientos (C).

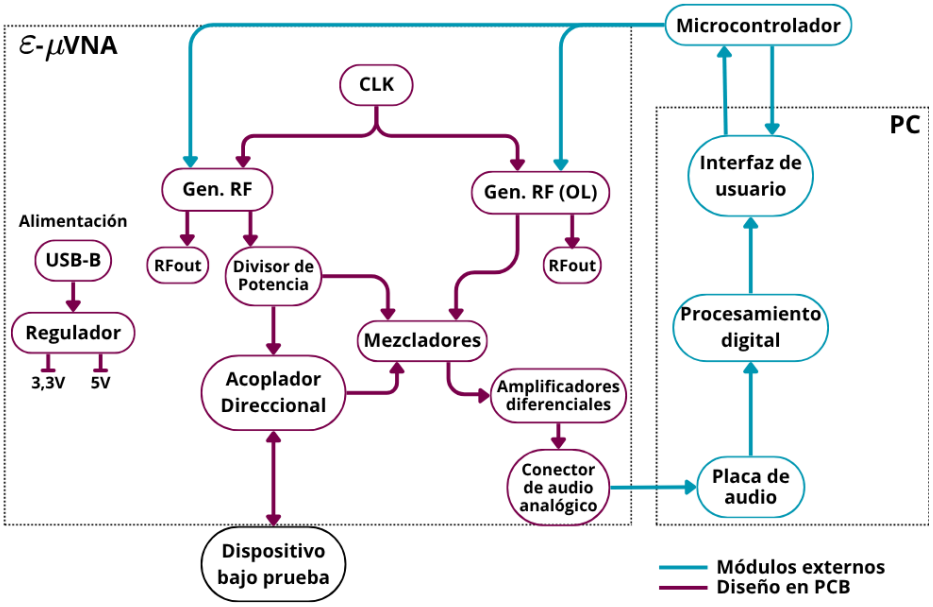


Figura 3.1: Diagrama en bloques del sistema

A continuación, se detalla la solución del hardware, software, firmware e interfaz de usuario.

3.2 Hardware

El hardware se diseñó en su totalidad en el software Altium Designer, con una licencia estudiantil gracias al programa Altium Student Lab [15]. Se creó una librería con los esquemáticos, footprints y modelos 3D de los componentes a utilizar. Se dividió el proyecto en esquemáticos

que representan los distintos bloques funcionales, que se pueden observar en el capítulo "Esquemáticos y códigos" (6.1). Se puede encontrar una descripción general de cada bloque en la Especificación Técnica (E).

3.2.1 Generadores de RF

Como explica la sección 2.1.2 de la Especificación Técnica (E) los generadores de RF se sintetizan con los circuitos integrados (IC - *Integrated Circuits*) ADF4351. Este es un sintetizador de banda ancha con VCO integrado, con frecuencia de salida configurable de 40 a 4400 MHz y potencia de salida máxima de 5dBm [16]. El hardware posee dos; uno incide directamente en el acoplador direccional, desviando una parte de su potencia hacia el mezclador para medir la señal incidente. El esquemático de su implementación se puede ver en 6.1.1. El otro se utiliza como oscilador local para el esquema heterodino (esquemático en 6.1.2). Es relevante mencionar que las salidas de RF no es una sinusoidal pura perfecta, sino que incluye componentes armónicos generados por el VCO y la etapa de salida. La hoja de datos aclara que el segundo armónico se encuentra entre -19 y -20 dBc por debajo de la potencia de la fundamental, mientras que el tercer armónico se encuentra entre -10 dBc y -13 dBc por debajo.

El diagrama en bloques del chip se puede observar en la figura 3.2:

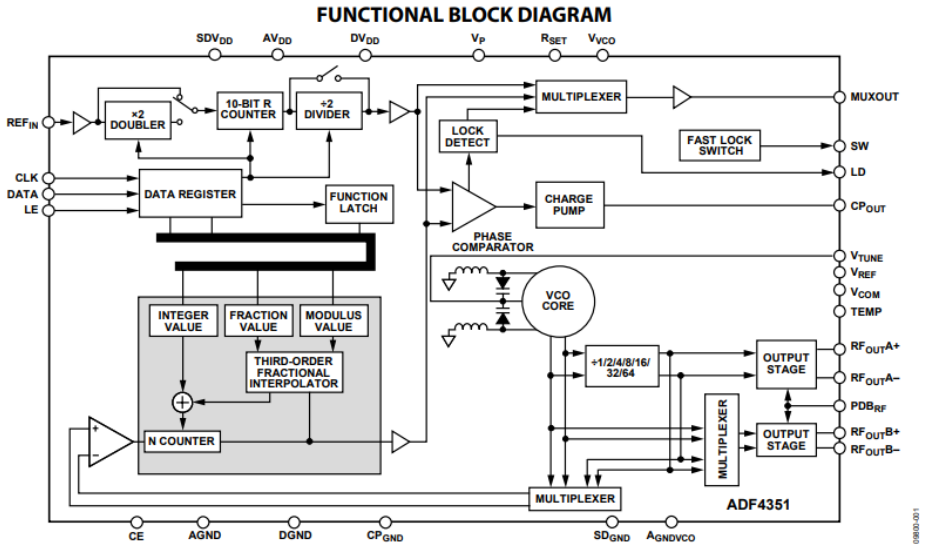


Figura 3.2: Diagrama en bloques del integrado ADF4351

Algunas consideraciones que se tuvieron en cuenta al momento de diseñar y rutear fueron:

- Tensión de alimentación (Vcc) de 3,3 V, ya que soporta de 3 a 3,6 V. Esta tensión debe ir a pines que se encuentran distribuidos en todos los lados del integrado, por lo que al estar ingresando con señales de RF se debe tener especial cuidado en el ruteo para mantener la integridad de las señales.

- El pad central expuesto del IC debe ser soldado a un pad GND (tierra) de al menos su mismo tamaño por razones térmicas, por indicación del datasheet [16]. Además, se sugiere colocar vías para mejorar el comportamiento térmico.

- La salida de RF es de colector abierto, por lo que para funcionar necesita una etapa de pull-up: la forma más sencilla (y de banda ancha) de hacerlo es con una resistencia de 50 Ω a V_{VCO}. De esta forma la salida del circuito es una potencia que equivale al valor registrado en el registro 4 (R4) (ver sección firmware 3.3). Por la misma razón, por usarse en modo desbalanceado, la salida de la rama que se deja sin usar debe terminarse en una

resistencia de 50 Ω. En este caso, se coloca una línea de transmisión hacia un conector SMA para mediciones auxiliares que en caso de no utilizarse, debe adaptarse. Se incluyó en el diseño un pad opcional para soldar una resistencia de 50 Ω una vez que no se necesiten los conectores.

- Es importante que la señal V_{TUNE}, una continua que es la entrada de control del VCO, esté libre de ruido, ya que determina la frecuencia de salida y es vital para el correcto enganche.
- La hoja de datos recomienda poner capacitores de desacople lo más cerca posible de los pines V_P, AV_{DD}, V_{VCO}, $TEMP$, V_{COM}, V_{REF}, DV_{DD}, y SDV_{DD}.
- Los pines que se utilizan para programar el integrado son una entrada de alta impedancia CMOS, con un límite máximo de 3,6 V. Para protegerlas por incompatibilidad de niveles lógicos, se utilizan atenuadores en cada una de las entradas (CLK, CE, LE, DATA) (6.1.7).
- Se coloca un LED en la salida de detección de enganche, para visualizar rápidamente si el integrado está enganchado.

3.2.1.1 Filtro externo de lazo de control

El filtro de lazo se diseña en la aplicación del fabricante, Analog Devices, ADISIMPLL 5.9 [17]. Se busca que cumpla con un margen de fase mínimo de 45° para preservar la estabilidad del sistema, y que el ancho de banda sea lo mayor posible para disminuir el tiempo de enganche, a costa de un incremento en el ruido que ingresa al lazo que proviene de la referencia, del detector de fase y la bomba de carga.

El esquema final se puede observar en el entorno del software en la figura 3.3.

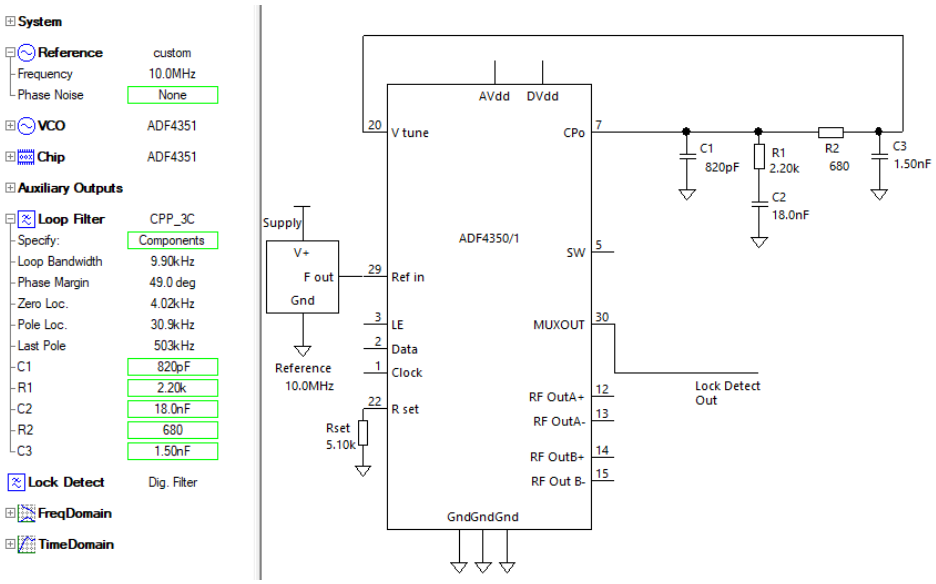


Figura 3.3: Diseño del filtro de lazo externo

La respuesta en frecuencia de todo el sistema (ADF4351 con el filtro diseñado), simulada para 2 GHz, se muestra en la figura 3.4

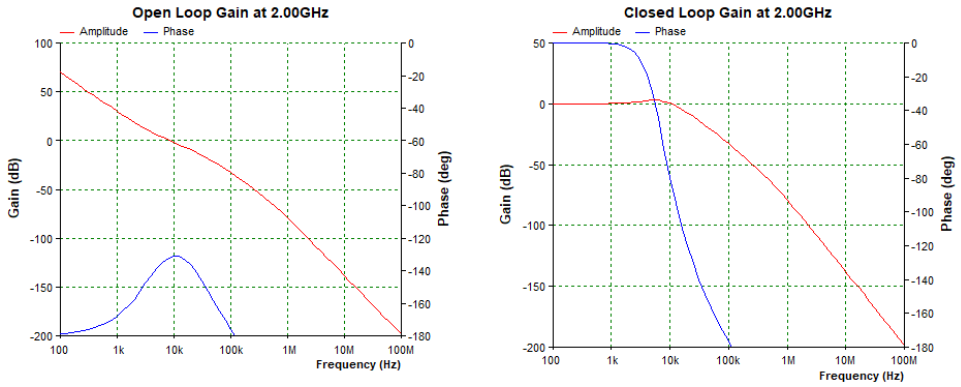


Figura 3.4: Respuesta en frecuencia de los generadores

3.2.2 Oscilador

La referencia para los generadores de RF se compone de un oscilador a cristal ECS-2200BX-100 [18] que proporciona una salida cuadrada y un buffer con salida open drain 74LVC2G07W6-7 [19]. Este permite no solo aumentar el fan-out (dividiendo la señal en dos caminos sin degradarla) sino realizar un reacondicionamiento. De esta manera se reduce el crosstalk (aislando las entradas de los generadores entre sí) y el ringing (oscilaciones de reflexión), garantizando flancos más limpios y rápidos. Se colocan dos resistencias de 47 Ω para absorber las ondas reflejadas que existan en la línea debido a que la entrada de referencia de los ADF es de alta impedancia y limitar la corriente de conmutación (ayudando a reducir el ruido de alta frecuencia). El esquemático se encuentra en la sección 6.1.4.

3.2.3 Mezclador

El mezclador se implementa con el IC ADL5802, un mezclador activo de alta linealidad, de dos canales, doble balanceado, con buffers de OL integrados para proveer conversión de frecuencia con alto rango dinámico. Su esquemático se puede observar en la sección 6.1.3.

Su función principal es realizar la heterodinación de las señales de radiofrecuencia. Se aplica la señal proveniente del DUT a la entrada de RF, y la del ADF4351 que actúa de OL a la entrada correspondiente.

El integrado se alimenta con una tensión continua de 5 V. Sus salidas en FI son de colector abierto con una impedancia diferencial de 200 Ω, por lo que se colocan resistencias de pull-up de 50 Ω conectadas a la línea de 5 V para su correcta polarización.

Las salidas diferenciales del integrado permiten obtener las componentes correspondientes a las señales incidente y reflejada, identificadas como IF_INC+, IF_INC-, IF_MED+ e IF_MED- en el esquemático. A la salida de cada canal se integran filtros pasabajo RC simples, calculados para atenuar la componente suma de alta frecuencia ($2f_{\mathrm{RF}} + f_{\mathrm{FI}}$) y dejar pasar únicamente las señales diferenciales de baja frecuencia correspondientes a la FI. Estas señales se destinan a las etapas posteriores de adquisición y procesamiento.

Se disponen capacitores de desacople en la alimentación y un divisor resistivo en la entrada VSET, que es el control de polarización para alto IP3 (lo que mejora la linealidad de entrada). Se aplica entre 4 y 5 V para mejorar la performance de IP3, y entre 2 y 3 V para mejorar la figura de ruido. Actualmente el valor está seteado en aproximadamente 4,1V.

3.2.4 Acoplador direccional

Basado en el puente de Wheatstone explicado en el marco teórico, permite dirigir la señal generada hacia el sensor, mientras que una parte de la señal reflejada por la carga se obtiene

mediante el puerto acoplado. De esta manera, es posible acceder a ambas señales sin necesidad de interrumpir directamente el camino principal de transmisión.

El puente diseñado, como se muestra en el esquemático (6.1.5), agrega dos transformadores de RF TC1-1-13M+ [20] en cascada, que permiten medir la señal reflejada de manera desbalanceada y filtrar el ruido de modo común que provocaría usar solo uno. Las resistencias de 68Ω logran que la impedancia vista desde el puerto de salida (J1) sea 50Ω.

En la figura 3.5 se presentan las curvas de respuesta en frecuencia simuladas para los cuatro parámetros característicos de un acoplador direccional, en el rango de 0,1 a 3 GHz. Como se puede observar, el acoplamiento tiene una respuesta plana en toda la banda, esperable para un puente resistivo, manteniéndose constante alrededor de los 15 dB. La pérdida de inserción tiene valores razonables teniendo en cuenta que las resistencias disipan potencia, oscilando cerca de los 10,5 dB. El aislamiento tiene su máximo nivel de atenuación en la banda de interés (entre 1,8 y 2,2 GHz) pero mantiene un buen nivel en toda la banda, por encima de los 48 dB. La directividad tiene la misma forma, con un pico elevado de 37,3 dB en torno a 1,9 GHz pero manteniéndose sobre los 36,5 dB para las frecuencias de interés. Esto garantiza una muy buena separación entre las ondas incidente y reflejada, reduciendo errores sistemáticos. Es importante aclarar que las simulaciones se realizaron en el software uSimmics v 5.9 [21] y las resistencias utilizadas para simular fueron ideales, por lo que se espera un rendimiento un poco menor en la implementación real, sobre todo por la tolerancia de las resistencias a altas frecuencias. Los transformadores de RF se caracterizaron con datos de sus parámetros S brindados por el fabricante.

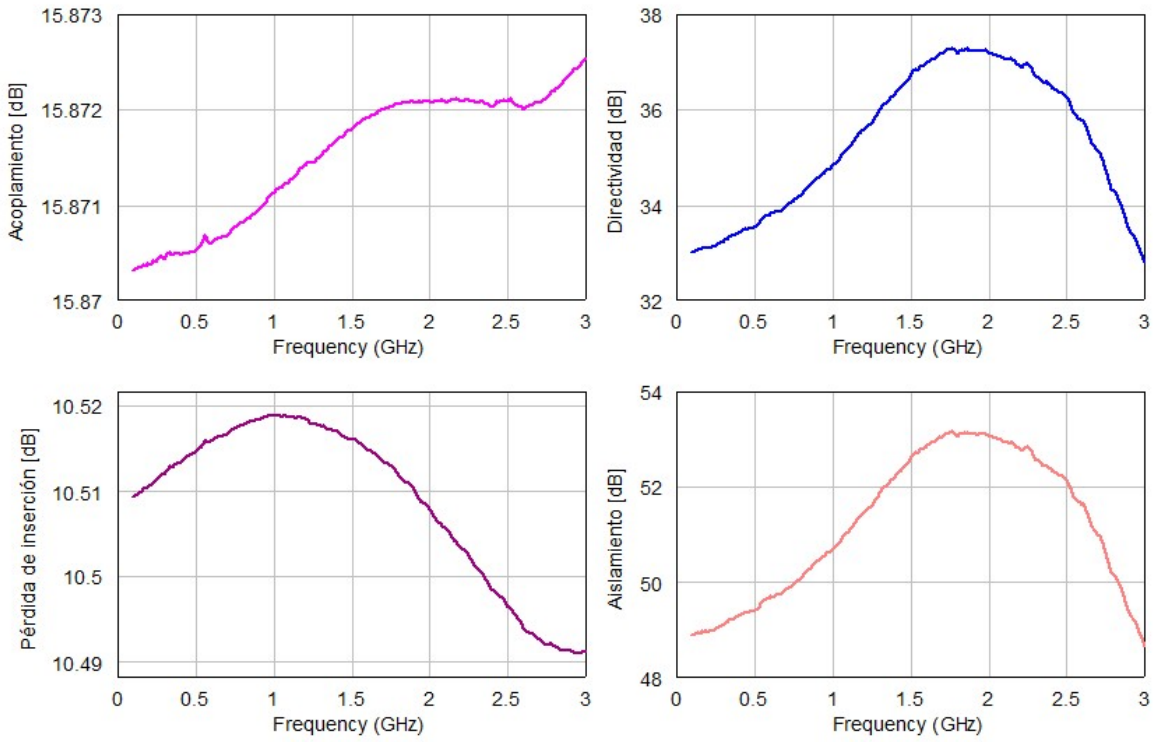


Figura 3.5: Parámetros simulados del puente resistivo

3.2.5 Amplificadores diferenciales

Al tener salidas diferenciales y de muy poca amplitud, se debe agregar una etapa de acondicionamiento que permita obtener una señal desbalanceada y apta para ser procesada por la placa de sonido de la PC. Esta etapa utiliza amplificadores operacionales MCP6023 de Microchip Technology [22] de alimentación simple (rail-to-rail). Se configuran como restadores, permitiendo referir las señales a masa y proveer la ganancia necesaria para trabajar dentro del

rango dinámico de la placa de sonido. Su esquemático se puede observar en 6.1.6. Aunque allí se usa una ganancia unitaria, en el ensamblado se midió la salida del mezclador y se calculó una ganancia de 8,33 veces para la señal incidente y 55,55 veces para la señal reflejada. De esta forma se aprovecha mejor el rango dinámico de la entrada de la placa de sonido, cuya entrada satura alrededor de 2 Vpp según mediciones realizadas en el laboratorio. Las resistencias utilizadas fueron de 180 Ω para R53 y R55, y 1200 Ω para R46 y R48.

Se agregaron capacitores de bloqueo de continua, y una tensión de referencia de $V_{CC}/2$ obtenida mediante un divisor resistivo. De este modo, la señal alterna excursiona alrededor de ese punto de operación y puede ser amplificada sin recortes, sin necesidad de una fuente negativa.

3.2.6 Alimentación y conectores

Para alimentar la placa se dispone de una selección (a través de un jumper) de pines para utilizar una fuente de laboratorio o un USB-B para conectarlo a alguna salida de PC o cargador de 5V. Se agrega un regulador de tensión UA78M33CDCYR [23] de Texas Instruments que provee los 3,3V necesarios para alimentar a los generadores de RF y oscilador.

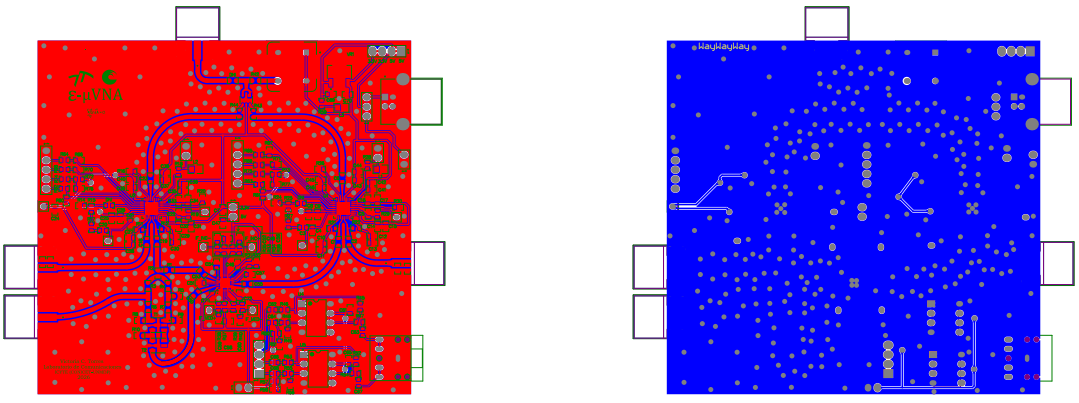
Los conectores que posee son del tipo SMA para mediciones de RF, conexión con el DUT y entrada de referencia auxiliar; conector de audio analógico para llevar las señales a la placa de sonido de la PC por medio de la entrada line-in, y tiras de pines que por medio de cables puente se conectan con el Arduino para escribir los registros de los generadores de RF. Además se ubican pares de pines para medir la detección de enganche en cada ADF4351.

3.2.7 Microcontrolador

Como se explica en la sección 2.3 de la Especificación Técnica (E), el microcontrolador se implementa utilizando una placa Arduino Uno que se conecta por medio de cables puente con los pines de control de los ADF4351: CLK, CE, LE y DATA. Actúa como un esclavo de ejecución que recibe las órdenes de barrido desde la PC a través de una interfaz serie. Opera a una velocidad de 115200 baudios.

3.2.8 Implementación en PCB

En principio se planteó un modelo de 2 capas, que se puede observar en las figuras 3.6.

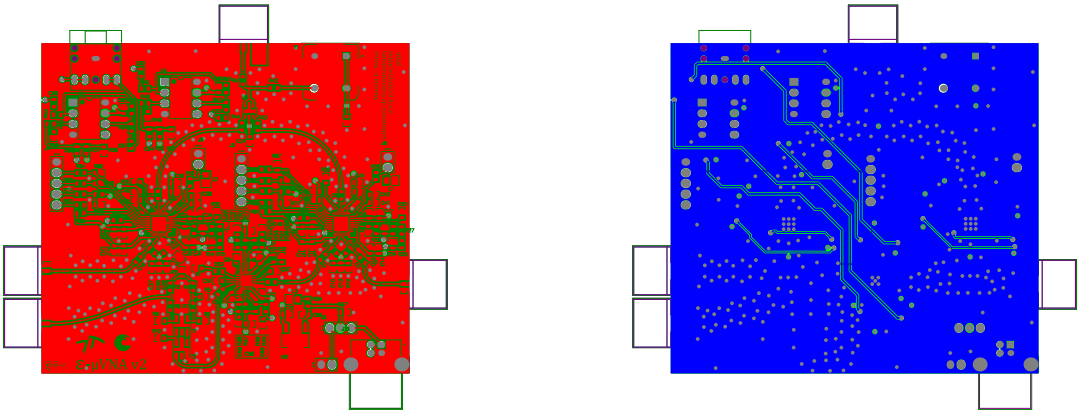


(a) Frente (b) Dorso

Figura 3.6: $\epsilon - \mu VNA$ versión 1, frente

Tiene FR4 como dieléctrico de 0,73 mm de espesor, con un espesor total de 0,9 mm, y un tamaño de 10 x 9,5 cm^2. Sin embargo, el ruteo del pin V_{TUNE} de los ADF4351 y la alimentación en la parte central de la placa, donde había un sector rodeado de pistas de RF, imponía el uso de

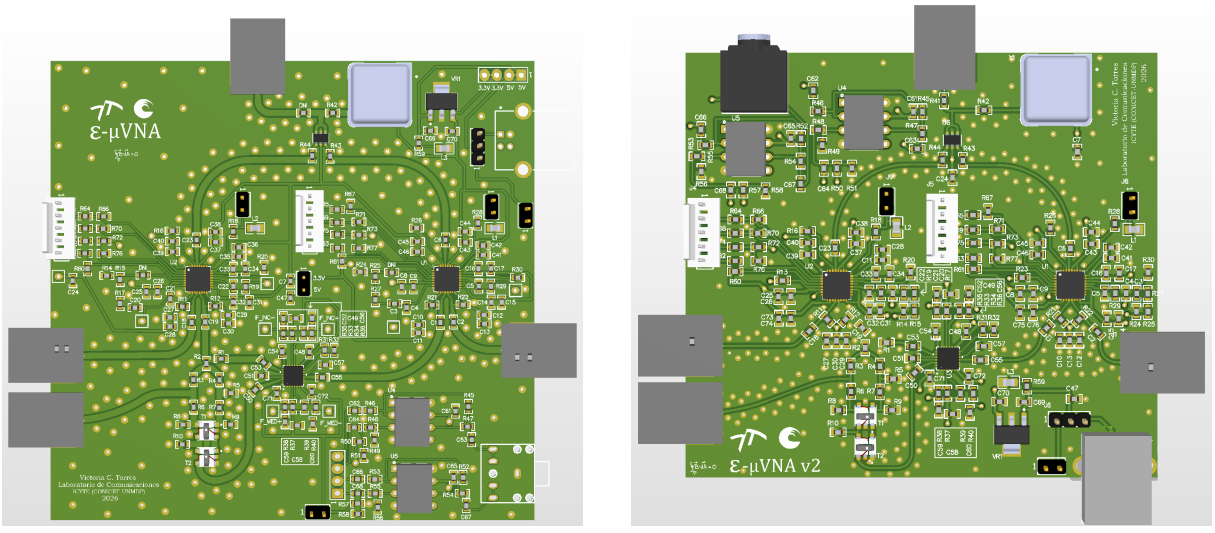
varios puentes. Se implementaron con postes y cables de arduino, pero no es la manera óptima porque son sensibles a interferencias. Por este motivo, se decidió hacer un modelo de 4 capas, que permitiera tener un plano de masa continuo bajo las pistas de RF y poder rutear usando la capa inferior, además de tener una tercera capa para el manejo de la alimentación. A este modelo se lo llamó $\epsilon - \mu VNA$ v2, tiene un tamaño de 8,7 x 7,8 cm^2 y se lo puede observar en la figura 3.7. Incorpora además algunas modificaciones en cuanto al ruteo de pines y posición de los transformadores de RF, debido a que no coinciden los pines descriptos en la hoja de datos con el integrado real.



(a) Frente (b) Dorso

Figura 3.7: $\epsilon - \mu VNA$ versión 2

Se pueden observar ambos modelos en 3D en la figura 3.8. La versión 2 logró una placa más compacta, con capas adicionales que facilitan el ruteo, reducen las interferencias y mantienen la integridad de las señales, a cambio de una fabricación más compleja y por lo tanto más costosa, que reduce la cantidad de posibles proveedores porque deben ser especializados.



(a) Versión 1 (b) Versión 2

Figura 3.8: $\epsilon - \mu VNA$ modelos 3D

3.3 Firmware

El firmware se programó en lenguaje C++ en el entorno Arduino IDE. Se puede leer en la sección "códigos" (6.2). Está desarrollado para un microcontrolador Arduino UNO y se encarga del control, configuración dinámica y sincronización de los sintetizadores ADF4351. Un diagrama de flujo se puede ver en la figura 3.9.

Al ser inicializado en la función de arranque (*setup*), configura los pines de control, inicializa el bus SPI y transmite la cadena "Listo" para confirmar que el sistema está disponible.

Al recibir una frecuencia f, el algoritmo configura el primer sintetizador (ADF1) exactamente en f (canal de RF) y el segundo sintetizador (ADF2) en $f + 0,010$ MHz (canal LO), introduciendo de forma automática el desplazamiento constante para la FI. Al finalizar la reconfiguración de ambos integrados, responde a la PC con la cadena "OK".

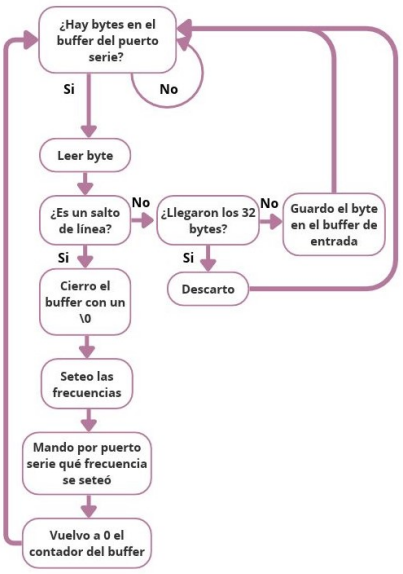


Figura 3.9: Diagrama de flujo general del firmware.

3.3.1 Cálculo de registros

Para cada valor de frecuencia solicitado, la función "calcularRegistros" determina matemáticamente el contenido de los 6 registros de 32 bits (R0 a R5) de cada chip ADF4351 según las especificaciones del fabricante:

- Divisor de Salida de RF (RF Divider): Selecciona el divisor adecuado (1, 2, 4, 8 o 16) según el rango de frecuencia de trabajo (desde menos de 275 MHz hasta más de 2200 MHz) para garantizar que el VCO interno funcione en su rango de 2.2 GHz a 4.4 GHz.
- Frecuencia del Detector de Fase (f_{PFD}): A partir del oscilador de referencia externo de 10 MHz, aplica la división previa por 2 (REF_DIV2 = true) y el contador R (R_COUNTER = 10), fijando una frecuencia de comparación en el PFD de 500 kHz ($f_{\text{PFD}} = 0.5$ MHz).
- Síntesis N-Fraccionaria: Divide la frecuencia del VCO por f_{PFD} para calcular la parte entera (INT) y la parte fraccionaria (FRAC) considerando un módulo fijo MOD = 1000.
- Configuración de Registros Internos:
  - R0: Almacena los bits correspondientes a 'INT' y 'FRAC'.
  - R1: Configura el *prescaler* en 8/9 y fija el módulo ('MOD') y la fase ('PHASE').
  - R2: Establece la polaridad positiva del detector de fase, habilita los divisores de referencia y fija la corriente de la bomba de carga (*charge pump*) en 5.00 mA.
  - R3: Habilita el modo de enganche rápido (*fast lock*).
  - R4: Selecciona el divisor de RF, habilita la salida principal, fija el reloj de selección de banda (*Band Select Divider*) y configura la potencia de salida en +5 dBm.
  - R5: Selecciona la operación del pin de enganche (*Lock Detect*). El resto de los bits son un registro de control fijado según recomendación de la hoja de datos.

3.3.2 Transmisión SPI y secuencia de sincronización

El envío de los registros hacia los integrados se realiza mediante el bus SPI de hardware y líneas de control independientes. Utiliza pines dedicados para la habilitación Latch Enable (LE) y Chip Enable (CE). Cada registro se envía dividiendo los 32 bits en 4 bytes (MSB primero) sobre la interfaz SPI, generando un pulso positivo de 2 microsegundos en la línea 'LE' para almacenar los datos en el integrado. Los registros se escriben en orden inverso, desde R5 hasta R0, debido a que la transferencia del registro R0 actúa como el disparador (*trigger*) que actualiza efectivamente la frecuencia del VCO.

Para mejorar la coherencia y estabilidad de fase en el canal del Oscilador Local (ADF2), se aplica un procedimiento en dos etapas: se escriben primero los registros R5 a R1, se genera un retardo intencional de 10 milisegundos para permitir la estabilización parcial del lazo, y finalmente se escribe el registro R0 para cerrar la sintonía.

3.4 Software

Como se explica en la sección 4 de la Especificación Técnica E el software se programa en lenguaje Python en el entorno Visual Studio Code. El código se encuentra en la sección 6.3. Un diagrama de bloques general se puede ver en la figura 3.10. Las funciones principales que cumple son:

- Gestionar el barrido en frecuencia;
- Controlar la adquisición de audio;
- Ejecutar el procesamiento digital de señales en tiempo real;
- Implementar el modelo de calibración SOL de un puerto para obtener el coeficiente de reflexión (Γ) del DUT;
- Mostrar resultados.

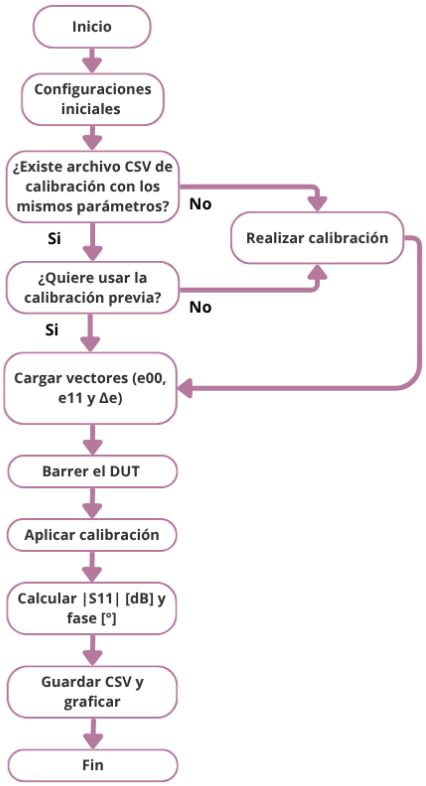


Figura 3.10: Diagrama de flujo general del software.

Las configuraciones iniciales son: Puerto COM, baudrate, frecuencias de inicio, fin y paso, offset (es decir, frecuencia de audio a utilizar), frecuencia de muestreo para la adquisición, duración de la captura y su descarte inicial debido al transitorio, los nombres de los archivos de salida, la frecuencia central del filtro FIR (que debe coincidir con la frecuencia de audio), su ancho de banda, numtaps y ventana a utilizar.

La función principal (*main*) genera el vector de frecuencias a partir de las frecuencias de inicio, fin y paso. Luego diseña el filtro pasabanda. Chequea en el directorio si hay algún archivo de calibración cuyo nombre, que está predefinido con las configuraciones de frecuencia coincida con las ingresadas por el usuario. En caso afirmativo, pregunta si se desea utilizar o se prefiere realizar una nueva calibración. Dependiendo de la respuesta del usuario por terminal, entra al bucle de calibración o carga los parámetros desde el archivo. Procede a realizar la conexión con el Arduino. Si se debe calibrar, realiza tres barridos para obtener los valores de Γ_{MED} y completar la matriz de calibración. Estos barridos corresponden a cortocircuito, circuito abierto y carga,

en ese orden. Una vez que se hacen los barridos calcula los errores y se guardan en el archivo CSV (archivo de texto plano donde los valores están separados por comas). Realiza el barrido del DUT y cierra el puerto serie. Aplica la corrección, calcula la fase en grados y el módulo en dB y se guardan en vectores separados. Crea un archivo CSV con estos datos y muestra los resultados en gráficos de módulo y fase en función de la frecuencia.

3.4.1 Flujo de barrido

La función "barrido" se encarga de enviar los datos de la frecuencia a generar, recibir la señal analógica y guardar los resultados en un vector. Su diagrama de flujo se puede ver en la figura 3.11.

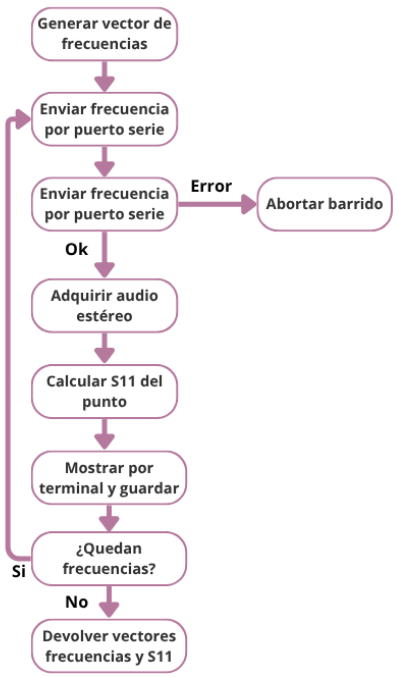


Figura 3.11: Diagrama de flujo de la función barrido.

Los parámetros que recibe son: el objeto de conexión serial con el Arduino (que da acceso al buffer de transmisión y recepción); el vector de frecuencias a medir, los coeficientes del filtro FIR, un nombre para la medición (para identificar si es un barrido de calibración o de medición), y los errores de calibración si fuese necesario. Devuelve un vector de S11 complejos (formato binario).

Consiste en un bucle *for* que recorre el vector de frecuencias, seteando cada una en el Arduino con la función "write" de la librería "serial" para luego leer su respuesta con "readline". Si esta comienza en "error" aborta el barrido. De otra forma, captura el audio usando la librería "sounddevice". Para eso calcula el número de muestras a adquirir como la multiplicación de la frecuencia de muestreo y la duración de la captura. Graba los dos canales, de tipo "float64" con la función "rec". Descarta la primera parte de la grabación para evitar incluir en el cálculo los transitorios, y guarda dos vectores: "canal_ref" y "canal_med". Por último, mide S11 y la frecuencia en la que se encontró el batido. Para ir mostrando los valores calibrados por terminal a medida que se hace el barrido, realiza la calibración por cada punto, calcula el S11 en módulo y fase y las imprime.

3.4.2 Estimación del S11

La función que realiza el cálculo del S11 tiene como parámetros de entrada los vectores de muestras, la frecuencia de muestreo y los coeficientes del filtro. Un diagrama de flujo simplificado se puede observar en la figura 3.12.

Lo primero que hace es aplicar el filtro a cada canal usando la función "lfilter" de scipy.signal. Luego calcula la FFT con la función "fft.fft()" de NumPy. Utiliza una ventana que setea el usuario, que por default es la FlatTop, y que crea la función "get_window" de scipy.signal.windows. Descarta la mitad negativa del espectro de frecuencias.

Luego busca el índice del bin frecuencial de mayor amplitud (en el canal de referencia) con la función "searchsorted" de NumPy dentro de un rango frecuencial acotado alrededor de f_{FI}, en la banda de paso del filtro. La ecuación 3.4.1 representa la búsqueda del índice, donde $X_{\mathrm{ref}}[k]$ es el valor complejo del espectro en el bin discreto de frecuencia k, y arg max el operador "argumento del máximo", que devuelve la posición o índice en el cual se alcanza la amplitud máxima.

$$k_{\mathrm{pico}} = \arg \max_{k \in \mathrm{Banda}} |X_{\mathrm{ref}}[k]| \tag{3.4.1}$$

Después crea un número infinitesimal basado en el pico del espectro para sumárselo al denominador del cálculo del S11, por si el valor de magnitud en la referencia es cero. Por último, calcula S11 como el conjugado de la división de la medición y la reflejada, usando para cada uno el bin donde se encontró el máximo de magnitud.

$$S_{11,\text{raw}} = \frac{V_{\text{med}}[k_{\text{pico}}]}{V_{\text{ref}}[k_{\text{pico}}]} \tag{3.4.2}$$

Este número se almacena como un tipo complex128, un tipo de dato numérico que representa números complejos de doble precisión. Este valor no calibrado sirve como la entrada directa para el posterior algoritmo de corrección de errores vectoriales mediante el modelo de tres términos (SOL).

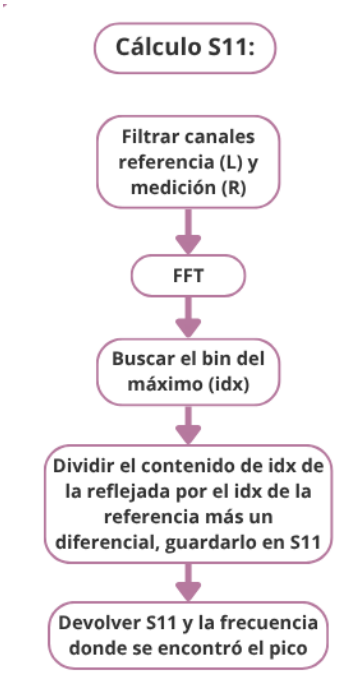


Figura 3.12: Diagrama de flujo de la función que calcula S11.

3.4.3 Calibración

El software guía al usuario para realizar tres barridos completos utilizando las cargas del método SOL. Almacena en tres vectores (GM_CC, GM_CA y GM_50) las mediciones complejas crudas del cortocircuito, circuito abierto y carga de 50 Ω respectivamente. Luego, la función "calculo_errores" procesa los datos punto a punto para cada frecuencia del barrido:

- Arma un sistema de ecuaciones lineales $A \cdot x = B$ con coeficientes complejos, donde A es la matriz cuya fila 1 corresponde a short: $[1, -\text{GM_CC}[i], 1]$; fila 2 a load: 1; y fila 3 a open: $[1, \text{GM_CA}[i], -1]$. El vector B corresponde a $[\text{GM_CC}[i], \text{GM_50}[i], \text{GM_CA}[i]]$.
- Resuelve el sistema mediante la función "linalg.solve(A,B)" de NumPy, que permite obtener las tres incógnitas por cada punto en frecuencia.

Los tres vectores de error se guardan en un archivo CSV, separando las partes reales e imaginarias de cada término con notación científica de 10 dígitos decimales junto con la frecuencia en MHz.

Cuando se ejecuta el barrido, el software aplica la función de corrección inversa sobre cada punto medido según la ecuación 2.2.5. Antes de hacerlo, se verifica que el denominador no sea nulo. Si la magnitud es menor a 10^{-15}, se reemplaza por un valor insignificante $(10^{-15} + j10^{-15})$ para evitar errores de división por cero.

3.5 Interfaz de usuario

La interfaz de usuario se programó con PyQt6, mediante un prompt de la inteligencia artificial (IA) Claude, modelo Sonnet 5 Esfuerzo Medio. Se alimentó del software original, que tenía la lógica de medición funcionando, pero que corría secuencialmente y se comunicaba con el usuario a través de la consola.

El prompt inicial fue: "Quiero hacer una interfaz de usuario sencilla que me permita comunicarme con este script. que se pueda elegir frecuencias y paso, visualizar modulo y fase de S11 y abaco de smith, que permita hacer calibración. Qué plataforma/lenguaje recomendas, y cuanto tiempo tardaría en ponerla en funcionamiento?". La IA recomendó Python + PyQt6/PySide6, con matplotlib embebido para los gráficos. Dio las razones (reutilizar funciones, permite actualizar el plot en vivo, aprovecha librerías de Python para graficar) y explicó otras alternativas y

por qué las descartó. Preguntó si hacía un esqueleto de la app con ventana principal, campos de frecuencia, threading de barrido y gráficos, y se le dio el permiso. Creó dos archivos, un "core" que contiene la misma lógica de cálculo que el código original pero reorganizada en funciones independientes (comunicación serial con Arduino, captura de audio, procesamiento de señal, detección del tono de batido y cálculo de S11 y álgebra de calibración) y un "GUI" (Graphical User Interface) que contiene la parte visual: ventana, botones, campos y gráficos. Este llama a las funciones implementadas en el core.

Se agregó la biblioteca PyQt6 para usar ventanas, botones, campos e hilos de ejecución. Se creó una clase (SweepWorker(QThread)) que se encarga de correr el barrido en segundo plano y avisar a la ventana qué se midió para que actualice los gráficos y la barra de progreso sin congelarse entre c ada medición.

La IA generó además el diagrama de flujo que se observa en la figura 3.13. Los códigos se encuentran en el capítulo "Esquemáticos y código" (6.4.1).

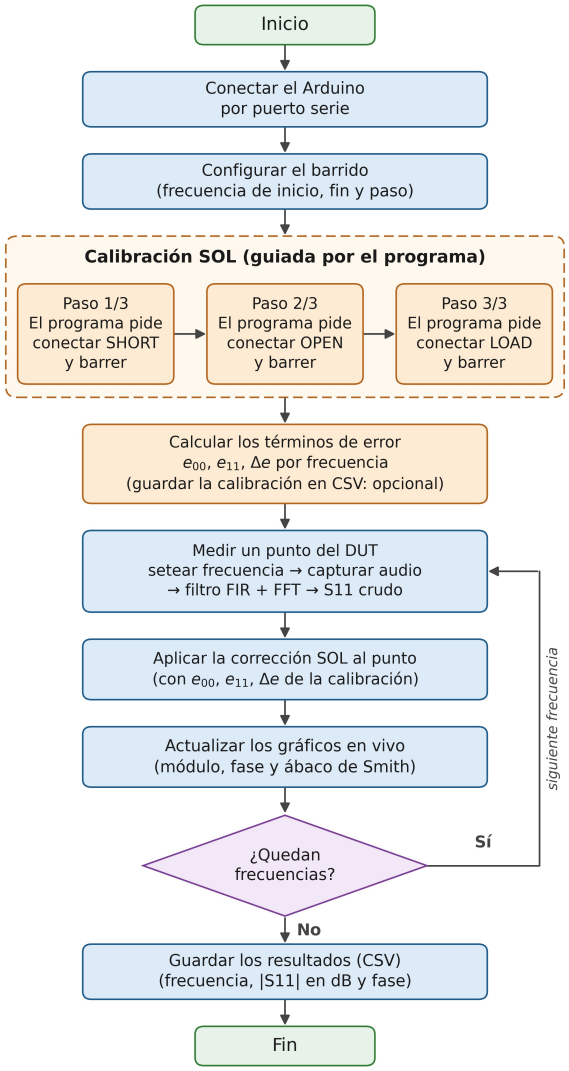


Figura 3.13: Diagrama de flujo de la interfaz de usuario

Capítulo 4

Resultados

4.1 Ensamblado de la placa final

Al terminar el diseño del PCB, se consultaron con proveedores nacionales para la impresión de la placa. Ante la falta de respuesta, se optó por un proveedor internacional, PCBWay [24]. Todo el proceso, desde el envío de los archivos gerber, revisión y cambios, hasta recibir el paquete, llevó aproximadamente 15 días. Las placas recibidas son las que muestra la figura 4.1.

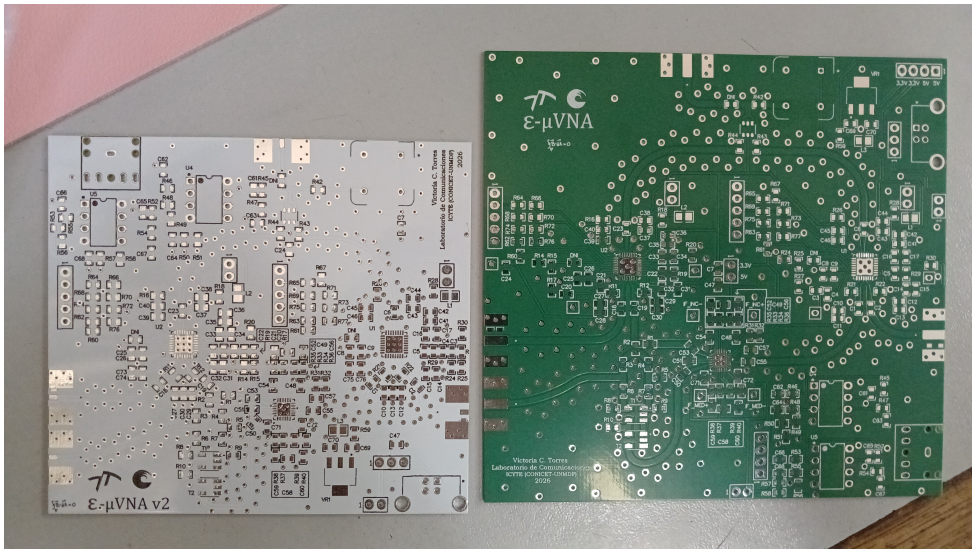


Figura 4.1: Placas sin soldar

Los componentes se soldaron a mano, siendo los primeros los tipo QFN (encapsulados planos cuádruples sin terminales - *Quad Flat No-Lead*). Se puede observar en la figura 4.2 que para hacerlo se utilizó una estación de precalentamiento y un soldador de aire.



Figura 4.2: Soldado de componentes QFN

Luego se soldaron el resto de componentes, en orden de menor a mayor altura. La figura 4.3 exhibe el banco de soldado. La placa final se muestra en la figura 4.5. Es interesante destacar que debido al plano de masa continuo en la segunda capa, la placa posee muy buenos niveles de disipación, lo que dificultó considerablemente el proceso de soldado. Este inconveniente se resolvió mediante la combinación del uso de precalentadora, pistola de aire y soldador de lápiz simultáneamente y generosas cantidades de flux, como ilustra la figura 4.4.

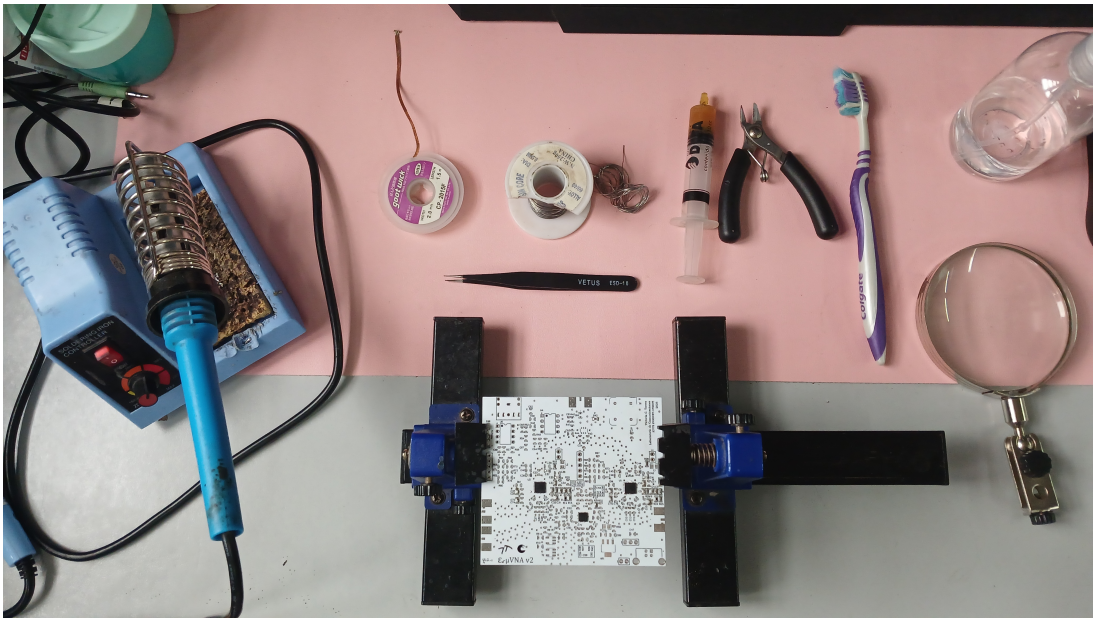


Figura 4.3: Banco de soldado

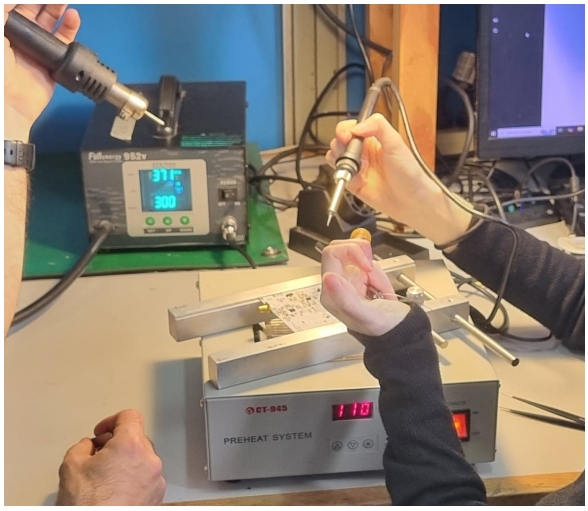


Figura 4.4: Proceso de soldado.

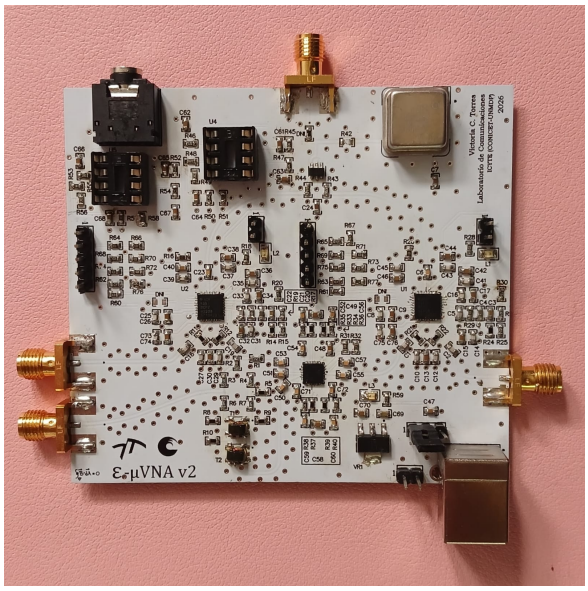


Figura 4.5: ϵ-μVNA v2 final.

4.2 Ficha técnica

Parámetro	Valor	Comentarios
Rango de frecuencia de trabajo	1,8 a 2,2 GHz	Ampliable por software
Resolución (paso mínimo) en RF	250 Hz	En la banda de trabajo. Según la hoja de datos del AF4351, el mínimo con este hardware sería 31,25 Hz pero no se han hecho pruebas.
Impedancia característica	50Ω	-
Directividad del acoplador resistivo	>36,5 dB	Para el rango de frecuencia de trabajo
Voltaje de alimentación	5V	Vía USB-B o fuente externa
Corriente de alimentación	350 mA	-
Frecuencia de muestreo	44,1 kHz	Configurable por software
Resolución del ADC (placa de sonido)	16 bits	Entrada Line-In de la PC
Baud rate (microcontrolador)	115200 baudios	Configurable

4.3 Pruebas de integración

Para verificar el correcto funcionamiento de la placa, la primera prueba unitaria realizada fue medir la entrada y salida del regulador de tensión. Se verificó que hubieran 5 V y 3,3 V respectivamente.

Se midió la frecuencia del cristal con un frecuencímetro Optoelectronics Handi-Counter 2810 [25], alimentando el oscilador con fuente de alimentación regulable Kenwood PA36-3A [26]. Como evidencia la figura 4.6, el cristal es de 10 MHz.



Figura 4.6: Banco de medición de la frecuencia del oscilador a cristal.

Luego se programó la generación de una única señal para cada generador de RF y así comprobar el enganche y el nivel de potencia de salida. Para esta medición se utilizaron las salidas auxiliares con conectores SMA. El banco de medición (véase la figura 4.7) consistió en la fuente regulable para alimentar la placa, una PC para programar el Arduino, un analizador de espectros Rigol RSA3030N [27], y un atenuador HP 8496B [28], que se utilizó para proteger la entrada del analizador.

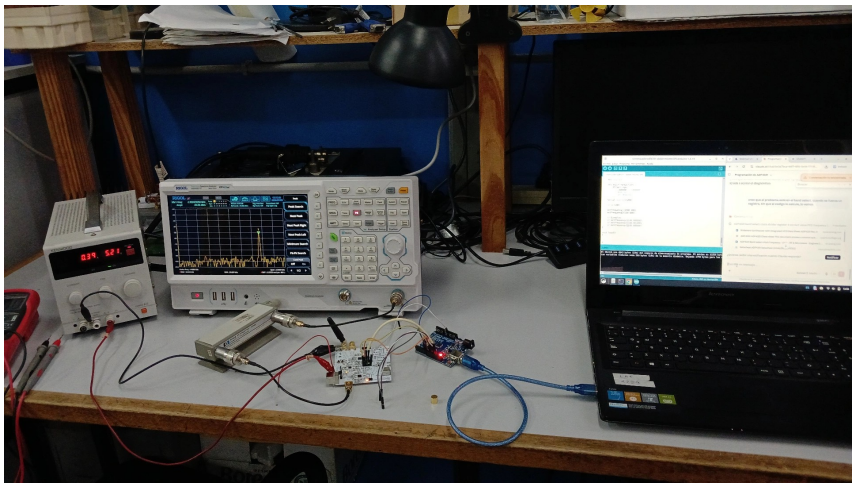
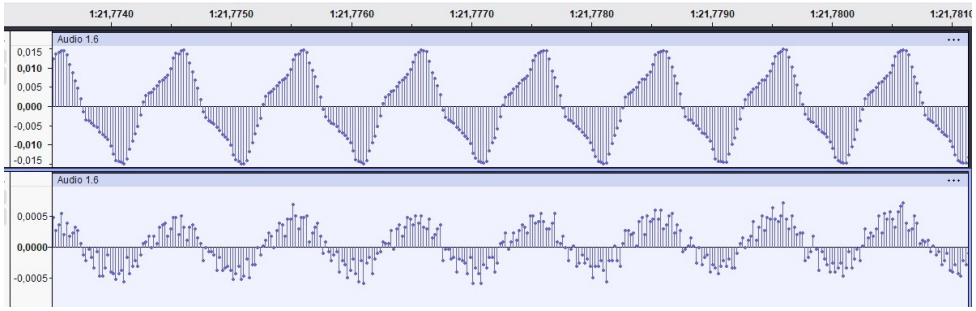


Figura 4.7: Banco de medición para generación de una única frecuencia.

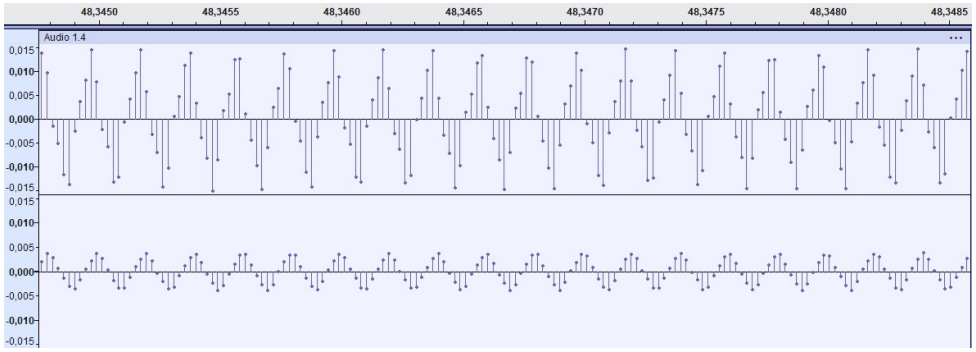
Con el mismo banco de medición, pero ajustando el programa del Arduino se generaron varios barridos de diferente frecuencia inicial, final y paso para verificar el enganche en toda la banda de interés, para ambos ADF4351.

Luego de comprobar con éxito el funcionamiento de estos subsistemas, se soldó la etapa de FI para avanzar hacia la adquisición. Para probar la placa de sonido, se generaron dos frecuencias separadas en 1, 5 y 10 kHz. Se adquirieron con el software Audacity [29] en modo estéreo, como muestran las figuras 4.8. Se eligió como FI de trabajo 10 kHz (figura c) porque una mayor FI reduce el ruido de fase y está alejada de las zonas de ruido de baja frecuencia (ruido de red y 1/f), pero los pasa bajos a la salida del mezclador limitan la frecuencia máxima que se puede elegir.

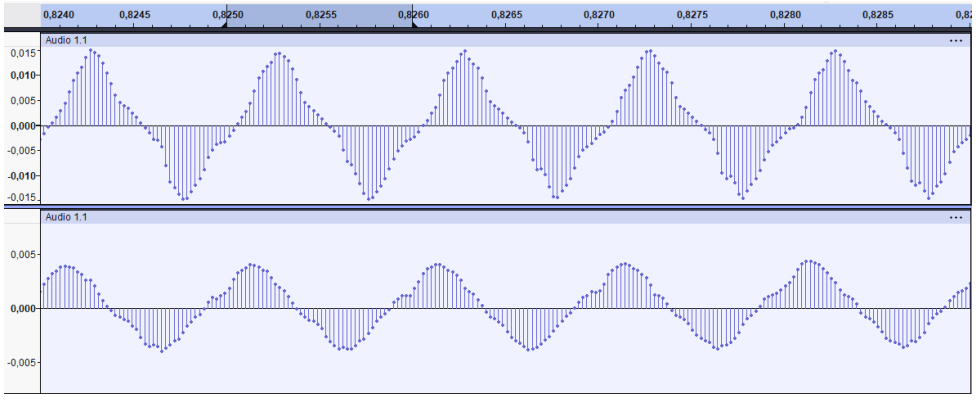
El mismo software tiene una herramienta para graficar el espectro de la señal, en el que se pudo comprobar la frecuencia de mayor amplitud y comprobar que el tercer armónico queda aproximadamente 13 dB por debajo del primero, como indica la hoja de datos de los ADF4351 y se evidencia en la figura 4.9.



(a) Grabación con FI = 1 kHz.



(b) Grabación con FI = 5 kHz.



(c) Grabación con FI = 10 kHz.

Figura 4.8: Grabaciones de prueba en Audacity.

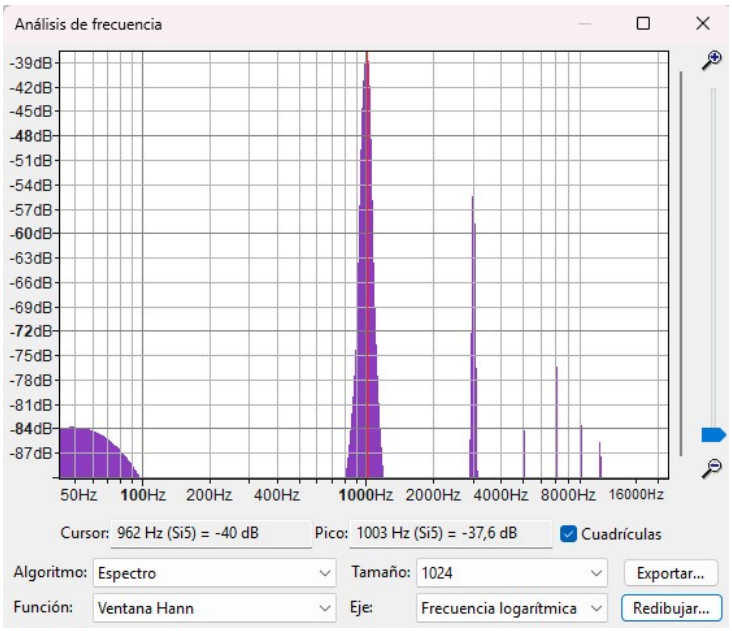
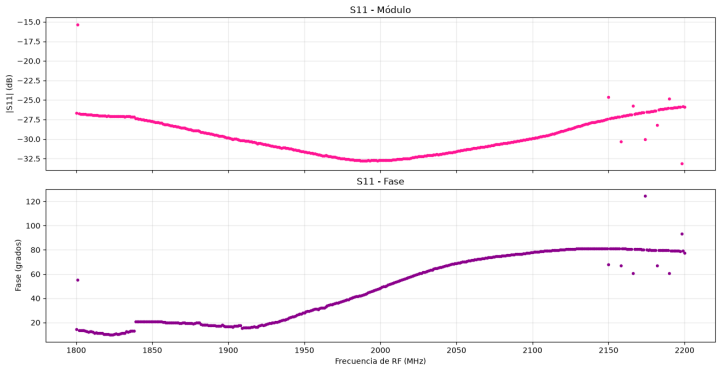
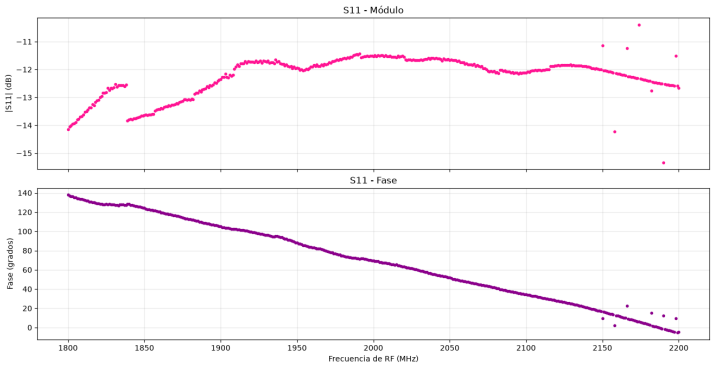


Figura 4.9: Espectro de la señal de referencia en una grabación con FI = 1 kHz.

Se procedió a probar la parte del software que aplica el algoritmo de barrido y extracción del S11 sin calibrar. Las figuras 4.10 muestran la medición de una carga de 50 Ω y un cortocircuito.



(a) Medición de carga adaptada.



(b) Medición de cortocircuito.

Figura 4.10: Mediciones pre calibración.

Se puede observar que aunque las mediciones tienen coherencia en cuanto a la forma (exceptuando algunas regiones, de las cuales se hablará luego), los valores de amplitud y fase medidos

no son los correctos. Esto se debe a los errores que se mencionaron en la sección 2.2.2.1, que se reducen mediante la calibración.

4.4 Mediciones de sistemas bajo prueba

Por último, se probó el software completo, incluyendo la calibración SOL, para varios DUT:

4.4.1 Cortocircuitos

Se midieron dos cortocircuitos. El de la figura 4.11 corresponde a una de las cargas de calibración, por lo que debería no tener ningún error que sea sistemático. **LA DEJO O SACO OTRA?**

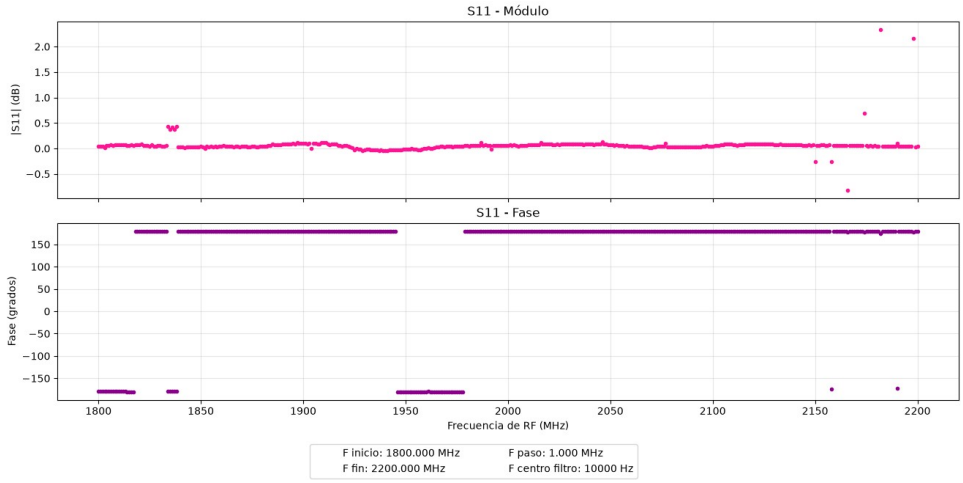


Figura 4.11: Medición de cortocircuito de calibración.

El otro cortocircuito que se usó para medir consiste en un conector SMA con su contacto central y exterior soldados mediante un pequeño tramo de alambre. Por alguna curiosa razón, este tiene una capacitancia **COMO ESTÁ AHORA ES UNA INDUCTANCIA. DE NUEVO?** tal que la fase todas las veces que se midió, con el $\epsilon - \mu VNA$ tanto como con dispositivos comerciales, dió 150°.

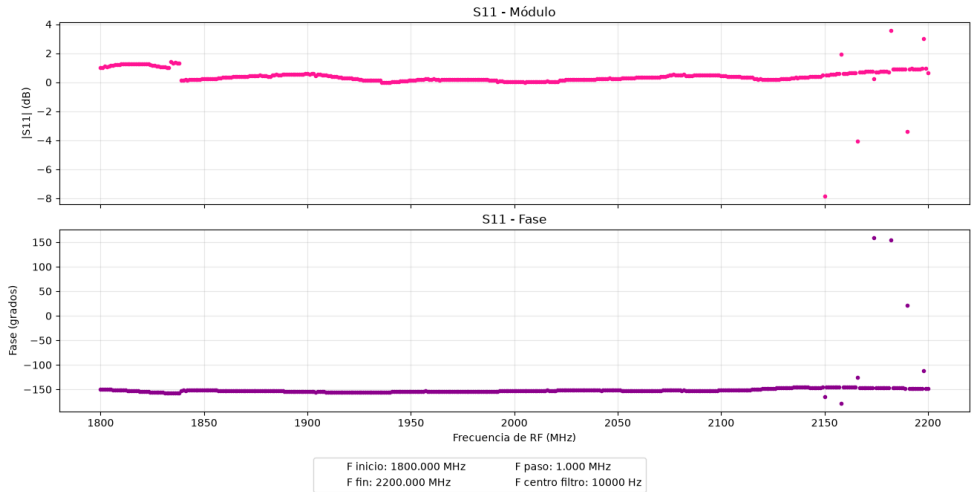


Figura 4.12: Medición de cortocircuito.

4.4.2 Circuito abierto

El circuito abierto de la figura 4.13 se implementó dejando simplemente sin conectar la entrada del DUT. Como se observa, la medición coincide con la de un circuito abierto teórico, excepto en algunos puntos.

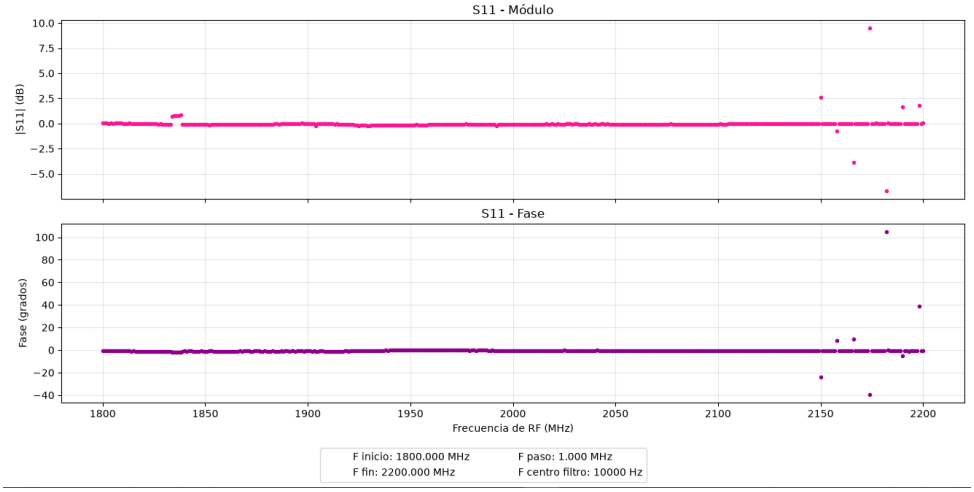


Figura 4.13: Medición de circuito abierto.

4.4.3 Adaptación (50 Ω)

DE NUEVO?

Una medición con una carga de 50 Ω equivale al puerto adaptado, por lo que la reflexión debería ser nula. Esto en escala logarítmica equivale a un módulo de - ∞ dB, y la fase sería solo ruido.

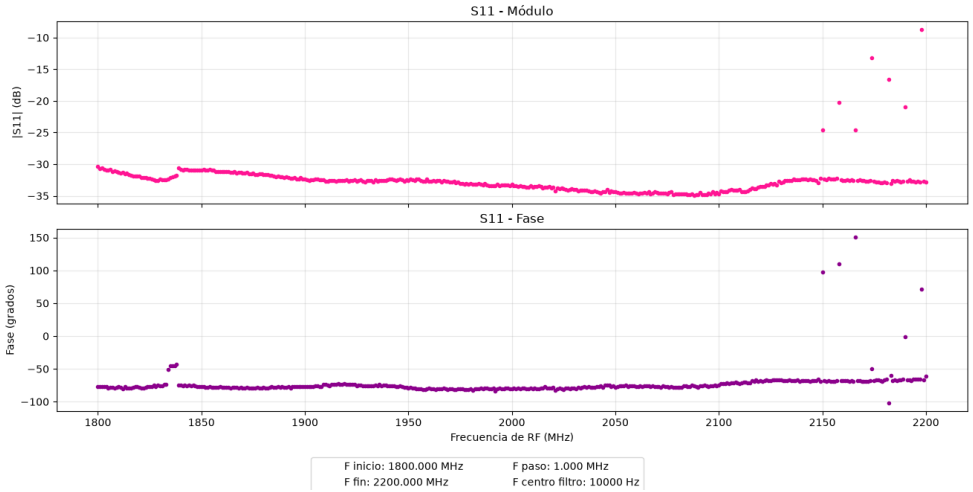


Figura 4.14: Medición de 50 Ω.

4.5 Análisis de resultados

La siguiente tabla muestra una comparación entre los coeficientes de reflexión teóricos o "reales" y el promedio (aproximado) de los medidos, en módulo [dB] y ángulo [°]:

Carga	Γ_{REAL}	Γ_{MEDIDO}
50 Ω	- ∞	-32,5 ∠ -70
68 Ω	-16,33 ∠ 0	-15,8 ∠ 6
Corto circuto	0 ∠ 180	0 ∠ 180
Circuito abierto	0 ∠ 0	0 ∠ 0
Cortocircuito de 150°	0 ∠ 150	

Que el dispositivo pueda medir correctamente una carga de 68 Ω da cuenta de que puede diferenciar una ROE de al menos 1,36 (que es la que corresponde a una carga de esa magnitud), superando el requerimiento RP-05 (ver apéndice "Especificación de Requerimientos" - C) que exigía una ROE mínima distinguible de 1,5.

Sobre los puntos en la región de 2150 a 2200 MHz que no siguen la misma tendencia que el resto, se puede notar que ocurren siempre en las mismas frecuencias. Aunque aún no se hicieron mediciones para comprobarlo, lo más probable es que el filtro FIR tenga un ancho de banda muy angosto para las mediciones en alta frecuencia. A veces el enganche de los ADF4351 no ocurre exactamente en la frecuencia deseada sino en una cerca, lo que provoca que la mezcla no caiga en la frecuencia de FI y por lo tanto el pico de máxima amplitud caiga fuera de la banda de paso. Esta teoría se basa en mediciones anteriores donde agrandando el ancho de banda ha disminuido la cantidad de puntos anómalos.

En la región cercana a 1850 MHz también se puede observar un comportamiento atípico. **NO TENGO NI IDEA**

4.5.1 Sensor resonante

Finalmente, para verificar que el sistema está adaptado para operar en conjunto con un sensor de glucemia no invasivo, se utiliza el sensor que resuena alrededor de los 2 GHz. Se mide en principio en vacío, como muestra la figura 4.15.

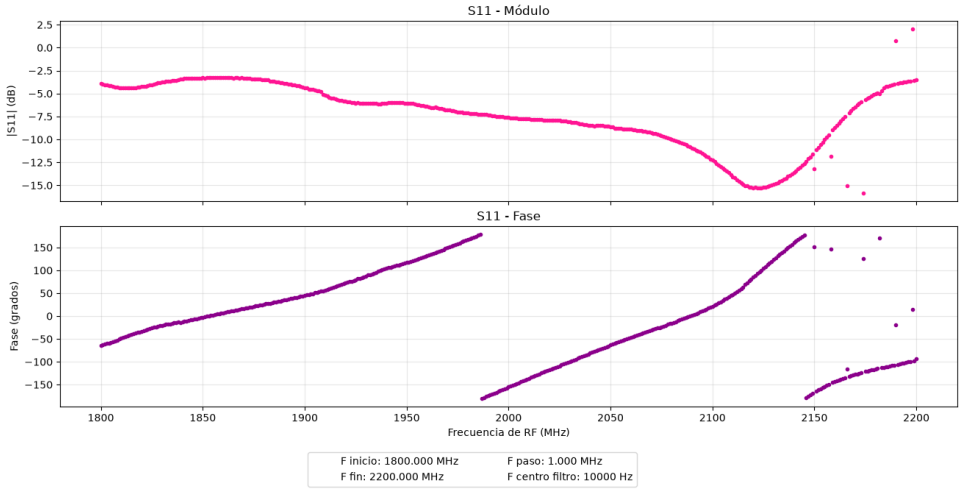


Figura 4.15: Medición del sensor de glucemia en vacío.

Si se introduce en el sensor un dedo, se espera que se mueva la frecuencia de resonancia por lo explicado en el capítulo 1. Este comportamiento es confirmado en la figura 4.16

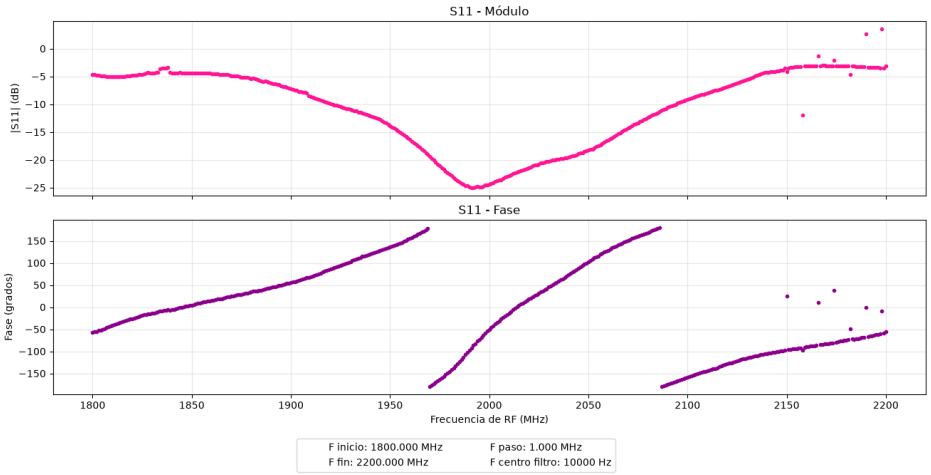
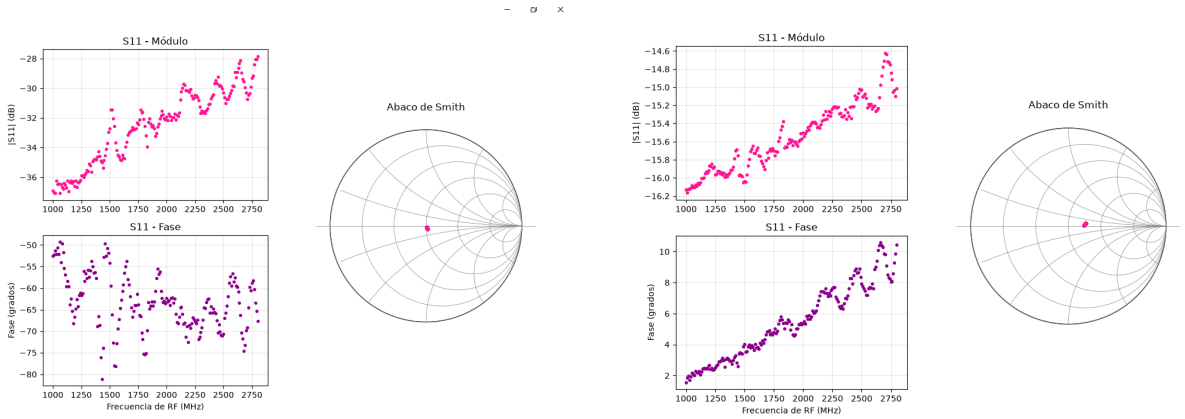


Figura 4.16: Medición del sensor de glucemia con un dedo.

4.6 Interfaz de usuario

Al incluir la interfaz de usuario, se hicieron algunas mediciones para comprobar su funcionamiento, como 50 y 68 Ω que se pueden ver en las figuras 4.17. La interfaz agregó el gráfico en un ábaco de Smith.



(a) 50 Ω (b) 68 Ω

Figura 4.17: Mediciones realizadas desde la interfaz de usuario

4.7 Comparación con dispositivo comercial

Para finalizar, se midieron algunas cargas con el Analizador Vectorial de Redes incorporado en el analizador de espectros Rigol Series RSA3000 [27], las cuales se pueden observar en las figuras 4.18. Es importante aclarar que ambas mediciones se hicieron con el mismo kit de calibración.

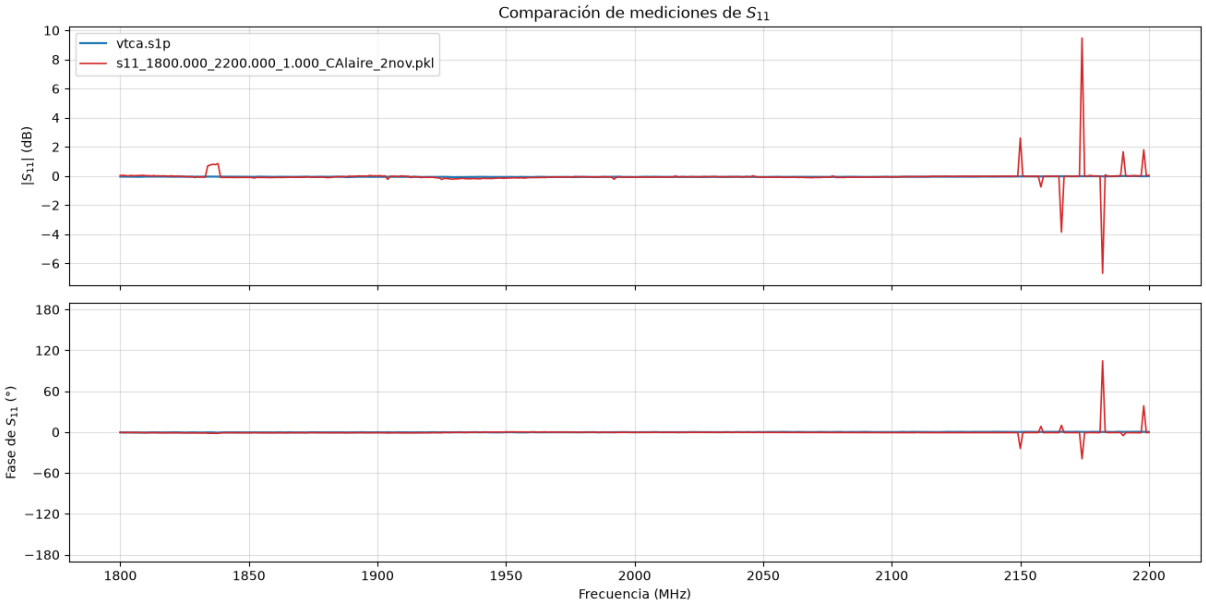


Figura 4.18: Comparación circuito abierto $\epsilon - \mu VNA$ vs dispositivo comercial.

la de 50 y 150 se ve mal por la fase. la de 68 no tan mal pero seria mejor si la fase estuviera bien. corto mmmm

Capítulo 5

Conclusiones y trabajos futuros

El desarrollo del proyecto permitió obtener un Analizador Vectorial de Redes de un puerto para la banda UHF de 1,8 a 2,2 GHz funcional, de bajo costo y tamaño compacto, orientado a la medición no invasiva de glucemia mediante sensores de microondas. Se diseñó, construyó, calibró y ensayó a nivel de hardware y software, y permite realizar mediciones comparables con las de un dispositivo comercial. El prototipo desarrollado integra la generación, reflectometría y heterodinación de señales en un módulo portable que satisface los requerimientos fijados para la aplicación. Por lo tanto, se puede afirmar que el objetivo general fue logrado.

A partir de los ensayos realizados y el análisis de los resultados obtenidos, y en base a los objetivos particulares, se concluye que el sistema:

- Fue implementado en una placa de circuito impreso de 4 capas, lo que permitió tener un plano de masa continuo bajo las líneas de transmisión coplanares y optimizar la distribución de la alimentación, reduciendo posibles fuentes de error y ruido electromagnético y garantizando la adaptación de impedancias;
- Fue capaz de generar un barrido en la banda de frecuencias de interés y esa banda pudo ser extendida, lo que permite extender las posibilidades a futuro de diseñar sensores que resuenen en otras frecuencias;
- Implementó un sistema heterodino basado en dos sintetizadores de RF (ADF4351) y un mezclador de dos canales (ADL5802) que demostró ser eficaz para trasladar la información desde la banda de microondas a una frecuencia intermedia, que simplifica la etapa de acondicionamiento de las señales y la digitalización, reduciendo el costo del hardware de adquisición;
- Incluyó un acoplador direccional basado en un puente resistivo con una directividad simulada superior a 36,5 dB en toda la banda de trabajo, superando ampliamente la directividad mínima de 15 dB planteada en la Especificación de Requerimientos C, asegurando una adecuada separación entre las ondas incidente y reflejada a costa de una pérdida por inserción mayor que la de un circuito comercial;
- Superó el requerimiento de ROE mínima aceptable de 1,5 al poder diferenciar entre una carga de 50 y 68 Ω;
- Implementó una rutina de calibración SOL de tres términos de error, que eliminó los errores sistemáticos garantizando la repetibilidad en la determinación del coeficiente de reflexión (S11).

Todos estos objetivos permitieron obtener exitosamente el coeficiente de reflexión del sistema bajo prueba, habiéndose medido cargas como cortocircuitos, circuitos abiertos, resistencias, cables y sensores resonantes. En cuanto a estos últimos, se detectaron desplazamientos en la

frecuencia de resonancia asociados a las variaciones de los parámetros eléctricos del medio, demostrando que el $\epsilon - \mu VNA$ es una herramienta útil para la línea de investigación en glucometría no invasiva del Laboratorio de Comunicaciones.

5.1 Trabajos futuros

A partir de los resultados obtenidos, se proponen como líneas de trabajo a futuro:

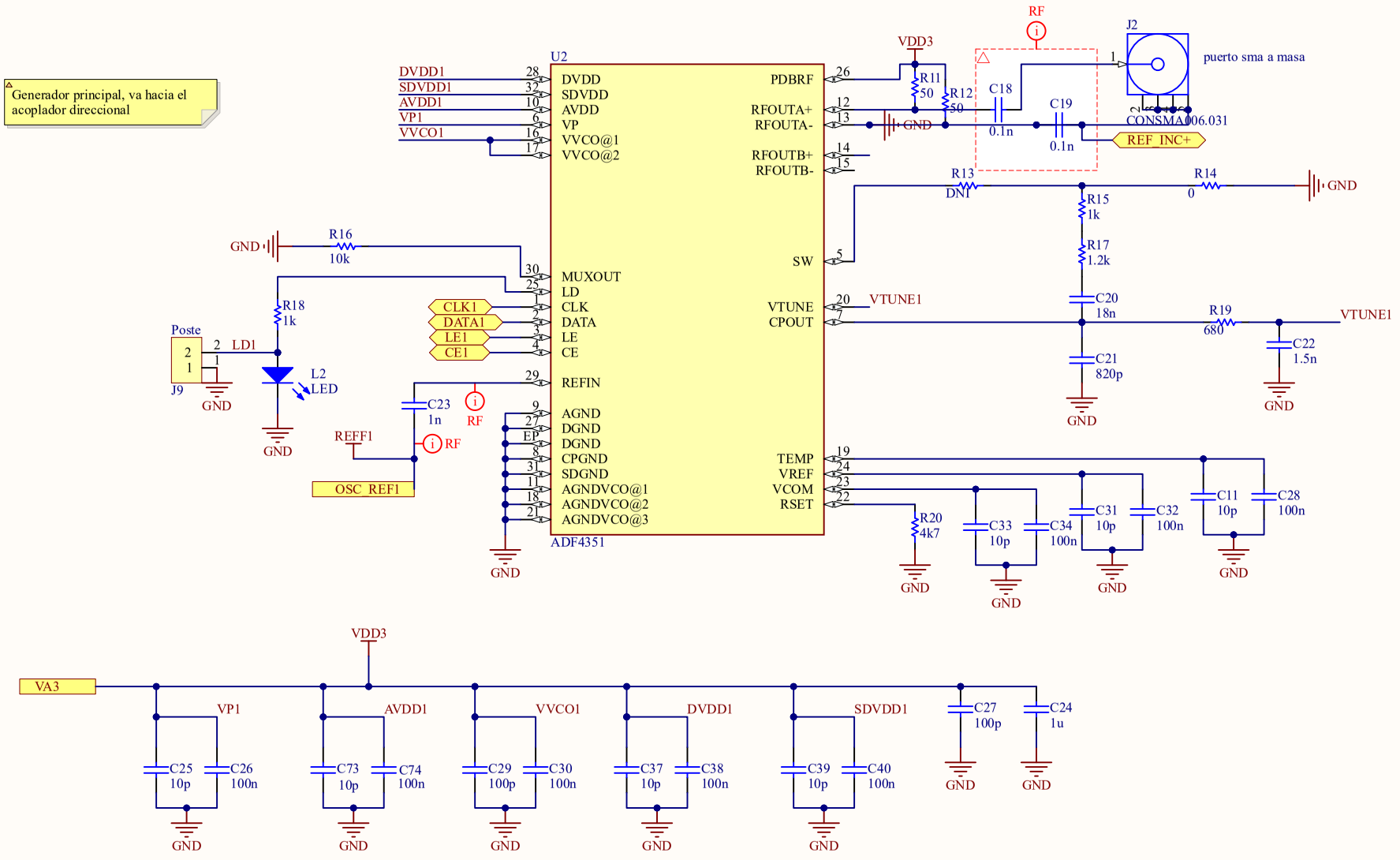
- Diseñar un gabinete que permita contener el VNA y transportarlo fácilmente;
- Embeber el software y firmware en un microcontrolador que permita independizar el dispositivo de una PC y prescindir del uso de cables puente, sensibles a radiación electromagnética;
- En línea con los puntos anteriores, agregar una forma de visualizar los resultados ya sea con una pantalla incorporada o en una aplicación móvil mediante Bluetooth o Wi-Fi;
- Optimizar el tiempo de barrido calculando el tiempo de estabilización del enganche y adquiriendo la menor cantidad de tiempo posible;
- Explorar la posibilidad de incorporar un segundo canal para medir transmisión (parámetro S_{21}) que permita caracterizar otro tipo de sensores;
- Seguir miniaturizando el PCB e incorporar blindaje electromagnético para minimizar el ruido;
- Utilizar un dieléctrico que se comporte mejor que el FR4 en alta frecuencia, como por ejemplo un Rogers.

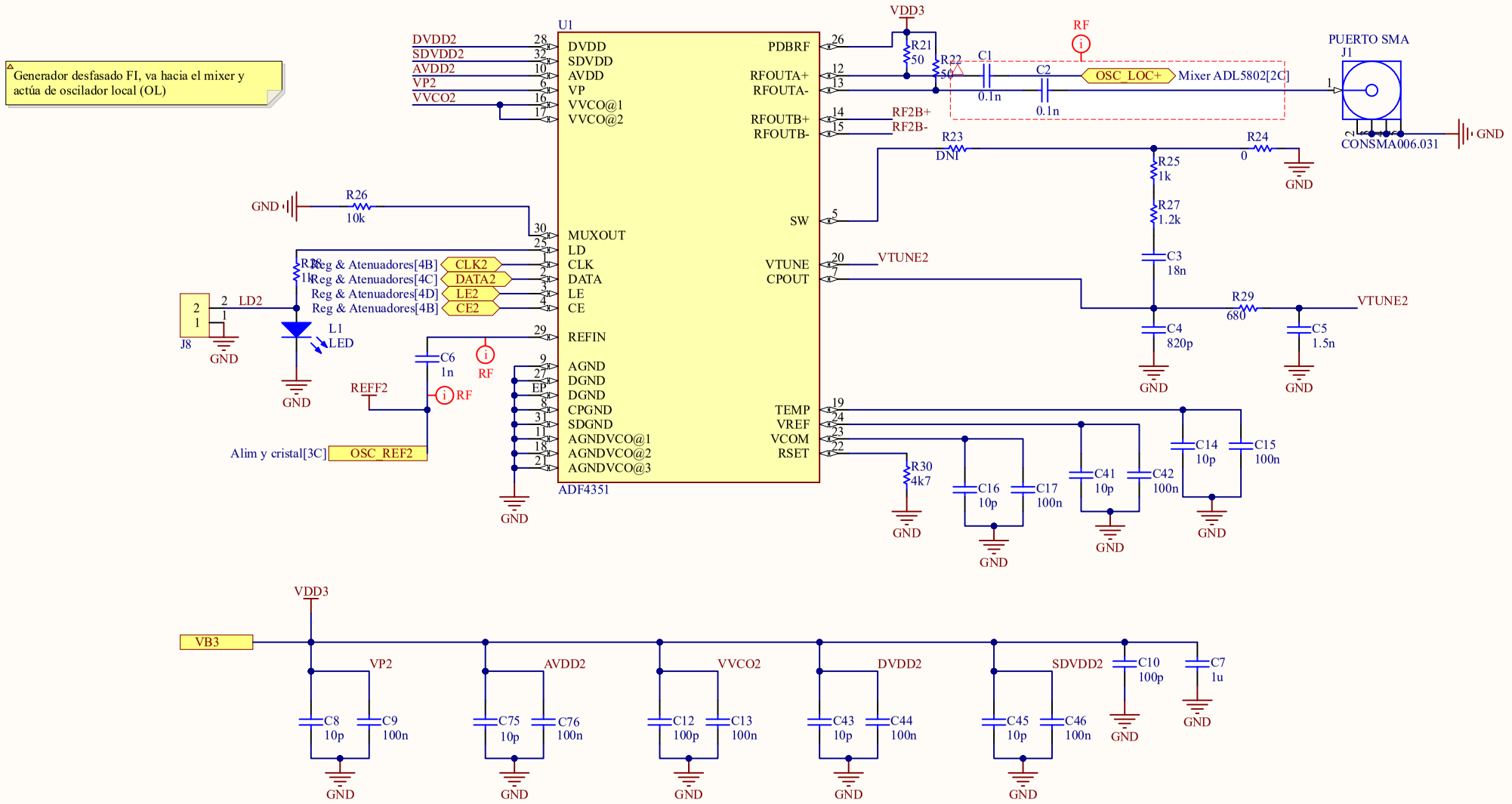
Capítulo 6

Esquemáticos y código

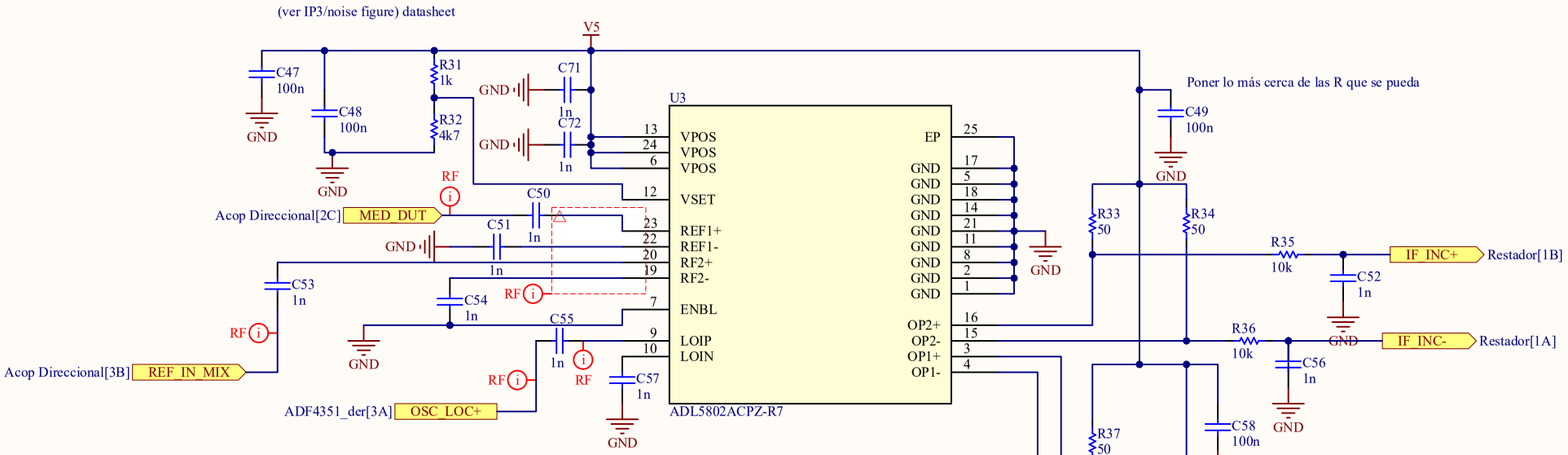
6.1 Esquemáticos Altium

A continuación se presentan los esquemáticos de los distintos bloques funcionales del sistema, exportados de Altium Designer.

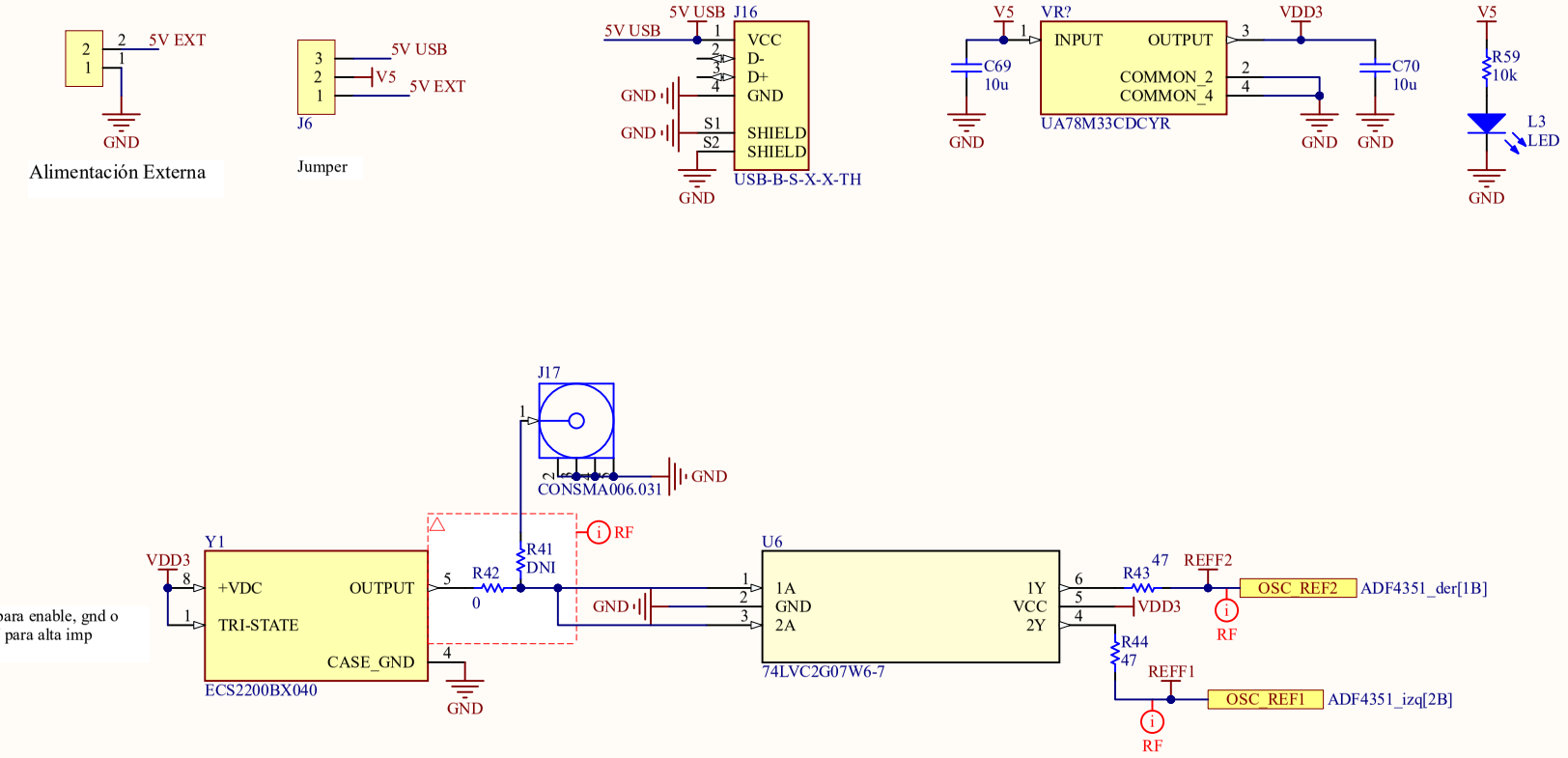

Generador principal, va hacia el acoplador direccional
U2
ADF4351
DVDD1
SDVDD1
AVDD1
VP1
VVCO1
DVDD
SDVDD
AVDD
VP
VVCO@1
VVCO@2
PDBRF
RFOUTA+
RFOUTA-
RFOUTB+
RFOUTB-
SW
MUXOUT
LD
CLK
DATA
LE
CE
REFIN
VTUNE
CPOUT
AGND
DGND
DGND
CPGND
SDGND
AGNDVCO@1
AGNDVCO@2
AGNDVCO@3
TEMP
VREF
VCOM
RSET
CLK1
DATA1
LE1
CE1
VDD3
RF
J2
puerto sma a masa
CONSMA006.031
REF_INC+
R11 50
R12 50
C18 0.1n
C19 0.1n
R13 DNI
R14 0
R15 1k
R16 10k
R17 1.2k
R18 1k
R19 680
R20 4k7
C20 18n
C21 820p
C22 1.5n
C23 1n
C11 10p
C28 100n
C31 10p
C32 100n
C33 10p
C34 100n
VTUNE1
Poste
LD1
L2 LED
J9
REFF1
OSC_REF1
GND
VA3
C25 10p
C26 100n
C73 10p
C74 100n
C29 100p
C30 100n
C37 10p
C38 100n
C39 10p
C40 100n
C27 100p
C24 1u
poner capacitores de desacople cerca de los pines: 6 (Vp), 10 (AVDD), 16, 17 (VVCO), 19 (TEMP), 23 (VCOM), 24 (VREF), 28 DVDD, 32 SDVDD
Title
ADF4351 (Gen RF) izquierdo
Size
A4
Number
Revision
Date: 10/29/2026
Sheet of
File: C:\Users\...\ADF4351_izq.SchDoc
Drawn By:



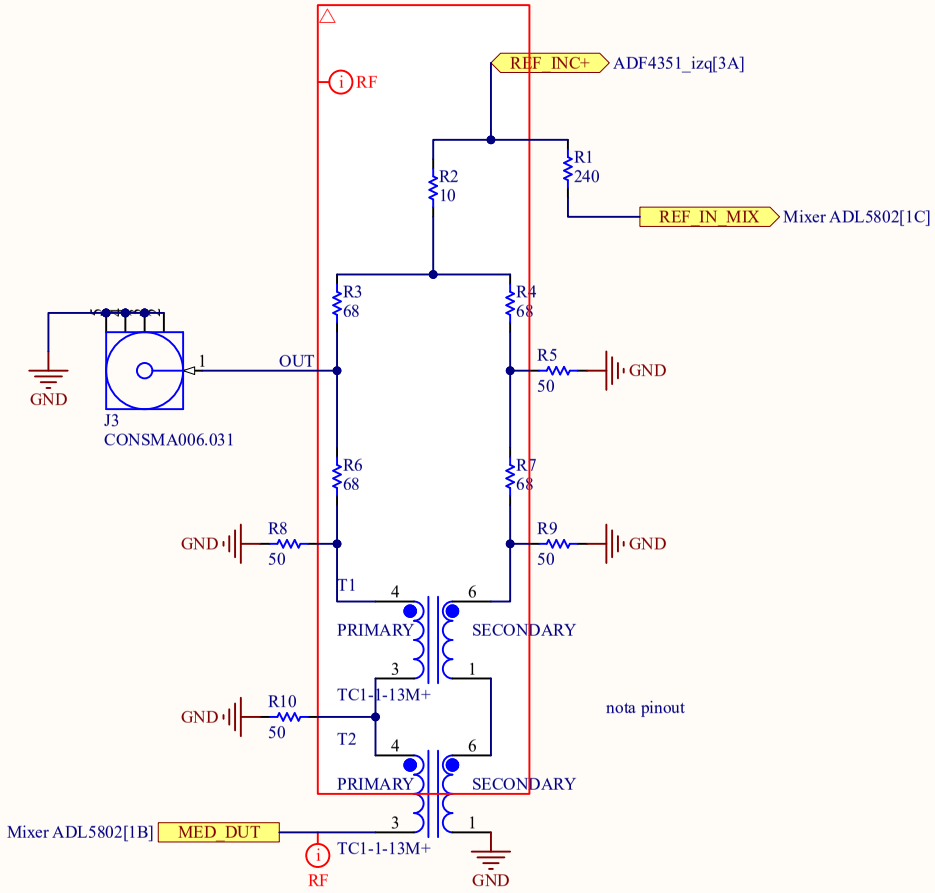
Title	ADF4351 (OL) derecho	
Size: A4	Number	Revision
Date: 10/29/2026		Sheet of
File: C:\Users\...\ADF4351_der.SchDoc		Drawn By:



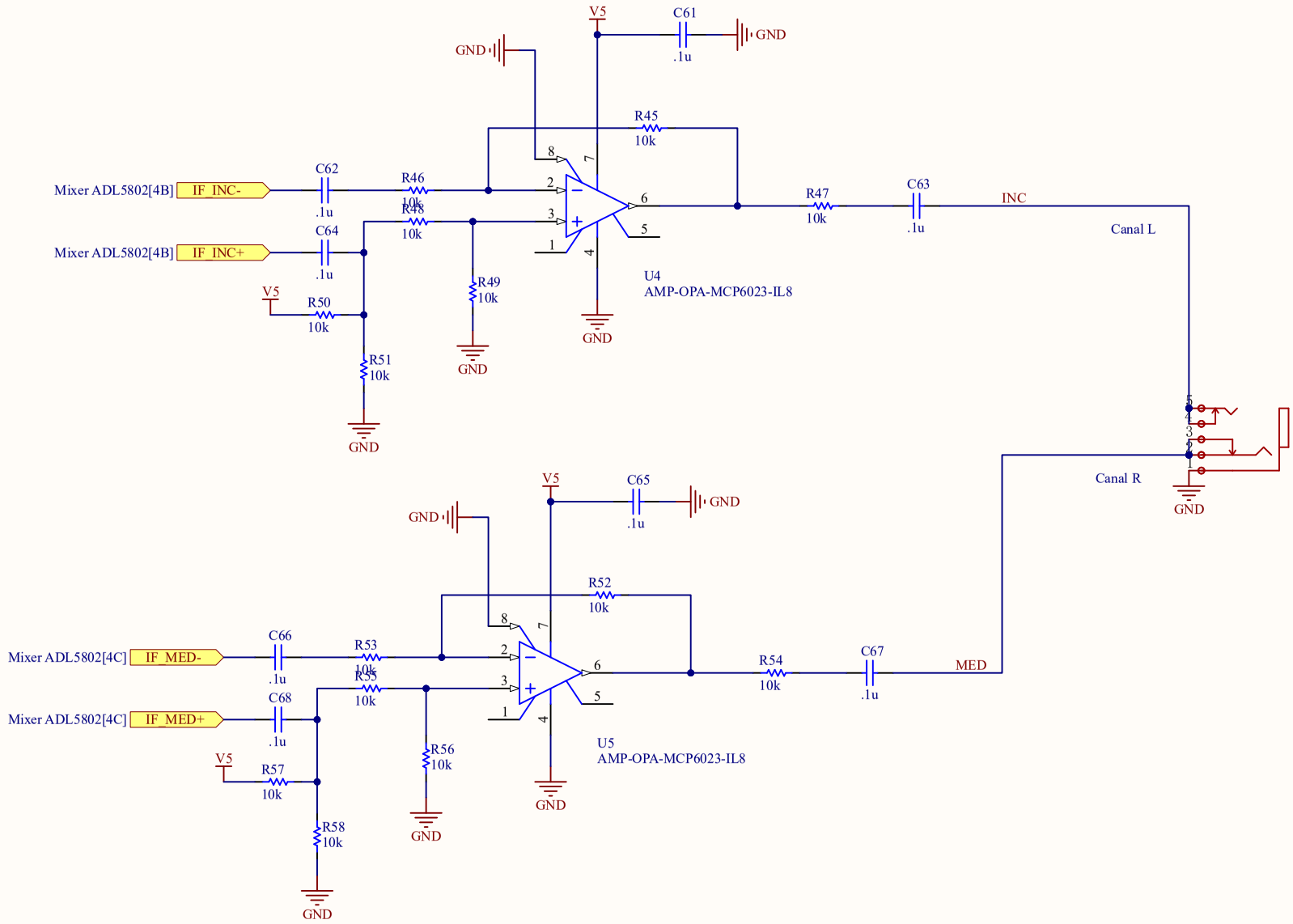
Title: Mezclador		
Size: A4	Number:	Revision:
Date: 10/29/2026		Sheet of
File: C:\Users\...\Mixer ADL5802.SchDoc		Drawn By:



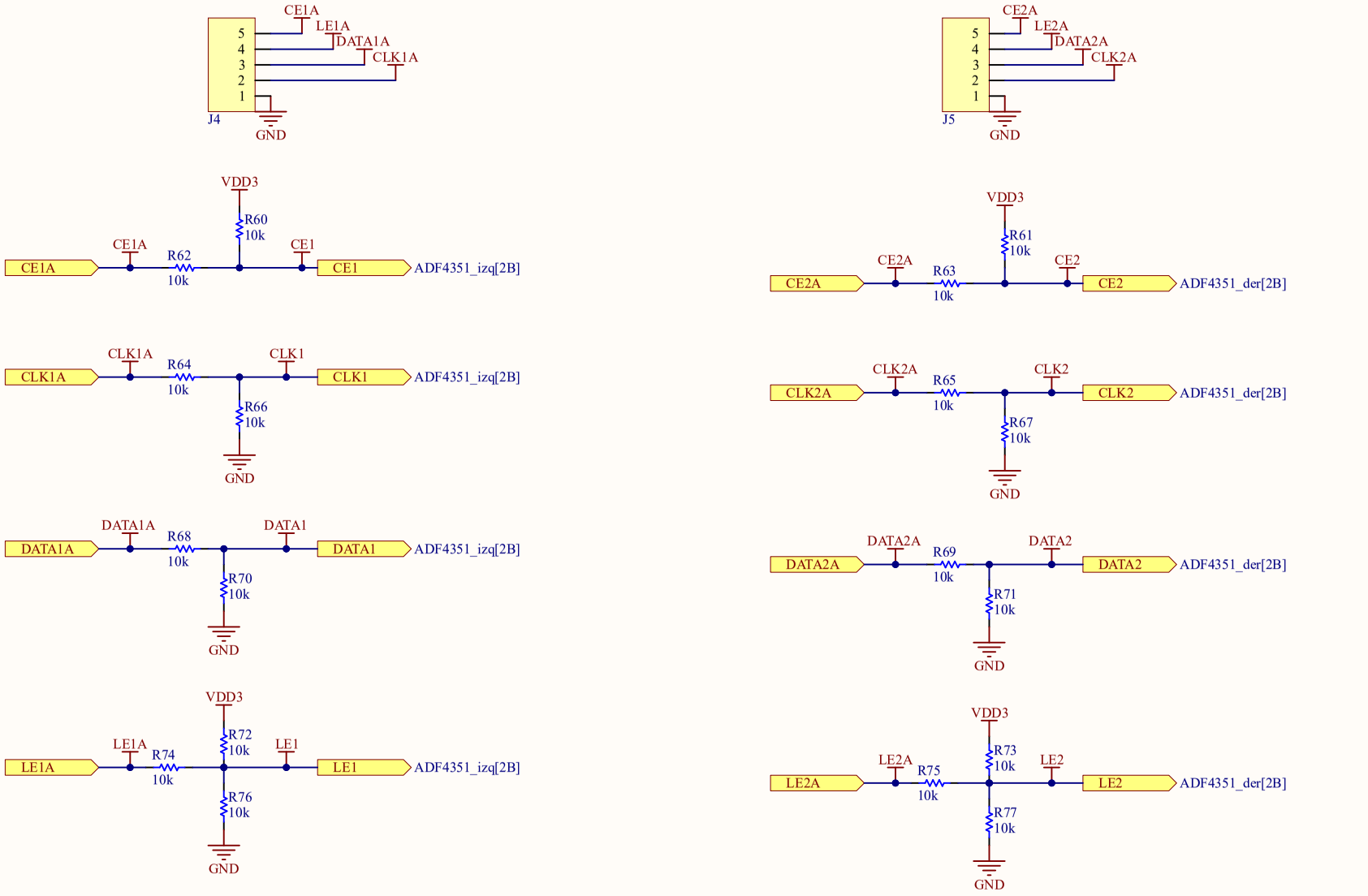
Title: Alimentación y CLK		
Size: A4	Number:	Revision:
Date: 10/29/2026		Sheet of
File: C:\Users\...\Alim y cristal.SchDoc		Drawn By:



Title: Acoplador Direccional		
Size: A4	Number:	Revision:
Date: 10/29/2026		Sheet of
File: C:\Users\..\Acop Direccional.SchDoc		Drawn By:



Title: Etapa FI		
Size: A4	Number:	Revision:
Date: 10/29/2026		Sheet of
File: C:\Users\..\Restador.SchDoc		Drawn By:



Title: Atenuadores		
Size: A4	Number:	Revision:
Date: 10/29/2026		Sheet of
File: C:\Users\...\Reg & Atenuadores.SchDoc		Drawn By:

6.2 Firmware

```
#include <SPI.h>

//-------------------- Pines --------------------

#define PIN_LE1   10
#define PIN_CE1    9

#define PIN_LE2   7
#define PIN_CE2   6

//-------------------- Constantes del PLL --------------------
const double OFFSET_MHZ = 0.010; // MHz
const double REF_FREQ = 10000000.0;      // 10 MHz
const uint16_t R_COUNTER = 10;
const bool REF_DOUBLER = false;
const bool REF_DIV2 = true;

const uint16_t MOD = 1000;
const uint16_t PHASE = 1;

//-------------------- Declaración de registros --------------------

uint32_t reg[6];

//----------------Escritura de registros----------------------------------

void writeRegister1(uint32_t value)   //le estoy pasando una variable local que va a ser el valor del registro
{
    digitalWrite(PIN_LE1, LOW);

    SPI.transfer((value >> 24) & 0xFF);
    SPI.transfer((value >> 16) & 0xFF);
    SPI.transfer((value >> 8) & 0xFF);
    SPI.transfer(value & 0xFF); // toma cada byte desplazando y multiplicando por FF

    digitalWrite(PIN_LE1, HIGH); //transfiere los datos internamente
    delayMicroseconds(2); //
    digitalWrite(PIN_LE1, LOW); // cierra la "compuerta de carga"
}
void writeRegister2(uint32_t value)
{
    digitalWrite(PIN_LE2, LOW);

    SPI.transfer((value >> 24) & 0xFF);
    SPI.transfer((value >> 16) & 0xFF);
    SPI.transfer((value >> 8) & 0xFF);
    SPI.transfer(value & 0xFF);

```

```
    digitalWrite(PIN_LE2, HIGH);
    delayMicroseconds(2);
    digitalWrite(PIN_LE2, LOW);
}
//-------------------------------------------------

void updateADF1()
{
    for (int i = 5; i >= 0; i--) {
        writeRegister1(reg[i]);
    }
}

void updateADF2()
{
    for (int i = 5; i >= 1; i--) {
        writeRegister2(reg[i]);
    }

    delay(10);   //delay acá para sincronizar fase

    writeRegister2(reg[0]);
}

//-------------------------------------------------
// Armo los 6 registros para una frecuencia dada en MHz.
//-------------------------------------------------
void calcularRegistros(double MHz)
{
    uint8_t rf_div_sel;
    uint8_t rf_div;

    if (MHz >= 2200) {
        rf_div_sel = 0;
        rf_div = 1;
    } else if (MHz >= 1100) {
        rf_div_sel = 1;
        rf_div = 2;
    } else if (MHz >= 550) {
        rf_div_sel = 2;
        rf_div = 4;
    } else if (MHz >= 275) {
        rf_div_sel = 3;
        rf_div = 8;
    } else {
        rf_div_sel = 4;
        rf_div = 16;
    }

    double fpfd = REF_FREQ;

    if (REF_DOUBLER)
```

```
101         fpfd *= 2.0;
102 
103     if (REF_DIV2)
104         fpfd /= 2.0;
105 
106     fpfd /= R_COUNTER;
107 
108     double fvco = MHz * 1000000.0 * rf_div;
109 
110     double N = fvco / fpfd;
111 
112     uint16_t INT = floor(N); // toma la parte entera hacia abajo
113     uint16_t FRAC = round((N - INT) * MOD);
114 
115     if (FRAC >= MOD) {
116         INT++;
117         FRAC = 0;
118     }
119 
120     //---------------- R0 ----------------
121 
122     reg[0] = 0;
123     reg[0] |= ((uint32_t)INT) << 15;
124     reg[0] |= ((uint32_t)FRAC) << 3;
125 
126     //---------------- R1 ----------------
127 
128     reg[1] = 0;
129     reg[1] |= (1UL << 27);             // Prescaler = 8/9
130     reg[1] |= ((uint32_t)PHASE) << 15;
131     reg[1] |= ((uint32_t)MOD) << 3;
132     reg[1] |= 1;
133 
134     //---------------- R2 ----------------
135 
136     reg[2] = 0;
137 
138     if (REF_DOUBLER)
139         reg[2] |= (1UL << 25);
140 
141     if (REF_DIV2)
142         reg[2] |= (1UL << 24);
143 
144     reg[2] |= ((uint32_t)R_COUNTER << 14);
145     reg[2] |= (15UL << 9);   // Charge Pump = 5.00 mA
146     reg[2] |= (1UL << 6);    // PD Polarity = Positive
147     //reg[2] |= (1UL << 13);   // divider select value de R4 double buffered
148     reg[2] |= 2;
149 
150     //---------------- R3 ----------------
151 
152     reg[3] = 0x008004B3; // para fast lock
```

```
153
154     //---------------- R4 ----------------
155
156     reg[4] = 0;
157     reg[4] |= (1UL << 23);                    // Feedback fundamental
158     reg[4] |= ((uint32_t)rf_div_sel << 20);   // RF Divider
159     reg[4] |= (20UL << 12);                   // Band Select Divider 80UL para
        ↪  enganche rapido, 20 para lento
160     reg[4] |= (1UL << 5);                     // RF Output Enable
161     reg[4] |= (3UL << 3);                     // +5 dBm
162     reg[4] |= 4;
163
164     //---------------- R5 ----------------
165
166     reg[5] = 0x00580005;
167 }
168 //-------------------------------------------------
169 void setFrequency1(double MHz)
170 {
171     calcularRegistros(MHz);
172     updateADF1();
173 }
174
175 void setFrequency2(double MHz)
176 {
177     calcularRegistros(MHz);
178     updateADF2();
179 }
180
181 //-------------------------------------------------
182 // Protocolo serial: se le manda una línea de texto con la frecuencia en
183 // MHz para ADF1 (ej: "870.000000\n"). El Arduino setea ADF1 en esa
184 // frecuencia y ADF2 en (frecuencia + OFFSET_MHZ), y responde con una
185 // línea "OK <frecuencia>" cuando terminó.
186 //-------------------------------------------------
187
188 char bufferEntrada[32];
189 uint8_t idxBuffer = 0;
190
191 void procesarComando(char *cmd)
192 {
193     double f = atof(cmd);
194
195     if (f <= 0.0) {
196         Serial.print("ERROR frecuencia invalida: ");
197         Serial.println(cmd);
198         return;
199     }
200
201     setFrequency1(f);
202     setFrequency2(f + OFFSET_MHZ);
203
```

```
    Serial.print("OK ");
    Serial.println(f, 6); // con 6 decimales
}

void setup()
{
    pinMode(PIN_LE1, OUTPUT); //configura pines como salidas
    pinMode(PIN_CE1, OUTPUT);
    pinMode(PIN_LE2, OUTPUT);
    pinMode(PIN_CE2, OUTPUT);

    SPI.begin();
    SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));

    Serial.begin(115200);
    delay(100);

    Serial.println("Listo");
}

void loop()
{
    while (Serial.available()) {
        char c = Serial.read();

        if (c == '\n') {
            bufferEntrada[idxBuffer] = '\0';
            procesarComando(bufferEntrada);
            idxBuffer = 0;
        } else if (idxBuffer < sizeof(bufferEntrada) - 1) {
            bufferEntrada[idxBuffer++] = c;
        }
        // si se llena el buffer sin '\n', se ignoran caracteres de más
        // hasta el próximo salto de línea (evita overflow)
    }
}
```

6.3 Software

```
"""
Barrido de RF + adquisición + cálculo de S11.

Por cada frecuencia del barrido:
    1. Se le manda la frecuencia al Arduino (adf4351_serial.ino) por serial.
    2. Se graba una captura corta desde la entrada line-in (ref = canal
       izquierdo, medición = canal derecho).
    3. Se filtra cada canal con un FIR pasabanda alrededor de 1 kHz.
    4. Se calcula la FFT compleja de ambos canales.
    5. Se busca el pico del tono de batido en el canal de referencia, y
       se toma el mismo bin en el canal de medición.
    6. S11_punto = medición[bin] / referencia[bin].

```

```
14  Al terminar se grafica y se guardan los resultados en un .csv y en un .pk la
    ↪  imagen interactiva de matplotlib.
15
16  Requisitos:
17      pip install pyserial sounddevice numpy scipy matplotlib
18  """
19
20  import csv
21  import time
22  import os
23  import pickle
24  import re
25  import tkinter as tk
26  from tkinter import simpledialog
27
28  import numpy as np
29  import matplotlib.pyplot as plt
30  import serial
31  import sounddevice as sd
32  from matplotlib.lines import Line2D
33  from scipy.signal import firwin, lfilter
34  from scipy.signal.windows import get_window
35
36  # ==================== Configuración: Arduino / barrido de RF
    ↪  ====================
37
38  PUERTO = "COM3"
39  BAUDRATE = 115200
40
41  F_INICIO_MHZ = 1800.000
42  F_FIN_MHZ = 2200.000
43  F_PASO_MHZ = 1.000
44  OFFSET_MHZ=0.010 # Frecuencia de audio
45  FS_AUDIO = 44100
46  DURACION_CAPTURA_S = 0.60
47  DESCARTE_S = 0.150
48  # ==================== Salidas ====================
49  CARPETA_CSV = os.path.join(os.path.dirname(os.path.abspath(__file__)), "Archivos
    ↪  csv resultados-calibracion")
50  ARCHIVO_CSV = os.path.join(CARPETA_CSV,
    ↪  f"resultado_{F_INICIO_MHZ}_{F_FIN_MHZ}_{F_PASO_MHZ}_cableaCA_2nov.csv")
51  NOMBRE_FIGURA_BASE = f"s11_{F_INICIO_MHZ:.3f}_{F_FIN_MHZ:.3f}_{F_PASO_MHZ:.3f}"
52  archivo_calibracion = os.path.join(
    ↪  CARPETA_CSV,f"calibracion_{F_INICIO_MHZ}_{F_FIN_MHZ}_{F_PASO_MHZ} -
    ↪  14oct.csv")
53
54  # ==================== Configuración: adquisición de audio ====================
55  # Índice del dispositivo de audio a usar (line-in). None = dispositivo de
56  # entrada por defecto del sistema. Si tenés dudas de cuál es, corré:
57  #   python -c "import sounddevice as sd; print(sd.query_devices())"
58  # y poné acá el índice que corresponda a tu placa de sonido / line-in.
59  DISPOSITIVO_AUDIO = None
```

```
60
61  # ==================== Configuración: filtro FIR + FFT ====================
62  F_CENTRO_FILTRO = 10000       # Hz, tono de batido esperado
63  ANCHO_BANDA_FILTRO = 200      # Hz
64  NUMTAPS_FILTRO = 1001         # debe ser menor que la cantidad de muestras por
    ↪  captura
65  VENTANA_FFT = "flattop"
66  # Banda donde se busca el pico del tono de batido (Hz)
67  BANDA_BUSQUEDA_PICO = (F_CENTRO_FILTRO - ANCHO_BANDA_FILTRO / 2,
68                         F_CENTRO_FILTRO + ANCHO_BANDA_FILTRO / 2)
69  # ========================================================================
70
71  FRECUENCIAS_EXCLUIDAS_MHZ = []
72  # ==================== Calibración ====================
73  def calculo_errores (GM_CC, GM_CA, GM_50, n):
74      e_00=np.empty(n,dtype=complex)   #inicializo un vector de long n vacio con
        ↪  valores complejos
75      e_11=np.empty(n,dtype=complex)
76      delta_e=np.empty(n,dtype=complex)
77
78      for i in range(n):
79          A=np.array([
80              [1, -GM_CC[i], 1], #CC
81              [1, 0, 0],         #50
82              [1, GM_CA[i], -1]  #CA
83          ],dtype=complex)
84          B=np.array([GM_CC[i], GM_50[i], GM_CA[i]])
85          sol=np.linalg.solve(A,B)
86          e_00[i]=sol[0]
87          e_11[i]=sol[1]
88          delta_e[i]=sol[2]
89      return e_00, e_11, delta_e
90
91  def correccion_punto(gm, e_00, e_11, delta_e):
92      denominador = gm * e_11 - delta_e
93      if abs(denominador) < 1e-15:
94          denominador = 1e-15 + 1e-15j
95      return (gm - e_00) / denominador
96
97  def correccion(GM,e_00,e_11,delta_e,n):
98      Gamma=np.empty(n,dtype=complex)
99      for i in range(n):
100         Gamma[i] = correccion_punto(GM[i], e_00[i], e_11[i], delta_e[i])
101     return Gamma
102
103 def guardar_calibracion_csv(path, freqs, e_00, e_11, delta_e):
104     with open(path, "w", newline="", encoding="utf-8") as f:
105         writer = csv.writer(f)
106         writer.writerow(["freq_MHz", "e_00_real", "e_00_imag", "e_11_real",
            ↪  "e_11_imag", "delta_e_real", "delta_e_imag"])
107         for i in range(len(freqs)):
108             writer.writerow([f"{freqs[i]:.6f}",
```

```
109                                 f"{e_00[i].real:.10e}", f"{e_00[i].imag:.10e}",
110                                 f"{e_11[i].real:.10e}", f"{e_11[i].imag:.10e}",
111                                 f"{delta_e[i].real:.10e}",
                                    ↪ f"{delta_e[i].imag:.10e}"])
112     print(f"Matriz de error guardada en {path}")
113
114 def cargar_calibracion_csv(path): #rearma los valores complejos
115     freqs, e_00, e_11, delta_e = [], [], [], []
116     with open(path, "r", encoding="utf-8") as f:
117         reader=csv.DictReader(f)
118         for row in reader:
119             freqs.append(float(row["freq_MHz"]))
120             e_00.append(complex(float(row["e_00_real"]),
                ↪ float(row["e_00_imag"])))
121             e_11.append(complex(float(row["e_11_real"]),
                ↪ float(row["e_11_imag"])))
122             delta_e.append(complex(float(row["delta_e_real"]),
                ↪ float(row["delta_e_imag"])))
123     return np.array(freqs), np.array(e_00), np.array(e_11), np.array(delta_e)
124
125 # ==================== Arduino / serial ====================
126
127 def generar_frecuencias(inicio, fin, paso):
128     """Genera la lista de frecuencias evitando arrastre de error de punto
        ↪ flotante."""
129     n_pasos = round((fin - inicio) / paso)
130     return [inicio + i * paso for i in range(n_pasos + 1)]
131
132
133 def conectar_arduino(puerto, baudrate):
134     ser = serial.Serial(puerto, baudrate, timeout=5)
135     time.sleep(2)  # el Arduino se resetea al abrir el puerto; esperamos a que
        ↪ arranque
136     ser.reset_input_buffer()
137
138     linea = ser.readline().decode(errors="ignore").strip()
139     if linea:
140         print(f"Arduino: {linea}")
141
142     return ser
143
144
145 def setear_frecuencia(ser, f_mhz):
146     """Manda la frecuencia al Arduino y espera la confirmación."""
147     comando = f"{f_mhz:.6f}\n"
148     ser.write(comando.encode())
149
150     respuesta = ser.readline().decode(errors="ignore").strip()
151     while respuesta and not respuesta.startswith("OK") and not
        ↪ respuesta.startswith("ERROR"):
152         print("  (Arduino):", respuesta)
153         respuesta = ser.readline().decode(errors="ignore").strip()
```

```
154
155     return respuesta
156
157
158 # ==================== Adquisición ====================
159
160 def capturar_audio(duracion_s, fs, dispositivo=None):
161     """
162     Graba desde la entrada configurada. Devuelve (canal_ref, canal_med)
163     como arrays float64, con canal_ref = izquierdo, canal_med = derecho.
164     """
165
166     n_muestras = int(duracion_s* fs)
167     grabacion_con_ruido = sd.rec(n_muestras, samplerate=fs, channels=2,
168                         dtype="float64", device=dispositivo)
169     sd.wait()
170
171     grabacion=grabacion_con_ruido[int(fs * DESCARTE_S):] # recorto los primeros
      ↪  milisegundos
172     canal_ref = grabacion[:, 0]
173     canal_med = grabacion[:, 1]
174
175     return canal_ref, canal_med
176
177
178 # ==================== Filtro FIR + FFT + S11 ====================
179
180 def diseñar_filtro_pasabanda(fs, f_centro, ancho_banda, numtaps):
181     n_muestras_captura = int(DURACION_CAPTURA_S * fs)
182     if numtaps >= n_muestras_captura:
183         raise ValueError(
184             f"NUMTAPS_FILTRO ({numtaps}) debe ser menor que la cantidad de "
185             f"muestras por captura ({n_muestras_captura}, con
              ↪  DURACION_CAPTURA_S="
186             f"{DURACION_CAPTURA_S}s y FS_AUDIO={fs}Hz). Bajá NUMTAPS_FILTRO o "
187             f"subí DURACION_CAPTURA_S."
188         )
189
190     f_baja = f_centro - ancho_banda / 2
191     f_alta = f_centro + ancho_banda / 2
192     return firwin(numtaps, [f_baja, f_alta], pass_zero=False, fs=fs,
      ↪  window="hamming")
193
194
195 def aplicar_filtro(señal, coeficientes):
196     return lfilter(coeficientes, 1.0, señal)
197
198
199 def calcular_espectro_complejo(señal, fs, ventana):
200     n = len(señal)
201     w = get_window(ventana, n)
202     fft_vals = np.fft.fft(señal * w)
```

```
203     fft_freqs = np.fft.fftfreq(n, d=1 / fs)
204     mitad = n // 2
205     return fft_freqs[:mitad], fft_vals[:mitad]
206 
207 
208 def buscar_indice_pico(freqs, espectro, banda):
209     """Devuelve el índice del bin de mayor magnitud dentro de la banda dada."""
210     f_baja, f_alta = banda
211     i0 = np.searchsorted(freqs, f_baja)
212     i1 = np.searchsorted(freqs, f_alta)
213 
214     if i1 <= i0:
215         raise ValueError("La banda de búsqueda no contiene ningún bin de
            ↪  frecuencia.")
216 
217     magnitud = np.abs(espectro[i0:i1])
218     return i0 + np.argmax(magnitud)
219 
220 
221 def medir_s11_punto(canal_ref, canal_med, fs, coef_filtro):
222     """
223     A partir de una captura de referencia y medición, devuelve un único
224     valor complejo de S11 correspondiente al tono de batido.
225     """
226     ref_filtrada = aplicar_filtro(canal_ref, coef_filtro)
227     med_filtrada = aplicar_filtro(canal_med, coef_filtro)
228 
229     freqs_ref, espectro_ref = calcular_espectro_complejo(ref_filtrada, fs,
        ↪  VENTANA_FFT)
230     freqs_med, espectro_med = calcular_espectro_complejo(med_filtrada, fs,
        ↪  VENTANA_FFT)
231 
232     idx_pico = buscar_indice_pico(freqs_ref, espectro_ref, BANDA_BUSQUEDA_PICO)
233 
234     epsilon = np.max(np.abs(espectro_ref)) * 1e-9
235     s11 = np.conj(espectro_med[idx_pico] / (espectro_ref[idx_pico] + epsilon))
236 
237     return s11, freqs_ref[idx_pico]
238 
239 
240 # ==================== Barrido ====================
241 
242 def barrido(ser,frecuencias_rf, coef_filtro, nombre_medida="",
    ↪  calibracion=None):
243     print(f"\n---> Iniciando adquisición: {nombre_medida}")
244     resultados_freq_rf = []
245     resultados_s11_complejo = []
246 
247     for i, f_rf in enumerate(frecuencias_rf):
248         respuesta = setear_frecuencia(ser, f_rf)
249 
250         if respuesta.startswith("ERROR"):
```

```
251             print(f"[{i + 1}/{len(frecuencias_rf)}] {respuesta} -- se aborta el
                ↪  barrido.")
252             break
253
254         canal_ref, canal_med = capturar_audio(DURACION_CAPTURA_S, FS_AUDIO,
            ↪  DISPOSITIVO_AUDIO)
255
256         s11, f_batido = medir_s11_punto(canal_ref, canal_med, FS_AUDIO,
            ↪  coef_filtro)
257
258         resultados_freq_rf.append(f_rf)
259         resultados_s11_complejo.append(s11)
260
261         gamma_mostrar = s11
262         etiqueta_modulo = "|S11|"
263         etiqueta_fase = "fase"
264         if calibracion is not None:
265             idx = len(resultados_s11_complejo) - 1
266             e_00, e_11, delta_e = calibracion
267             if idx < min(len(e_00), len(e_11), len(delta_e)):
268                 gamma_mostrar = correccion_punto(
269                     s11, e_00[idx], e_11[idx], delta_e[idx]
270                 )
271                 etiqueta_modulo = "|S11| calibrado"
272                 etiqueta_fase = "fase calibrada"
273             else:
274                 continue
275
276         modulo_db = 20 * np.log10(np.abs(gamma_mostrar) + 1e-12)
277         fase_deg = np.degrees(np.angle(gamma_mostrar))
278         print(f"[{i + 1}/{len(frecuencias_rf)}] RF={f_rf:.3f} MHz  "
279               f"batido={f_batido:.1f} Hz  {etiqueta_modulo}={modulo_db:.2f} dB
                  ↪  "
280               f"{etiqueta_fase}={fase_deg:.1f}°")
281     return np.array(resultados_freq_rf), np.array(resultados_s11_complejo)
282
283
284 def guardar_csv(path, freq_rf, modulo_db, fase_deg):
285     with open(path, "w", newline="", encoding="utf-8") as f:
286         writer = csv.writer(f)
287         writer.writerow(["freq_rf_MHz", "modulo_S11_dB", "fase_S11_deg"])
288         for f_rf, m, fa in zip(freq_rf, modulo_db, fase_deg):
289             writer.writerow([f"{f_rf:.6f}", f"{m:.4f}", f"{fa:.4f}"])
290     print(f"Resultados guardados en {path}")
291
292
293 def graficar_resultado(freq_rf, modulo_db, fase_deg):
294     fig, axs = plt.subplots(2, 1, figsize=(10, 8), sharex=True)
295
296     axs[0].plot(freq_rf, modulo_db, color="deeppink", marker=".",
        ↪  linestyle="None")
297     axs[0].set_title("S11 - Módulo")
```

```
298     axs[0].set_ylabel("|S11| (dB)")
299     axs[0].grid(True, alpha=0.3)
300 
301     axs[1].plot(freq_rf, fase_deg, color="darkmagenta", marker=".",
        ↪  linestyle="None")
302     axs[1].set_title("S11 - Fase")
303     axs[1].set_xlabel("Frecuencia de RF (MHz)")
304     axs[1].set_ylabel("Fase (grados)")
305     axs[1].grid(True, alpha=0.3)
306 
307     elementos_leyenda = [
308         Line2D([], [], color="none", label=f"F inicio: {F_INICIO_MHZ:.3f} MHz"),
309         Line2D([], [], color="none", label=f"F fin: {F_FIN_MHZ:.3f} MHz"),
310         Line2D([], [], color="none", label=f"F paso: {F_PASO_MHZ:.3f} MHz"),
311         Line2D([], [], color="none", label=f"F centro filtro:
            ↪  {F_CENTRO_FILTRO:.0f} Hz"),
312     ]
313     fig.legend(handles=elementos_leyenda, loc="lower center", ncol=2,
314                frameon=True)
315     plt.tight_layout(rect=(0, 0.10, 1, 1))
316 
317     ventana = tk.Tk()
318     ventana.withdraw()
319     try:
320         medicion = simpledialog.askstring(
321             "Nombre de la medición",
322             "¿A qué medición corresponde esta figura?\n"
323             "El nombre se agregará al archivo .pkl:"
324         )
325     finally:
326         ventana.destroy()
327 
328     if medicion and medicion.strip():
329         nombre_medicion = re.sub(r"[<>:\"/\\|?*]", "_", medicion.strip())
330         archivo_figura = f"{NOMBRE_FIGURA_BASE}_{nombre_medicion}.pkl"
331         try:
332             with open(archivo_figura, "wb") as f:
333                 pickle.dump(fig, f)
334             print(f"Figura interactiva guardada en {archivo_figura} "
335                   f"(reabrila con abrir_figura.py)")
336         except Exception as e:
337             print(f"AVISO: no se pudo guardar la figura con pickle ({e}). "
338                   "El CSV con los datos sigue disponible igual.")
339     else:
340         print("AVISO: no se guardó la figura porque no se indicó la medición.")
341 
342     plt.show()
343 
344 def main():
345     os.makedirs(CARPETA_CSV, exist_ok=True)
346 
347     frecuencias_rf = generar_frecuencias(F_INICIO_MHZ, F_FIN_MHZ, F_PASO_MHZ)
```

```
348     print(f"Barrido: {len(frecuencias_rf)} pasos, de {F_INICIO_MHZ} a {F_FIN_MHZ} MHz "
349           f"(paso {F_PASO_MHZ} MHz)")
350 
351     coef_filtro = diseñar_filtro_pasabanda(
352         FS_AUDIO, F_CENTRO_FILTRO, ANCHO_BANDA_FILTRO, NUMTAPS_FILTRO
353     )
354 
355     hacer_cal = True
356 
357     # 1. Comprobar si existe la matriz de calibración para este barrido
358     if os.path.exists(archivo_calibracion):
359         print(f"\nSe encontró un archivo de calibración coincidente: '{archivo_calibracion}'")
360         rta = input("¿Deseas usar esta calibración previa (s) o realizar una nueva calibración SOL (n)? [s/n]: ")
361         if rta.strip().lower() == 's':
362             hacer_cal = False
363 
364             freqs_cal, e_00, e_11, delta_e = cargar_calibracion_csv(archivo_calibracion)
365             print("Matriz de error cargada correctamente.")
366 
367     ser = conectar_arduino(PUERTO, BAUDRATE)
368 
369 # 3. Procedimiento de calibración SOL si corresponde
370     if hacer_cal:
371         print("\n==========================================")
372         print("   INICIANDO CALIBRACIÓN S.O.L.")
373         print("==========================================")
374 
375         input("Paso 1/3: Conecte la carga SHORT (Cortocircuito) y presione Enter para barrer...")
376         freqs_cal, gm_cc = barrido(ser, frecuencias_rf, coef_filtro, "CALIBRACIÓN SHORT")
377 
378         input("\nPaso 2/3: Conecte la carga OPEN (Circuito Abierto) y presione Enter para barrer...")
379         _, gm_ca = barrido(ser, frecuencias_rf, coef_filtro, "CALIBRACIÓN OPEN")
380 
381         input("\nPaso 3/3: Conecte la carga LOAD (50 Ohms) y presione Enter para barrer...")
382         _, gm_50 = barrido(ser, frecuencias_rf, coef_filtro, "CALIBRACIÓN LOAD")
383 
384         # Calcular los términos de error con la función de álgebra lineal
385         n_puntos = len(freqs_cal)
386         e_00, e_11, delta_e = calculo_errores(gm_cc, gm_ca, gm_50, n_puntos)
387 
388         # Guardar en archivo .csv
389         guardar_calibracion_csv(archivo_calibracion, freqs_cal, e_00, e_11, delta_e)
390 
```

```
    # 4. Medición del Dispositivo Bajo Prueba (DUT)
    print("\n========================================")
    print("   MEDICIÓN DEL DISPOSITIVO (DUT)")
    print("========================================")
    input("Conecte el dispositivo a medir y presione Enter para iniciar el
    ↪  barrido final...")

    freqs_dut, gm_dut = barrido(
        ser, frecuencias_rf, coef_filtro, "MEDICIÓN DUT",
        calibracion=(e_00, e_11, delta_e),
    )

    ser.close()

     # Chequeo de consistencia (por si cambiaron las frecuencias excluidas entre
     ↪  runs)
    if len(freqs_dut) != len(e_00):
        print("\n¡ADVERTENCIA! El número de puntos no coincide con la
        ↪  calibración. Mal el rango de frecuencias")

     # 6. Aplicar la corrección SOL al DUT
    gamma_corregido = correccion(gm_dut, e_00, e_11, delta_e, len(gm_dut))

    fase_deg = np.degrees(np.angle(gamma_corregido))
    modulo_db = 20 * np.log10(np.abs(gamma_corregido) + 1e-12)

    guardar_csv(ARCHIVO_CSV, freqs_dut, modulo_db, fase_deg)
    graficar_resultado(freqs_dut, modulo_db, fase_deg)

main()
```

6.4 Interfaz de usuario

6.4.1 Core

```
"""
vna_core.py
============
Nucleo de adquisicion y calculo para el VNA casero (2x ADF4351 + Arduino +
placa de sonido). Adaptado de VNA_calibrado.py para poder ser llamado desde
una interfaz grafica: sin input()/tkinter bloqueantes, con callback de
progreso por punto y posibilidad de abortar un barrido en curso.

La logica de senales (filtro FIR, FFT, deteccion del tono de batido,
algebra de calibracion SOL) es la misma que la del script original; lo
unico que cambia es como se invoca.

Requisitos:
    pip install pyserial sounddevice numpy scipy
"""

import csv
```

```python
import time

import numpy as np
import serial
import sounddevice as sd
from scipy.signal import firwin, lfilter
from scipy.signal.windows import get_window

# ==================== Configuracion por defecto ====================

def config_por_defecto():
    """Devuelve un dict de configuracion editable desde la GUI."""
    return {
        "fs_audio": 44100,
        "duracion_captura_s": 0.6,
        "recorte_inicial_s": 0.150,
        "dispositivo_audio": None,
        "f_centro_filtro": 10000,      # Hz, tono de batido esperado
        "ancho_banda_filtro": 500,     # Hz
        "numtaps_filtro": 1001,
        "ventana_fft": "flattop",

    }


def banda_busqueda_pico(config):
    fc = config["f_centro_filtro"]
    bw = config["ancho_banda_filtro"]
    return (fc - bw / 2, fc + bw / 2)


# ==================== Arduino / serial ====================

class ErrorArduino(Exception):
    pass


def conectar_arduino(puerto, baudrate=115200, timeout=5):
    """Abre el puerto serie y devuelve (objeto_serial,
    ↪ primera_linea_recibida)."""
    ser = serial.Serial(puerto, baudrate, timeout=timeout)
    time.sleep(2)  # el Arduino se resetea al abrir el puerto; esperamos a que
    ↪ arranque
    ser.reset_input_buffer()
    linea = ser.readline().decode(errors="ignore").strip()
    return ser, linea


def setear_frecuencia(ser, f_mhz):
    """Manda la frecuencia al Arduino y espera la confirmacion (OK/ERROR)."""
    comando = f"{f_mhz:.6f}\n"
    ser.write(comando.encode())
```

```
68      respuesta = ser.readline().decode(errors="ignore").strip()
69      intentos_vacios = 0
70      while respuesta and not respuesta.startswith("OK") and not
        ↪ respuesta.startswith("ERROR"):
71          respuesta = ser.readline().decode(errors="ignore").strip()
72          intentos_vacios += 1
73          if intentos_vacios > 50:
74              break
75      return respuesta
76
77
78  def generar_frecuencias(inicio, fin, paso):
79      """Genera la lista de frecuencias evitando arrastre de error de punto
        ↪ flotante."""
80      if paso <= 0:
81          raise ValueError("El paso de frecuencia debe ser mayor que 0.")
82      n_pasos = round((fin - inicio) / paso)
83      return [inicio + i * paso for i in range(n_pasos + 1)]
84
85
86  # ==================== Adquisicion de audio ====================
87
88  def capturar_audio(duracion_s, fs, dispositivo=None, recorte_inicial_s=0.150):
89      """
90      Graba desde la entrada configurada. Devuelve (canal_ref, canal_med) como
91      arrays float64, con canal_ref = izquierdo, canal_med = derecho.
92      """
93      n_muestras = int(duracion_s * fs)
94      grabacion_con_ruido = sd.rec(
95          n_muestras, samplerate=fs, channels=2, dtype="float64",
            ↪ device=dispositivo
96      )
97      sd.wait()
98      grabacion = grabacion_con_ruido[int(fs * recorte_inicial_s):]
99      canal_ref = grabacion[:, 0]
100     canal_med = grabacion[:, 1]
101     return canal_ref, canal_med
102
103
104 # ==================== Filtro FIR + FFT + S11 ====================
105
106 def diseñar_filtro_pasabanda(fs, f_centro, ancho_banda, numtaps,
    ↪ duracion_captura_s):
107     n_muestras_captura = int(duracion_captura_s * fs)
108     if numtaps >= n_muestras_captura:
109         raise ValueError(
110             f"NUMTAPS_FILTRO ({numtaps}) debe ser menor que la cantidad de "
111             f"muestras por captura ({n_muestras_captura}, con
                ↪ duracion_captura_s="
112             f"{duracion_captura_s}s y fs={fs}Hz). Bajar numtaps o subir la
                ↪ duracion."
113         )
```

```
114     f_baja = f_centro - ancho_banda / 2
115     f_alta = f_centro + ancho_banda / 2
116     return firwin(numtaps, [f_baja, f_alta], pass_zero=False, fs=fs,
        ↪  window="hamming")
117
118
119 def aplicar_filtro(señal, coeficientes):
120     return lfilter(coeficientes, 1.0, señal)
121
122
123 def calcular_espectro_complejo(señal, fs, ventana):
124     n = len(señal)
125     w = get_window(ventana, n)
126     fft_vals = np.fft.fft(señal * w)
127     fft_freqs = np.fft.fftfreq(n, d=1 / fs)
128     mitad = n // 2
129     return fft_freqs[:mitad], fft_vals[:mitad]
130
131
132 def buscar_indice_pico(freqs, espectro, banda):
133     """Devuelve el indice del bin de mayor magnitud dentro de la banda dada."""
134     f_baja, f_alta = banda
135     i0 = np.searchsorted(freqs, f_baja)
136     i1 = np.searchsorted(freqs, f_alta)
137     if i1 <= i0:
138         raise ValueError("La banda de busqueda no contiene ningun bin de
            ↪  frecuencia.")
139     magnitud = np.abs(espectro[i0:i1])
140     return i0 + np.argmax(magnitud)
141
142
143 def medir_s11_punto(canal_ref, canal_med, fs, coef_filtro, ventana_fft, banda):
144     """
145     A partir de una captura de referencia y medicion, devuelve un unico
146     valor complejo de S11 (sin corregir) correspondiente al tono de batido,
147     y la frecuencia de batido detectada (Hz).
148     """
149     ref_filtrada = aplicar_filtro(canal_ref, coef_filtro)
150     med_filtrada = aplicar_filtro(canal_med, coef_filtro)
151
152     freqs_ref, espectro_ref = calcular_espectro_complejo(ref_filtrada, fs,
        ↪  ventana_fft)
153     _, espectro_med = calcular_espectro_complejo(med_filtrada, fs, ventana_fft)
154
155     idx_pico = buscar_indice_pico(freqs_ref, espectro_ref, banda)
156
157     epsilon = np.max(np.abs(espectro_ref)) * 1e-9
158     s11 = np.conj(espectro_med[idx_pico] / (espectro_ref[idx_pico] + epsilon))
159
160     return s11, freqs_ref[idx_pico]
161
162
```

```
163 # ==================== Barrido (con callback de progreso + aborto)
    ↪ ====================
164
165 class BarridoAbortado(Exception):
166     """Se lanza cuando should_abort() devuelve True durante un barrido."""
167     pass
168
169
170 def barrido(ser, frecuencias_rf, coef_filtro, config, on_point=None,
    ↪ should_abort=None):
171     """
172     Ejecuta un barrido completo de frecuencias, midiendo un punto de S11
173     (sin corregir) por cada frecuencia.
174
175     on_point(i, n, f_rf, s11, f_batido_hz): callback opcional, llamado
176         despues de medir cada punto (i es 1-based).
177     should_abort(): callable opcional; si devuelve True se aborta el
178         barrido lanzando BarridoAbortado.
179
180     Devuelve (freqs_medidas, s11_medidos) como arrays de numpy, salteando
181     las frecuencias que esten en config["frecuencias_excluidas"].
182     """
183     resultados_freq_rf = []
184     resultados_s11 = []
185     banda = banda_busqueda_pico(config)
186
187     for i, f_rf in enumerate(frecuencias_rf):
188         if should_abort is not None and should_abort():
189             raise BarridoAbortado()
190
191
192         respuesta = setear_frecuencia(ser, f_rf)
193         if respuesta.startswith("ERROR"):
194             raise ErrorArduino(f"Arduino devolvio error en {f_rf:.3f} MHz:
                ↪ {respuesta}")
195
196         canal_ref, canal_med = capturar_audio(
197             config["duracion_captura_s"], config["fs_audio"],
                ↪ config.get("dispositivo_audio"),
198             config.get("recorte_inicial_s", 0.150),
199         )
200         s11, f_batido = medir_s11_punto(
201             canal_ref, canal_med, config["fs_audio"], coef_filtro,
202             config["ventana_fft"], banda,
203         )
204
205         resultados_freq_rf.append(f_rf)
206         resultados_s11.append(s11)
207
208         if on_point is not None:
209             on_point(i + 1, len(frecuencias_rf), f_rf, s11, f_batido)
210
```

```
211     return np.array(resultados_freq_rf), np.array(resultados_s11)
212 
213 
214 # ==================== Calibracion SOL ====================
215 
216 def calculo_errores(GM_CC, GM_CA, GM_50, n):
217     """Resuelve, punto a punto, el sistema 3x3 de la calibracion SOL."""
218     e_00 = np.empty(n, dtype=complex)
219     e_11 = np.empty(n, dtype=complex)
220     delta_e = np.empty(n, dtype=complex)
221 
222     for i in range(n):
223         A = np.array([
224             [1, -GM_CC[i], 1],  # CC (short)
225             [1, 0, 0],          # 50 (load)
226             [1, GM_CA[i], -1],  # CA (open)
227         ], dtype=complex)
228         B = np.array([GM_CC[i], GM_50[i], GM_CA[i]])
229         sol = np.linalg.solve(A, B)
230         e_00[i] = sol[0]
231         e_11[i] = sol[1]
232         delta_e[i] = sol[2]
233 
234     return e_00, e_11, delta_e
235 
236 
237 def correccion(GM, e_00, e_11, delta_e, n):
238     """Aplica la correccion de error de la calibracion SOL a un array de GM
    ↪ medidos."""
239     Gamma = np.empty(n, dtype=complex)
240     for i in range(n):
241         denominador = (GM[i] * e_11[i] - delta_e[i])
242         if abs(denominador) < 1e-15:
243             denominador = 1e-15 + 1e-15j
244         Gamma[i] = (GM[i] - e_00[i]) / denominador
245     return Gamma
246 
247 
248 def correccion_punto(gm, e_00_i, e_11_i, delta_e_i):
249     """Version de correccion() para un unico punto (usada para graficar en
    ↪ vivo)."""
250     denominador = gm * e_11_i - delta_e_i
251     if abs(denominador) < 1e-15:
252         denominador = 1e-15 + 1e-15j
253     return (gm - e_00_i) / denominador
254 
255 
256 def guardar_calibracion_csv(path, freqs, e_00, e_11, delta_e):
257     with open(path, "w", newline="", encoding="utf-8") as f:
258         writer = csv.writer(f)
259         writer.writerow(["freq_MHz", "e_00_real", "e_00_imag", "e_11_real",
            ↪ "e_11_imag",
```

```
                            "delta_e_real", "delta_e_imag"])
        for i in range(len(freqs)):
            writer.writerow([
                f"{freqs[i]:.6f}",
                f"{e_00[i].real:.10e}", f"{e_00[i].imag:.10e}",
                f"{e_11[i].real:.10e}", f"{e_11[i].imag:.10e}",
                f"{delta_e[i].real:.10e}", f"{delta_e[i].imag:.10e}",
            ])


def cargar_calibracion_csv(path):
    """Rearma los valores complejos guardados por guardar_calibracion_csv()."""
    freqs, e_00, e_11, delta_e = [], [], [], []
    with open(path, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            freqs.append(float(row["freq_MHz"]))
            e_00.append(complex(float(row["e_00_real"]),
            ↪  float(row["e_00_imag"])))
            e_11.append(complex(float(row["e_11_real"]),
            ↪  float(row["e_11_imag"])))
            delta_e.append(complex(float(row["delta_e_real"]),
            ↪  float(row["delta_e_imag"])))
    return np.array(freqs), np.array(e_00), np.array(e_11), np.array(delta_e)


def guardar_resultados_csv(path, freq_rf, modulo_db, fase_deg):
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["freq_rf_MHz", "modulo_S11_dB", "fase_S11_deg"])
        for f_rf, m, fa in zip(freq_rf, modulo_db, fase_deg):
            writer.writerow([f"{f_rf:.6f}", f"{m:.4f}", f"{fa:.4f}"])


def modulo_fase(s11_array):
    """Convierte un array de S11 complejos a (modulo_dB, fase_grados)."""
    modulo_db = 20 * np.log10(np.abs(s11_array) + 1e-12)
    fase_deg = np.degrees(np.angle(s11_array))
    return modulo_db, fase_deg
```

6.4.2 Graphical User Interface

```
"""
vna_gui.py
==========
Interfaz grafica (PyQt6) para el VNA casero: permite elegir frecuencia de
inicio/fin/paso, conectar el Arduino, correr la calibracion SOL (short /
open / load) paso a paso, medir un DUT, y visualizar en vivo el modulo y
la fase de S11 junto con el abaco de Smith.

Requisitos:
    pip install PyQt6 pyserial sounddevice numpy scipy matplotlib

```

```
Ejecutar con:
    python vna_gui.py
"""

import os
import sys

import numpy as np

from PyQt6.QtCore import Qt, QThread, pyqtSignal
from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
    QGroupBox, QLabel, QLineEdit, QPushButton, QDoubleSpinBox,
    QProgressBar, QFileDialog, QMessageBox, QPlainTextEdit, QSplitter,
)

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas
from matplotlib.figure import Figure

import vna_core_FINAL as core


# ==================== Hilo de trabajo: corre un barrido sin trabar la GUI ====================

class SweepWorker(QThread):
    """Ejecuta core.barrido() en un hilo aparte y reenvia sus resultados como señales Qt."""

    punto_medido = pyqtSignal(int, int, float, complex, float)  # i, n, f_rf, s11, f_batido
    terminado = pyqtSignal(object, object)                      # freqs (ndarray), s11 (ndarray)
    abortado = pyqtSignal()
    error = pyqtSignal(str)

    def __init__(self, ser, frecuencias_rf, coef_filtro, config, parent=None):
        super().__init__(parent)
        self.ser = ser
        self.frecuencias_rf = frecuencias_rf
        self.coef_filtro = coef_filtro
        self.config = config
        self._abortar = False

    def abortar(self):
        self._abortar = True

    def run(self):
        try:
            freqs, s11 = core.barrido(
                self.ser, self.frecuencias_rf, self.coef_filtro, self.config,
```

```
59                  on_point=lambda i, n, f, s, fb: self.punto_medido.emit(i, n, f,
                    ↪  s, fb),
60                  should_abort=lambda: self._abortar,
61              )
62              self.terminado.emit(freqs, s11)
63          except core.BarridoAbortado:
64              self.abortado.emit()
65          except Exception as e:  # noqa: BLE001 - se muestra en la GUI
66              self.error.emit(str(e))
67
68
69  # ==================== Dibujo del abaco de Smith (fondo estatico)
    ↪  ====================
70
71  def dibujar_fondo_smith(ax):
72      """Dibuja el grillado clasico de un abaco de Smith (r y x constantes)."""
73      ax.clear()
74      ax.set_aspect("equal")
75      ax.axis("off")
76      ax.set_xlim(-1.15, 1.15)
77      ax.set_ylim(-1.15, 1.15)
78      ax.set_title("Abaco de Smith")
79
80      theta = np.linspace(0, 2 * np.pi, 400)
81
82      # Circulo unitario (|Gamma| = 1)
83      ax.plot(np.cos(theta), np.sin(theta), color="black", linewidth=1.0)
84      ax.axhline(0, color="gray", linewidth=0.5)
85
86      # Circulos de resistencia constante (r = 0, 0.2, 0.5, 1, 2, 5)
87      for r in (0.0, 0.2, 0.5, 1.0, 2.0, 5.0):
88          cx = r / (r + 1)
89          rad = 1 / (r + 1)
90          x = cx + rad * np.cos(theta)
91          y = rad * np.sin(theta)
92          ax.plot(x, y, color="gray", linewidth=0.5)
93
94      # Arcos de reactancia constante (x = +/- 0.2, 0.5, 1, 2, 5)
95      t = np.linspace(0, 2 * np.pi, 800)
96      for x_val in (0.2, 0.5, 1.0, 2.0, 5.0):
97          for signo in (1, -1):
98              xv = signo * x_val
99              cx, cy = 1.0, 1.0 / xv
100             rad = abs(1.0 / xv)
101             gx = cx + rad * np.cos(t)
102             gy = cy + rad * np.sin(t)
103             mask = gx ** 2 + gy ** 2 <= 1.0 + 1e-6
104             ax.plot(gx[mask], gy[mask], color="gray", linewidth=0.5)
105
106
107 # ==================== Ventana principal ====================
108
```

```
109 class MainWindow(QMainWindow):
110     def __init__(self):
111         super().__init__()
112         self.setWindowTitle("VNA - Control y medición de S11")
113         self.resize(1200, 750)
114 
115         self.config = core.config_por_defecto()
116         self.ser = None
117         self.hilo = None
118 
119         # Estado de calibracion
120         self.cal_paso = 0            # 0 = sin empezar, 1=SHORT, 2=OPEN, 3=LOAD,
        ↪  4=lista
121         self.cal_gm_cc = None
122         self.cal_gm_ca = None
123         self.cal_gm_50 = None
124         self.cal_freqs = None
125         self.cal_e00 = None
126         self.cal_e11 = None
127         self.cal_delta_e = None
128         self.cal_disponible = False
129 
130         # Buffers de la medicion en curso (para graficar en vivo)
131         self.buf_freq = []
132         self.buf_mod_db = []
133         self.buf_fase_deg = []
134         self.buf_gamma = []
135         self.ultima_medicion_freqs = None
136         self.ultima_medicion_gamma = None
137         self.ultima_medicion_mod_db = None
138         self.ultima_medicion_fase_deg = None
139 
140         self._armar_ui()
141 
142     # ------------------------------------------------------------------ UI --
143 
144     def _armar_ui(self):
145         central = QWidget()
146         self.setCentralWidget(central)
147         layout_principal = QHBoxLayout(central)
148 
149         splitter = QSplitter(Qt.Orientation.Horizontal)
150         layout_principal.addWidget(splitter)
151 
152         panel_izq = self._armar_panel_controles()
153         panel_der = self._armar_panel_graficos()
154 
155         splitter.addWidget(panel_izq)
156         splitter.addWidget(panel_der)
157         splitter.setStretchFactor(0, 0)
158         splitter.setStretchFactor(1, 1)
159 
```

```
160     def _armar_panel_controles(self):
161         panel = QWidget()
162         panel.setMinimumWidth(340)
163         panel.setMaximumWidth(420)
164         v = QVBoxLayout(panel)
165 
166         # --- Conexion Arduino ---
167         gb_conexion = QGroupBox("Conexion Arduino")
168         f = QVBoxLayout(gb_conexion)
169         fila_puerto = QHBoxLayout()
170         fila_puerto.addWidget(QLabel("Puerto:"))
171         self.ed_puerto = QLineEdit("COM3")
172         fila_puerto.addWidget(self.ed_puerto)
173         f.addLayout(fila_puerto)
174 
175         fila_baud = QHBoxLayout()
176         fila_baud.addWidget(QLabel("Baudrate:"))
177         self.ed_baudrate = QLineEdit("115200")
178         fila_baud.addWidget(self.ed_baudrate)
179         f.addLayout(fila_baud)
180 
181         self.btn_conectar = QPushButton("Conectar")
182         self.btn_conectar.clicked.connect(self._conectar_arduino)
183         f.addWidget(self.btn_conectar)
184 
185         self.lbl_estado_conexion = QLabel("Desconectado")
186         f.addWidget(self.lbl_estado_conexion)
187         v.addWidget(gb_conexion)
188 
189         # --- Barrido ---
190         gb_barrido = QGroupBox("Barrido de frecuencia")
191         f2 = QVBoxLayout(gb_barrido)
192 
193         self.sp_f_inicio = self._nuevo_spin(1800.0, 0, 6000, " MHz")
194         self.sp_f_fin = self._nuevo_spin(2200.0, 0, 6000, " MHz")
195         self.sp_f_paso = self._nuevo_spin(1.0, 0.001, 1000, " MHz")
196 
197         f2.addWidget(self._fila("F inicio:", self.sp_f_inicio))
198         f2.addWidget(self._fila("F fin:", self.sp_f_fin))
199         f2.addWidget(self._fila("F paso:", self.sp_f_paso))
200         v.addWidget(gb_barrido)
201 
202         # --- Calibracion SOL ---
203         gb_cal = QGroupBox("Calibracion SOL")
204         f3 = QVBoxLayout(gb_cal)
205 
206         self.lbl_estado_cal = QLabel("Sin calibrar")
207         f3.addWidget(self.lbl_estado_cal)
208 
209         self.btn_cal_iniciar = QPushButton("Iniciar calibracion nueva")
210         self.btn_cal_iniciar.clicked.connect(self._iniciar_calibracion)
211         f3.addWidget(self.btn_cal_iniciar)
```

```
212
213         self.btn_cal_paso = QPushButton("Barrer paso actual")
214         self.btn_cal_paso.setEnabled(False)
215         self.btn_cal_paso.clicked.connect(self._barrer_paso_calibracion)
216         f3.addWidget(self.btn_cal_paso)
217
218         self.btn_cal_cargar = QPushButton("Cargar calibracion desde archivo...")
219         self.btn_cal_cargar.clicked.connect(self._cargar_calibracion)
220         f3.addWidget(self.btn_cal_cargar)
221
222         self.btn_cal_guardar = QPushButton("Guardar calibracion actual...")
223         self.btn_cal_guardar.setEnabled(False)
224         self.btn_cal_guardar.clicked.connect(self._guardar_calibracion)
225         f3.addWidget(self.btn_cal_guardar)
226
227         v.addWidget(gb_cal)
228
229         # --- Medicion DUT ---
230         gb_dut = QGroupBox("Medición del DUT")
231         f4 = QVBoxLayout(gb_dut)
232
233         self.btn_medir = QPushButton("Medir DUT")
234         self.btn_medir.setEnabled(False)
235         self.btn_medir.clicked.connect(self._iniciar_medicion_dut)
236         f4.addWidget(self.btn_medir)
237
238         self.btn_abortar = QPushButton("Abortar barrido")
239         self.btn_abortar.setEnabled(False)
240         self.btn_abortar.clicked.connect(self._abortar_barrido)
241         f4.addWidget(self.btn_abortar)
242
243         self.progreso = QProgressBar()
244         f4.addWidget(self.progreso)
245
246         self.btn_guardar_resultados = QPushButton("Guardar resultados CSV...")
247         self.btn_guardar_resultados.setEnabled(False)
248         self.btn_guardar_resultados.clicked.connect(self._guardar_resultados)
249         f4.addWidget(self.btn_guardar_resultados)
250
251         v.addWidget(gb_dut)
252
253         # --- Log ---
254         gb_log = QGroupBox("Log")
255         f5 = QVBoxLayout(gb_log)
256         self.txt_log = QPlainTextEdit()
257         self.txt_log.setReadOnly(True)
258         f5.addWidget(self.txt_log)
259         v.addWidget(gb_log, stretch=1)
260
261         return panel
262
263     def _armar_panel_graficos(self):
```

```
264         panel = QWidget()
265         v = QVBoxLayout(panel)
266 
267         # Figura módulo + fase (dos subplots apilados)
268         self.fig_mf = Figure(figsize=(6, 5), facecolor="#ffffff")
269         self.ax_mod = self.fig_mf.add_subplot(211)
270         self.ax_fase = self.fig_mf.add_subplot(212, sharex=self.ax_mod)
271         self._preparar_ejes_mod_fase()
272         self.canvas_mf = FigureCanvas(self.fig_mf)
273 
274         # Figura abaco de Smith
275         self.fig_smith = Figure(figsize=(5, 5), facecolor="#ffffff")
276         self.ax_smith = self.fig_smith.add_subplot(111)
277         dibujar_fondo_smith(self.ax_smith)
278         (self.linea_smith,) = self.ax_smith.plot([], [], color="deeppink",
279                                                  marker=".", linestyle="None")
280         self.canvas_smith = FigureCanvas(self.fig_smith)
281 
282         fila_graficos = QHBoxLayout()
283         fila_graficos.addWidget(self.canvas_mf, stretch=1)
284         fila_graficos.addWidget(self.canvas_smith, stretch=1)
285         v.addLayout(fila_graficos)
286 
287         return panel
288 
289     def _preparar_ejes_mod_fase(self):
290         self.ax_mod.clear()
291         self.ax_mod.set_facecolor("#ffffff")
292         self.ax_mod.set_title("S11 - Módulo")
293         self.ax_mod.set_ylabel("|S11| (dB)")
294         self.ax_mod.grid(True, alpha=0.3)
295         (self.linea_mod,) = self.ax_mod.plot([], [], color="deeppink",
296                                              marker=".", linestyle="None")
297 
298         self.ax_fase.clear()
299         self.ax_fase.set_facecolor("#ffffff")
300         self.ax_fase.set_title("S11 - Fase")
301         self.ax_fase.set_xlabel("Frecuencia de RF (MHz)")
302         self.ax_fase.set_ylabel("Fase (grados)")
303         self.ax_fase.grid(True, alpha=0.3)
304         (self.linea_fase,) = self.ax_fase.plot([], [], color="darkmagenta",
305                                                marker=".", linestyle="None")
306         self.fig_mf.tight_layout()
307 
308     @staticmethod
309     def _nuevo_spin(valor, minimo, maximo, sufijo):
310         sp = QDoubleSpinBox()
311         sp.setDecimals(3)
312         sp.setRange(minimo, maximo)
313         sp.setValue(valor)
314         sp.setSuffix(sufijo)
315         return sp
```

```
316
317     @staticmethod
318     def _fila(etiqueta, widget):
319         w = QWidget()
320         h = QHBoxLayout(w)
321         h.setContentsMargins(0, 0, 0, 0)
322         h.addWidget(QLabel(etiqueta))
323         h.addWidget(widget)
324         return w
325
326     def _log(self, texto):
327         self.txt_log.appendPlainText(texto)
328
329     # ------------------------------------------------------------ Conexion --
330
331     def _conectar_arduino(self):
332         puerto = self.ed_puerto.text().strip()
333         try:
334             baudrate = int(self.ed_baudrate.text().strip())
335         except ValueError:
336             QMessageBox.warning(self, "Baudrate invalido", "El baudrate debe ser
                ↪  un numero entero.")
337             return
338
339         try:
340             self.ser, linea = core.conectar_arduino(puerto, baudrate)
341         except Exception as e:
342             QMessageBox.critical(self, "Error de conexion", str(e))
343             self.lbl_estado_conexion.setText("Error de conexion")
344             return
345
346         self.lbl_estado_conexion.setText(f"Conectado ({puerto} @ {baudrate})")
347         if linea:
348             self._log(f"Arduino: {linea}")
349         self._log(f"Conectado a {puerto} @ {baudrate} baud.")
350
351     # ------------------------------------------------------------ Utilidades --
352
353     def _construir_frecuencias(self):
354         inicio = self.sp_f_inicio.value()
355         fin = self.sp_f_fin.value()
356         paso = self.sp_f_paso.value()
357         return core.generar_frecuencias(inicio, fin, paso)
358
359     def _construir_filtro(self):
360         return core.diseñar_filtro_pasabanda(
361             self.config["fs_audio"], self.config["f_centro_filtro"],
362             self.config["ancho_banda_filtro"], self.config["numtaps_filtro"],
363             self.config["duracion_captura_s"],
364         )
365
366     def _validar_listo_para_barrer(self):
```

```
367             if self.ser is None:
368                 QMessageBox.warning(self, "Sin conexion", "Primero conecta el
                    ↪ Arduino.")
369                 return False
370             if self.hilo is not None and self.hilo.isRunning():
371                 QMessageBox.warning(self, "Barrido en curso", "Ya hay un barrido
                    ↪ corriendo.")
372                 return False
373             return True
374
375         def _lanzar_barrido(self, frecuencias, on_terminado, on_abortado=None,
            ↪ on_error=None):
376             coef_filtro = self._construir_filtro()
377             self.hilo = SweepWorker(self.ser, frecuencias, coef_filtro, self.config)
378             self.hilo.punto_medido.connect(self._on_punto_generico)
379             self.hilo.terminado.connect(on_terminado)
380             self.hilo.abortado.connect(on_abortado or self._on_abortado_generico)
381             self.hilo.error.connect(on_error or self._on_error_generico)
382             self.progreso.setValue(0)
383             self.btn_abortar.setEnabled(True)
384             self.hilo.start()
385
386         def _on_punto_generico(self, i, n, f_rf, s11, f_batido):
387             self.progreso.setMaximum(n)
388             self.progreso.setValue(i)
389             mod_db = 20 * np.log10(abs(s11) + 1e-12)
390             fase_deg = np.degrees(np.angle(s11))
391             self._log(f"[{i}/{n}] RF={f_rf:.3f} MHz  batido={f_batido:.1f} Hz  "
392                       f"|S11|={mod_db:.2f} dB  fase={fase_deg:.1f} deg")
393
394         def _on_abortado_generico(self):
395             self.btn_abortar.setEnabled(False)
396             self._log("Barrido abortado por el usuario.")
397
398         def _on_error_generico(self, mensaje):
399             self.btn_abortar.setEnabled(False)
400             QMessageBox.critical(self, "Error durante el barrido", mensaje)
401             self._log(f"ERROR: {mensaje}")
402
403         def _abortar_barrido(self):
404             if self.hilo is not None and self.hilo.isRunning():
405                 self.hilo.abortar()
406
407         # ------------------------------------------------------ Calibracion --
408
409         ETIQUETAS_PASO_CAL = {
410             1: "Paso 1/3: conecte SHORT (cortocircuito) y presione 'Barrer paso
                ↪ actual'.",
411             2: "Paso 2/3: conecte OPEN (circuito abierto) y presione 'Barrer paso
                ↪ actual'.",
412             3: "Paso 3/3: conecte LOAD (50 ohm) y presione 'Barrer paso actual'.",
413         }
```

```
414
415     def _iniciar_calibracion(self):
416         if not self._validar_listo_para_barrer():
417             return
418         self.cal_paso = 1
419         self.cal_gm_cc = self.cal_gm_ca = self.cal_gm_50 = None
420         self.cal_disponible = False
421         self.btn_medir.setEnabled(False)
422         self.btn_cal_guardar.setEnabled(False)
423         self.btn_cal_paso.setEnabled(True)
424         self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[1])
425
426     def _barrer_paso_calibracion(self):
427         if not self._validar_listo_para_barrer():
428             return
429         frecuencias = self._construir_frecuencias()
430         self.btn_cal_paso.setEnabled(False)
431         self.btn_cal_iniciar.setEnabled(False)
432         self._lanzar_barrido(
433             frecuencias,
434             on_terminado=self._on_paso_calibracion_terminado,
435             on_abortado=self._on_calibracion_abortada,
436         )
437
438     def _on_calibracion_abortada(self):
439         self._on_abortado_generico()
440         self.btn_cal_paso.setEnabled(True)
441         self.btn_cal_iniciar.setEnabled(True)
442
443     def _on_paso_calibracion_terminado(self, freqs, gm):
444         self.btn_abortar.setEnabled(False)
445         self.btn_cal_iniciar.setEnabled(True)
446         self.cal_freqs = freqs
447
448         if self.cal_paso == 1:
449             self.cal_gm_cc = gm
450             self.cal_paso = 2
451             self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[2])
452             self.btn_cal_paso.setEnabled(True)
453         elif self.cal_paso == 2:
454             self.cal_gm_ca = gm
455             self.cal_paso = 3
456             self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[3])
457             self.btn_cal_paso.setEnabled(True)
458         elif self.cal_paso == 3:
459             self.cal_gm_50 = gm
460             self._finalizar_calculo_calibracion()
461
462     def _finalizar_calculo_calibracion(self):
463         n = len(self.cal_freqs)
464         if not (len(self.cal_gm_cc) == len(self.cal_gm_ca) ==
        ↪  len(self.cal_gm_50) == n):
```

```
465             QMessageBox.warning(
466                 self, "Inconsistencia",
467                 "Los tres barridos de calibracion (SHORT/OPEN/LOAD) no tienen la "
468                 "misma cantidad de puntos. Revisa que no se haya abortado ninguno "
469                 "a mitad de camino."
470             )
471             self.cal_paso = 0
472             self.btn_cal_paso.setEnabled(False)
473             return
474 
475         self.cal_e00, self.cal_e11, self.cal_delta_e = core.calculo_errores(
476             self.cal_gm_cc, self.cal_gm_ca, self.cal_gm_50, n
477         )
478         self.cal_disponible = True
479         self.cal_paso = 4
480         self.lbl_estado_cal.setText(f"Calibracion lista ({n} puntos).")
481         self.btn_cal_paso.setEnabled(False)
482         self.btn_cal_guardar.setEnabled(True)
483         self.btn_medir.setEnabled(True)
484         self._log("Calibracion SOL calculada correctamente.")
485 
486     def _cargar_calibracion(self):
487         path, _ = QFileDialog.getOpenFileName(
488             self, "Cargar calibracion", "", "CSV (*.csv)"
489         )
490         if not path:
491             return
492         try:
493             freqs, e00, e11, delta_e = core.cargar_calibracion_csv(path)
494         except Exception as e:
495             QMessageBox.critical(self, "Error al cargar", str(e))
496             return
497 
498         self.cal_freqs, self.cal_e00, self.cal_e11, self.cal_delta_e = freqs,
        ↪  e00, e11, delta_e
499         self.cal_disponible = True
500         self.cal_paso = 4
501         self.lbl_estado_cal.setText(f"Calibracion cargada desde archivo
        ↪  ({len(freqs)} puntos).")
502         self.btn_medir.setEnabled(True)
503         self.btn_cal_guardar.setEnabled(False)
504         self._log(f"Calibracion cargada de {path}")
505 
506     def _guardar_calibracion(self):
507         if not self.cal_disponible:
508             return
509         path, _ = QFileDialog.getSaveFileName(
510             self, "Guardar calibracion", "calibracion.csv", "CSV (*.csv)"
511         )
512         if not path:
```

```
513             return
514         core.guardar_calibracion_csv(path, self.cal_freqs, self.cal_e00,
            ↪  self.cal_e11, self.cal_delta_e)
515         self._log(f"Calibracion guardada en {path}")
516 
517     # ------------------------------------------------------ Medicion DUT --
518 
519     def _iniciar_medicion_dut(self):
520         if not self._validar_listo_para_barrer():
521             return
522         if not self.cal_disponible:
523             QMessageBox.warning(self, "Sin calibracion", "Primero completa o
                ↪  carga una calibracion.")
524             return
525 
526         frecuencias = self._construir_frecuencias()
527 
528         # Reiniciar buffers y graficos
529         self.buf_freq, self.buf_mod_db, self.buf_fase_deg, self.buf_gamma = [],
            ↪  [], [], []
530         self._preparar_ejes_mod_fase()
531         self.canvas_mf.draw()
532         dibujar_fondo_smith(self.ax_smith)
533         (self.linea_smith,) = self.ax_smith.plot([], [], color="deeppink",
            ↪  marker=".", linestyle="None")
534         self.canvas_smith.draw()
535 
536         self.btn_medir.setEnabled(False)
537         self.btn_guardar_resultados.setEnabled(False)
538 
539         self._lanzar_barrido(
540             frecuencias,
541             on_terminado=self._on_medicion_dut_terminada,
542             on_abortado=self._on_medicion_dut_abortada,
543         )
544         # Reconectamos punto_medido a un manejador especifico para DUT
545         # (ademas del log generico que ya se conecto en _lanzar_barrido)
546         self.hilo.punto_medido.connect(self._on_punto_dut)
547 
548     def _on_medicion_dut_abortada(self):
549         self._on_abortado_generico()
550         self.btn_medir.setEnabled(True)
551 
552     def _indice_calibracion_valido(self, i_1_based):
553         idx = i_1_based - 1
554         return self.cal_e00 is not None and 0 <= idx < len(self.cal_e00)
555 
556     def _on_punto_dut(self, i, n, f_rf, s11_crudo, f_batido):
557         # Si hay calibracion punto-a-punto disponible, corregimos en vivo.
558         if self._indice_calibracion_valido(i):
559             idx = i - 1
560             gamma = core.correccion_punto(
```

```
561                 s11_crudo, self.cal_e00[idx], self.cal_e11[idx],
                    ↪ self.cal_delta_e[idx]
562             )
563         else:
564             gamma = s11_crudo  # fallback: se corrige todo al final
565 
566         mod_db = 20 * np.log10(abs(gamma) + 1e-12)
567         fase_deg = np.degrees(np.angle(gamma))
568 
569         self.buf_freq.append(f_rf)
570         self.buf_mod_db.append(mod_db)
571         self.buf_fase_deg.append(fase_deg)
572         self.buf_gamma.append(gamma)
573 
574         self.linea_mod.set_data(self.buf_freq, self.buf_mod_db)
575         self.linea_fase.set_data(self.buf_freq, self.buf_fase_deg)
576         for ax in (self.ax_mod, self.ax_fase):
577             ax.relim()
578             ax.autoscale_view()
579         self.canvas_mf.draw_idle()
580 
581         self.linea_smith.set_data([g.real for g in self.buf_gamma],
582                                   [g.imag for g in self.buf_gamma])
583         self.canvas_smith.draw_idle()
584 
585     def _on_medicion_dut_terminada(self, freqs, gm_dut):
586         self.btn_abortar.setEnabled(False)
587         self.btn_medir.setEnabled(True)
588 
589         if len(freqs) != len(self.cal_e00):
590             QMessageBox.warning(
591                 self, "Advertencia de consistencia",
592                 "La cantidad de puntos medidos no coincide con la calibracion "
593                 "(revisa el rango de frecuencias). Se muestran los valores "
594                 "corregidos en vivo, pero puede haber un corrimiento de indice."
595             )
596 
597         n = min(len(freqs), len(self.cal_e00))
598         gamma_corregido = core.correccion(
599             gm_dut[:n], self.cal_e00[:n], self.cal_e11[:n],
                ↪ self.cal_delta_e[:n], n
600         )
601         mod_db, fase_deg = core.modulo_fase(gamma_corregido)
602 
603         self.ultima_medicion_freqs = freqs[:n]
604         self.ultima_medicion_gamma = gamma_corregido
605         self.ultima_medicion_mod_db = mod_db
606         self.ultima_medicion_fase_deg = fase_deg
607 
608         self.btn_guardar_resultados.setEnabled(True)
609         self._log(f"Medicion de DUT finalizada: {n} puntos corregidos.")
610 
```

```
611     def _guardar_resultados(self):
612         if self.ultima_medicion_freqs is None:
613             return
614         path, _ = QFileDialog.getSaveFileName(
615             self, "Guardar resultados", "resultado_S11.csv", "CSV (*.csv)"
616         )
617         if not path:
618             return
619         core.guardar_resultados_csv(
620             path, self.ultima_medicion_freqs,
621             self.ultima_medicion_mod_db, self.ultima_medicion_fase_deg,
622         )
623         self._log(f"Resultados guardados en {path}")
624 
625     # ------------------------------------------------------------- Cierre --
626 
627     def closeEvent(self, event):
628         if self.hilo is not None and self.hilo.isRunning():
629             self.hilo.abortar()
630             self.hilo.wait(2000)
631         if self.ser is not None:
632             try:
633                 self.ser.close()
634             except Exception:
635                 pass
636         event.accept()
637 
638 
639 def main():
640     app = QApplication(sys.argv)
641     app.setStyleSheet("""
642         QMainWindow, QWidget, QGroupBox {background-color: #ffffff; }""")
643     ventana = MainWindow()
644     ventana.show()
645     sys.exit(app.exec())
646 
647 
648 if __name__ == "__main__":
649     main()
```

Apéndices

UNMDP-FI

Carrera: Ingeniería Electrónica

Seminario-Taller para el Diseño de Soluciones Tecnológicas

Victoria Carla Torres

Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia



Versión 2.0

Victoria Carla Torres

Contenido

Acerca del Equipo **3**

Integrantes 3
Director/a 3
Co-Director/a 3

Acerca del Trabajo Final **3**

Descripción de la necesidad o idea 3
Beneficiarios / Clientes / Usuarios 3
Producto/s o entregable/s resultante/s 3
Esquema preliminar de la solución 3
Grado de avance del Trabajo Final 3

Acerca del Equipo

Integrantes

Victoria Carla Torres
Estudiante y becaria en investigación, con una beca de Estudiante Avanzado de la UNMDP otorgada en 2025 y renovada para 2026. Ayudante estudiante en Física 2/B desde 2023 hasta la actualidad. Bachiller en Economía y Administración.

Director/a

Avalos Ribas, Ramiro. Jefe de Trabajos Prácticos, exclusiva.

Co-Director/a

Maximiliano Gluzman, Brian. Ayudante graduado, exclusiva.

Acerca del Trabajo Final

Descripción de la necesidad o idea

Actualmente, el monitoreo de los niveles de glucemia requiere que los pacientes se sometan a métodos invasivos (punciones) que resultan incómodos y dolorosos. Por esta razón, existe una necesidad en el ámbito de la ingeniería de buscar la optimización de un sistema de medición no invasiva de glucemia.
El método se basa en medir de forma indirecta cómo varían las propiedades electromagnéticas de la sangre al ser atravesadas por señales de radiofrecuencia. Ya se han desarrollado en el Grupo de Investigación sistemas con este propósito, basados en un Analizador Vectorial de Redes (VNA) que complementan el sensor desarrollado. Se busca optimizar, compactar y mejorar el rendimiento del VNA actual.

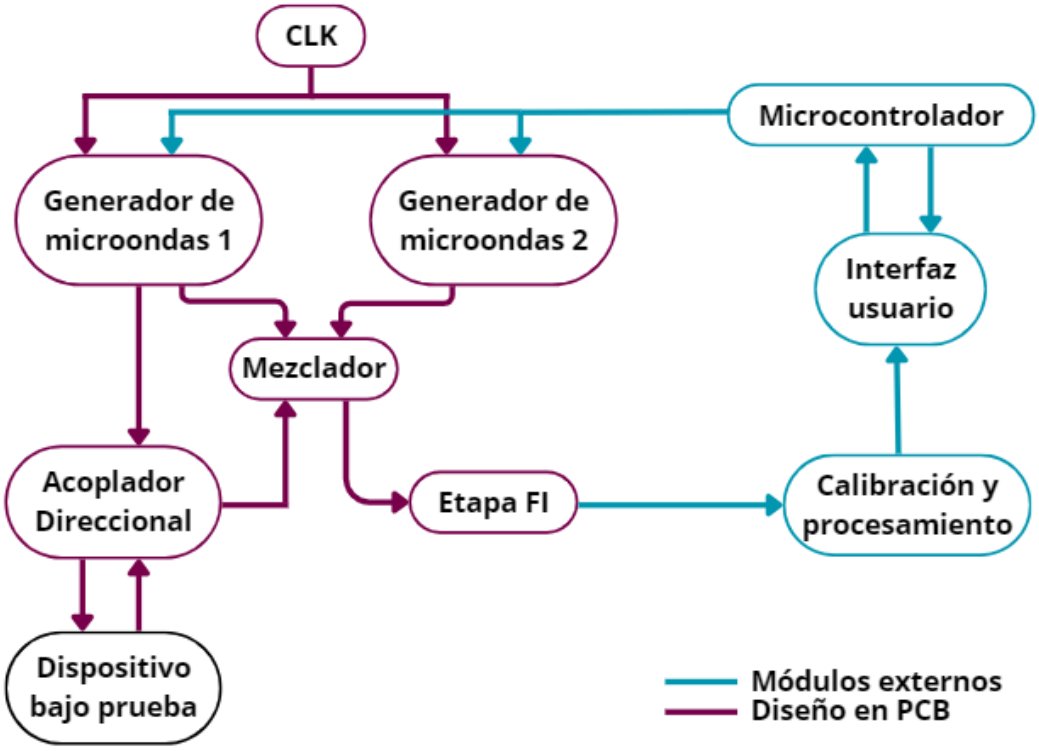
Partes interesadas (stakeholders)

Solicita el desarrollo el Grupo de Investigación del Laboratorio de Comunicaciones. Ellos serán los receptores de la solución propuesta, y por lo tanto los usuarios serán ingenieros electrónicos, o estudiantes bajo supervisión.

Producto/s o entregable/s resultante/s

ε-μVNA: placa de circuito impreso (PCB) que integra los bloques de reloj (CLK), generadores de microondas, acoplador direccional, mezclador y etapa de FI. Con conectores SMA, USB-B y conector de audio analógico, para medición, alimentación y conexión respectivamente.

Esquema preliminar de la solución



Grado de avance del Trabajo Final

Hasta el momento se ha desarrollado la placa con todos sus bloques con el software Altium Designer, y está en proceso de fabricación.
Las actividades pendientes son:

- Ensamblado;
- Implementación del sistema de adquisición, procesamiento y visualización de datos;
- Mediciones para determinar los resultados, funcionamiento, y si se deben hacer ajustes o modificaciones;
- Escribir el informe final.

UNMDP-FI

Carrera: Ingeniería Electrónica

Plan de Trabajo

Victoria Carla Torres

Proyecto: Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia



14/05/2026
Versión 1.0

Introducción

Se presenta la planificación de tareas, objetivos, cronograma de actividades e identificación de riesgos del desarrollo del trabajo final de grado “Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia”

Tareas a realizar

Las tareas a desarrollar para poder finalizar el proyecto son:

- Lectura de bibliografía.
- Diseño de la arquitectura del sistema.
- Análisis de los posibles componentes a utilizar a modo de: oscilador local, mezclador, generadores de RF, acoplador direccional.
- Elección y caracterización de los componentes.
- Diseño y ruteo en PCB.
- Ensamblado de las partes del proyecto.
- Ensayos y pruebas para verificar el funcionamiento.
- Redacción de entregables e informe final.

Cronograma de actividades

El cronograma se presenta en la figura 1 mediante un diagrama de Gantt, donde se detallan las tareas y el tiempo aproximado que se destina a resolverlas. Las figuras con forma de diamante corresponden a hitos del proyecto, explicados en la próxima sección de este documento.

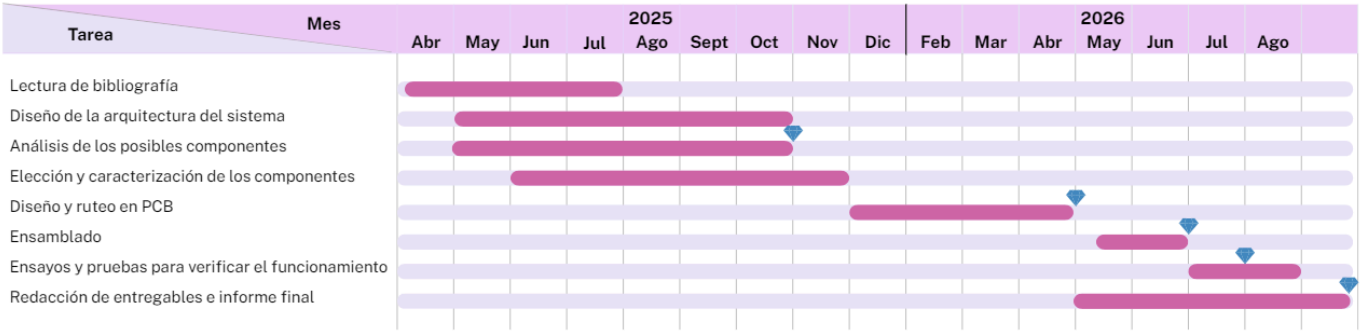


Figura 1: diagrama de Gantt

Hitos del proyecto

Los eventos importantes para marcar el avance del proyecto son:

- Aprobación por parte de los receptores de la arquitectura final y componentes a utilizar.
- Aprobación del diseño de PCB y ruteo.
- Ensamblado de la placa terminado.
- Pruebas exitosas:
    - Que se pueda generar un barrido de señales.

- Que el acoplador direccional presente una impedancia de 50Ω, y los niveles deseados de acoplamiento y directividad.
- Que el microcontrolador sea capaz de pasar los datos al generador correctamente.
- Que la salida del mezclador tenga la señal de FI buscada.

- Aprobación por parte de los directores del informe final.

Asignación de responsables y recursos

En la tabla 1 se observa la distribución de tareas generales, con sus respectivos encargados y medios para cumplirlas.

Tarea	Responsable	Recursos
Simulaciones	Estudiante y profesor	Software de simulación (CST Studio, QucsStudio)
Elección de los componentes	Estudiante y profesor	Hojas de datos, bibliografía
Diseño del PCB	Estudiante	Software Altium Designer Pro, licencia estudiantil.
Ensamblado	Estudiante	Instrumental de laboratorio
Ensayos y pruebas	Estudiante y profesor (supervisión)	Instrumental de laboratorio
Redacción de entregables	Estudiante	Sistemas de composición de texto (Google Docs, LaTex)

Tabla 1: asignación de responsables y recursos

Identificación de riesgos

En la tabla 2 se muestran los posibles riesgos para el cumplimiento del proyecto, junto con la probabilidad de ocurrencia, nivel de impacto en el proyecto, qué consecuencia tendría y qué se debe hacer en caso de que ocurra.

Riesgo	Probabilidad de ocurrencia	Impacto	Consecuencia	Acción reparadora
Inicio trabajo de tiempo completo	Bajo	Alto	Retraso en el cronograma	Redistribución de tiempos
Viaje al exterior por beca de intercambio	Medio bajo	Alto	Retraso en el cronograma	Redistribución de tiempos
Fallas de componentes	Medio	Medio	Retraso en el	Reemplazar

durante las pruebas			cronograma	componentes
Fallas en el diseño del PCB	Medio	Alto	Retraso en el cronograma	Rediseñar
Demora en la impresión del PCB	Medio alto	Medio	Retraso en el cronograma	Búsqueda de nuevos proveedores

Tabla 2: identificación de riesgos

Seguimiento del proyecto

El seguimiento del proyecto se realizará presencialmente en el lugar de trabajo de estudiante y directores, recurriendo a vías digitales (email, videoconferencias) ante cualquier eventualidad que condicione la presencialidad.

UNMDP-FI

Carrera: Ingeniería Electrónica

Especificación de Requerimientos

Victoria Carla Torres

Proyecto:
Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia



30/04/2026

Indice

1. Introducción.....3
1.1 Alcance del proyecto..... 3
1. 2 Definiciones, Abreviaturas y Siglas..... 3
2. Descripción General.....3
2.1 Perspectiva del Producto..... 3
2.1 Funciones del Producto..... 4
2.3 Características del Usuario..... 4
2.4 Datos a Asumir y Dependencias.....4
3. Requerimientos Específicos.....5

1. Introducción

Este documento tiene como propósito definir y describir los requerimientos del proyecto final "Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia" correspondiente a la carrera de Ingeniería Electrónica.

Un Analizador Vectorial de Redes (*Vector Network Analyzer* - VNA) es un instrumento de medición de radiofrecuencia (RF) diseñado para caracterizar el comportamiento de un dispositivo bajo prueba (*Device Under Test* - DUT). Su funcionamiento se basa en inyectar una señal de estímulo y medir su transmisión y reflexión, para extraer los parámetros "S" (*scattering*).

1.1 Alcance del proyecto

El proyecto propone el diseño, desarrollo, calibración y ensayo de un VNA que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 GHz hasta los 2,2 GHz, adaptado para las necesidades de la aplicación. Para lograrlo se utilizan circuitos integrados comerciales, dispositivos de montaje superficial (*Surface-Mount Device* - SMD), y conectores. Se aplican técnicas de reflectometría y de diseño en RF. Para realizar las mediciones se utiliza instrumental de laboratorio provisto por el Laboratorio de Comunicaciones (LAC) de la Facultad de Ingeniería de la Universidad Nacional de Mar del Plata (UNMDP).

1. 2 Definiciones, Abreviaturas y Siglas

Nombre	Descripción
VNA	Analizador Vectorial de Redes
PCB	Printed Circuit Board
RF	Radio Frecuencia
LAC	Laboratorio de Comunicaciones
UNMDP	Universidad Nacional de Mar del Plata
SMD	Surface - Mount Device
SMA	SubMiniature version A
FI	Frecuencia Intermedia
ROE	Razón de Onda Estacionaria
DUT	Device Under Test
UFH	Ultra High Frequency

2. Descripción General

2.1 Perspectiva del Producto

El VNA es un subsistema que forma parte del proyecto de un Grupo de Investigación en el LAC para desarrollar un medidor no invasivo de glucemia, por lo tanto está pensado para que actúe en conjunto con un sensor que actúa de DUT cuando se introduce en él un dedo índice. El principio en el cual se basa el desarrollo es que las propiedades dieléctricas de la sangre varían proporcionalmente a los niveles de glucosa presentes en ella.

La conexión entre estos dos componentes se realiza mediante conectores SMA.

La interfaz de usuario consiste en un programa ejecutable de PC desde el cual se eligen los parámetros y se visualizan los resultados. Se utiliza además la placa de sonido de la PC para adquisición y tratamiento de datos.

2.1 Funciones del Producto

El sistema debe ser capaz de generar un barrido de señales de frecuencias que abarquen desde los 1,8 GHz hasta los 2,2 GHz. Estas se deben inyectar en el DUT con el fin de medir la onda reflejada. Luego, tanto esta señal como una muestra de la señal inducida deben pasar por un proceso de heterodinación, para trasladarlas a una frecuencia intermedia (FI) que permita adquirirlas y digitalizarlas por una etapa de adquisición de audio. En la figura 1 se puede observar un diagrama general del sistema completo.

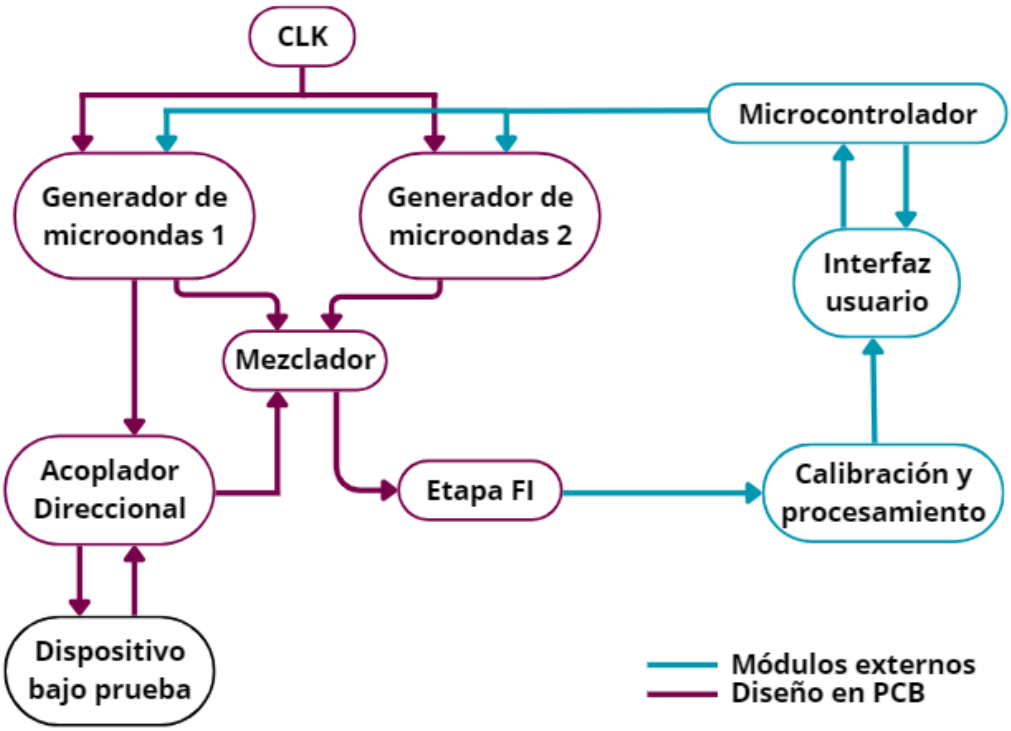


Figura 1: diagrama de bloques VNA

2.3 Características del Usuario

El Grupo de Investigación del LAC es quien solicita el desarrollo. Por esta razón, el equipo se utilizará bajo su supervisión, y sus posibles usuarios son:

- Ingenieros, técnicos y/o radioaficionados;
- Estudiantes de ingeniería.

2.4 Datos a Asumir y Dependencias

Se debe considerar a la hora de diseñar la arquitectura del sistema:

- Que la PC a utilizar debe poseer una placa de sonido con entrada de línea de audio analógica que permita la adquisición, cuya resolución limita rango dinámico y sensibilidad;
- Que se debe hacer una calibración para compensar errores sistemáticos del sistema, como los producidos por pérdidas de inserción en materiales conductores y conectores, desfasajes ocasionados por diferencias en longitudes eléctricos o retardos en los circuitos integrados, así como reflexiones generadas por desadaptación de impedancias;
- Que se deben considerar los comportamientos propios del trabajo en alta frecuencia;
- Que el instrumental disponible en el laboratorio permite medir señales de hasta 3GHz;
- Que los microcontroladores aceptan determinados lenguajes para su programación.

3. Requerimientos Específicos

3.1 Requerimientos funcionales

RF-01 [Ajuste del rango de medición]: el usuario debe poder definir el rango de frecuencias en el que va a realizar la medición, dentro del rango permitido, por medio de la interfaz de usuario.

RF-02 [Escritura de registros]: el microcontrolador debe recibir la información del barrido y calcular los registros que debe mandar a los generadores.

RF-03 [Barrido en frecuencia]: los bloques generadores deben recibir la información y comenzar el barrido de frecuencias.

RF-04 [Separación de onda incidente]: una parte de la potencia de la onda que es generada debe desviarse hacia el mezclador para tener una muestra de la onda incidente.

RF-05 [Heterodinación de señales]: el mezclador debe ser capaz de tomar las señales incidente, reflejada, y la señal externa que actúa como oscilador local y devolver dos señales diferenciales en FI.

RF-06 [Adquisición de audio]: la PC debe tomar las señales y digitalizarlas, devolviendo dos vectores con muestras.

RF-07 [Procesamiento de datos y muestra de resultados]: se debe poder procesar los datos y mostrarlos en un gráfico al usuario.

RF-08 [Rutina de Calibración]: el usuario debe poder realizar la calibración del dispositivo utilizando cargas de impedancias conocidas.

3.2 Requerimientos de performance

RP-01 [Rango de operación]: 1800 a 2200 MHz

RP-02 [Impedancia característica de líneas de transmisión]: 50 Ω.

RP-03 [Resolución del barrido]: los generadores deben permitir una resolución mínima de 1kHz.

RP-04 [Directividad mínima aceptable del acoplador direccional]: 15dB en la banda de interés

RP-05 [ROE mínima distinguible]: menor o igual a 1,5.

RP-06 [Frecuencia intermedia]: 1 a 20 kHz.

3.3 Pautas de diseño y Restricciones

El diseño del sistema está condicionado fuertemente por la naturaleza de las señales de RF. Por esta razón surgen pautas como la elección de utilizar componentes SMD, placas cuyo sustrato tiene ciertas propiedades dieléctricas, el control de impedancias en el diseño de líneas de transmisión, y la utilización de técnicas como el via fencing.

Otras pautas a tener en cuenta son las definidas por los estándares de fabricación de los proveedores, sus capacidades y especificaciones técnicas, que también limitan la elección de materiales y técnicas a utilizar por medio de stackups predefinidos, tolerancias, limitaciones de ancho de pistas y espaciados. A su vez la selección de componentes depende de la disponibilidad y posibilidad de fabricación, bajo el criterio de optimizar el diseño de forma funcional, económica y espacial. En este último aspecto, se debe llegar a una placa de un tamaño menor a 18x17.5 cm^2.

En cuanto a las restricciones en el diseño de la arquitectura, se debe utilizar un oscilador a cristal de 10MHz, dando además la opción de usar un generador de señales de forma auxiliar. Se deben emplear generadores ADF4351 y tensiones de alimentación entre 3,3 y 5V.

Por último se deben considerar las características de los instrumentos de medición disponibles, que limitan la posibilidad de verificar el funcionamiento del sistema en ciertas señales internas a los integrados o en ciertas frecuencias, amplitudes de señales y potencias. Por lo tanto, se deben diseñar puntos de prueba accesibles que garanticen la integridad de las mediciones.

3.4 Atributos

- Confiabilidad: antes de realizar una medición, se debe realizar una calibración con distintas cargas de valores conocidos (típicamente circuito abierto, cortocircuito y 50Ω) a las frecuencias de operación para descartar errores sistemáticos.
- Mantenibilidad: se puede modificar y actualizar el firmware del microcontrolador y la interfaz de usuario, además de reemplazar componentes en caso de fallas, por otros semejantes.

3.5 Requerimientos de interface externa

3.5.1 Interfaces de usuario

El usuario debe poder acceder a una interfaz gráfica que le permita:

- Ajustar parámetros de medición
- Visualizar los resultados

3.5.2 Interfaces de hardware

Las interfaces de hardware presentes en el proyecto son:

- Entre el dispositivo bajo prueba (DUT) y VNA: conector SMA
- Entre la placa de sonido de la PC y la salida del VNA: conector de audio analógico
- Alimentación: USB-B
- Conectores SMA para mediciones de RF auxiliares;
- Conector SMA para entrada de referencia de RF externa.

UNMDP-FI

Carrera: Ingeniería Electrónica

Proyecto:
Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia

Especificación Funcional

Solución



Versión 1.0

Fecha	Versión	Descripción	Autor/a
27/05/2026	1.0	Versión inicial	Victoria Carla Torres

Índice

1. Introducción.... 4
1.1 Propósito del documento.... 4
1.2 Alcance del proyecto.... 4
1.3 Personal involucrado.... 4
1.4 Definiciones, acrónimos y abreviaturas.... 5
1.5 Referencias.... 5
2. Descripción del dispositivo.... 5
3. Especificaciones funcionales.... 6
3.1 RF-01: Ajuste del rango de medición.... 6
3.2 RF-02 Escritura de registros.... 6
3.3 RF-03 Barrido en frecuencia.... 6
3.4 RF-04 Separación de onda incidente.... 6
3.5 RF-05 Heterodinación de señales.... 7
3.6 RF-06 Adquisición de audio.... 7
3.7 RF-07 Procesamiento de datos y muestra de resultados.... 7
3.8 RF-08 Rutina de Calibración.... 7
3.9 RF-09: Alimentación.... 8
4. Requerimientos no funcionales.... 8
4.1 RNF-01: Tamaño de placa de PCB.... 8
4.2 RNF-03: Rango de operación.... 8
4.3 RNF-04: Impedancia característica.... 8
4.4 RNF-05: Resolución del barrido.... 8
4.5 RNF-06: Directividad mínima aceptable del acoplador direccional.... 8
4.6 RNF-07: ROE mínima distinguible.... 8

1. Introducción

Este documento corresponde a la Especificación Funcional para la solución “Analizador Vectorial de Redes para medición no invasiva de glucemia”. Esta especificación se ha estructurado basándose en la información mencionada en el documento de Especificación de Requerimientos (ER) “ER_Torres-v1” .

1.1 Propósito del documento

El presente documento tiene como propósito proveer información detallada de cómo debe funcionar el sistema, cuáles son sus comportamientos deseados y cómo se debe construir, con base en los requerimientos anteriormente definidos en la ER.

Se dirige al desarrollador del proyecto (quien lo va a construir) y a los solicitantes del proyecto, directores y evaluadores.

1.2 Alcance del proyecto

El proyecto propone el diseño, desarrollo, calibración y ensayo de un Analizador Vectorial de Redes (VNA) que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 GHz hasta los 2,2 GHz, adaptado para medición no invasiva de glucemia. Para lograrlo se utilizan circuitos integrados comerciales, dispositivos de montaje superficial (*Surface-Mount Device* - SMD), y conectores. Se aplican técnicas de reflectometría y de diseño en radiofrecuencia. Para realizar las mediciones se utiliza instrumental de laboratorio provisto por el Laboratorio de Comunicaciones (LAC) de la Facultad de Ingeniería de la Universidad Nacional de Mar del Plata (UNMDP).

1.3 Personal involucrado

Nombre	Victoria Carla Torres
Rol	Estudiante
Categoría Profesional	Bachiller
Responsabilidad	Desarrollo, diseño, escritura de documentos
Información de contacto	victoriatorres@mdp.edu.ar

Nombre	Ramiro Avalos Ribas
Rol	Director
Categoría Profesional	Doctor Ingeniero
Responsabilidad	Supervisión, aprobación y revisión del desarrollo
Información de contacto	avalosribas@fi.mdp.edu.ar

Nombre	Brian Maximiliano Gluzman
Rol	Codirector
Categoría Profesional	Ingeniero
Responsabilidad	Supervisión, aprobación y revisión del desarrollo
Información de contacto	briangluzman@mdp.edu.ar

1.4 Definiciones, acrónimos y abreviaturas

Nombre	Descripción
RF	Requerimiento Funcional
RNF	Requerimiento No Funcional
VNA	Analizador Vectorial de Redes
PC	Personal Computer
PCB	Printed Circuit Board
LAC	Laboratorio de Comunicaciones
UNMDP	Universidad Nacional de Mar del Plata
SMD	Surface - Mount Device
SMA	SubMiniature version A
FI	Frecuencia Intermedia
ROE	Razón de Onda Estacionaria
DUT	Device Under Test
UFH	Ultra High Frequency

1.5 Referencias

[1] D. Rytting, “Network analyzer error models and calibration methods,” White paper, Infineon Technologies AG, Santa Clara, CA, USA, Sep. 1998.

2. Descripción del dispositivo

En la Figura 1 se puede observar un diagrama en bloques del sistema propuesto. Se compone de tres subsistemas principales:

- ε-μVNA: placa de PCB (Printed Circuit Board) donde se deben realizar la generación de señales, la medición y la heterodinación para obtener dos señales de una FI (Frecuencia Intermedia) en la banda de audio, una de las cuales contiene información del dispositivo bajo prueba (DUT - Device Under Test)

- PC: allí se deben realizar la adquisición y el procesamiento de las señales, además de alojar la interfaz de usuario. El ε-μVNA se debe comunicar con ella por medio de un conector de audio analógico. A su vez, ésta se debe comunicar con el microcontrolador para indicar los parámetros de configuración de barrido.

- Microcontrolador: se debe encargar de programar los registros de los generadores de radiofrecuencia a partir de las frecuencias elegidas por el usuario en la PC.

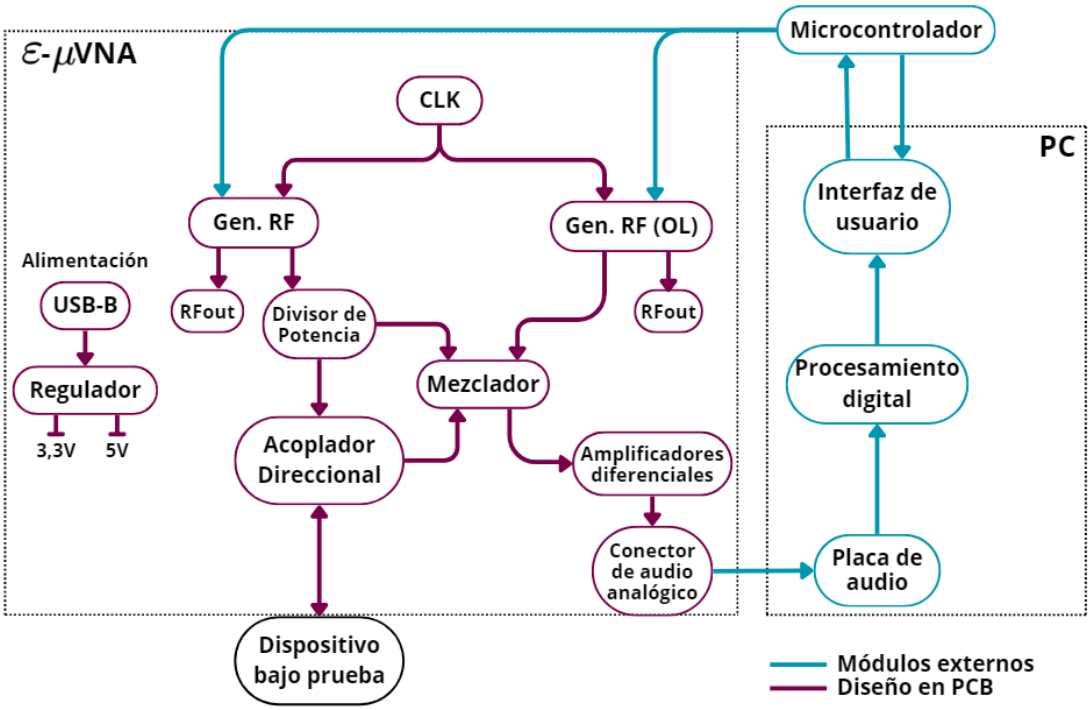


Figura 1: diagrama en bloques del sistema

3. Especificaciones funcionales

3.1 RF-01: Ajuste del rango de medición

El usuario debe poder definir el rango de frecuencias en el que va a realizar la medición, dentro del rango permitido, por medio de la interfaz de usuario.

3.2 RF-02 Escritura de registros

El microcontrolador debe recibir la información del barrido y calcular los registros que debe mandar a los generadores.

3.3 RF-03 Barrido en frecuencia

Los bloques generadores deben guardar los registros y comenzar el barrido de frecuencias definido por el usuario.

3.4 RF-04 Separación de onda incidente

Una parte de la potencia de la onda que es generada debe desviarse hacia el mezclador para tener una muestra de la onda incidente (inc). La otra parte debe ingresar al acoplador direccional para incidir en el DUT, lo que produce una onda reflejada, de menor amplitud, que depende de la impedancia de entrada. Esta onda (ref) se debe dirigir al mezclador como muestra la Figura 2.

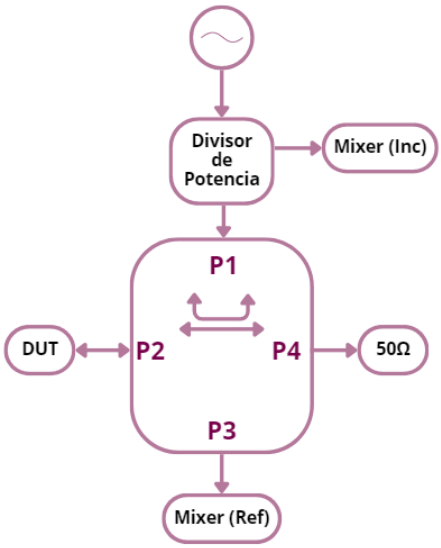


Figura 2: diagrama de la separación de ondas

3.5 RF-05 Heterodinación de señales

El mezclador debe ser capaz de recibir las señales incidente, reflejada, y la señal externa que actúa como oscilador local y devolver dos señales diferenciales en FI, como se puede visualizar en la Figura 3.

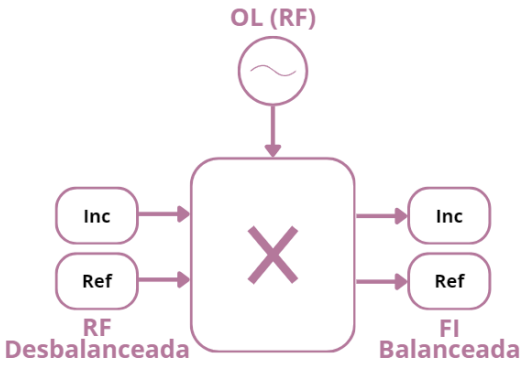


Figura 3: heterodinación de señales

3.6 RF-06 Adquisición de audio

La PC debe adquirir las señales y digitalizarlas, generando dos vectores con sus muestras.

3.7 RF-07 Procesamiento de datos y muestra de resultados

Se debe aplicar un procesamiento a los datos para calcular los parámetros de amplitud y fase de cada señal y generar el parámetro S correspondiente, para todo el barrido de frecuencias. Luego se los debe enviar hacia un bloque de almacenamiento para mostrarse en un gráfico al usuario o bien guardarse para calibración.

3.8 RF-08 Rutina de Calibración

El usuario debe poder realizar la calibración del dispositivo utilizando el método SOL (Short-Open-Load), cuyo modelo es el de la Figura 4 [1].

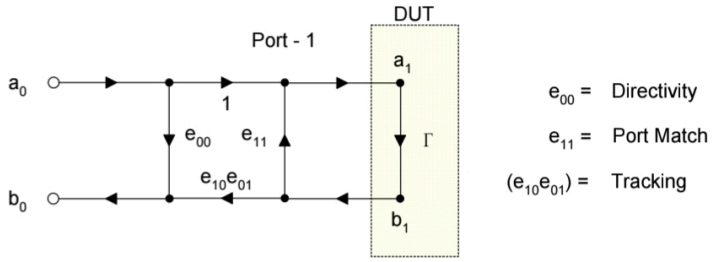


Figura 4: gráfico de flujo de calibración de un puerto y tres términos.

3.9 RF-09: Alimentación

Las tensiones de alimentación deben encontrarse entre 3,3 y 5V.

4. Requerimientos no funcionales

4.1 RNF-01: Tamaño de placa de PCB

El tamaño final de la placa debe ser menor a 18x17.5 cm^2.

4.2 RNF-02: Rango de operación

Se debe poder operar entre 1800 y 2200 MHz

4.3 RNF-03: Impedancia característica

Las líneas de transmisión deben tener una impedancia característica de 50Ω

4.4 RNF-04: Resolución del barrido

Los generadores de radiofrecuencia deben permitir una resolución mínima de 1kHz.

4.5 RNF-05: Directividad mínima aceptable del acoplador direccional

La directividad mínima que debe presentar el acoplador direccional debe ser de 15dB en la banda de interés.

4.6 RNF-06: ROE mínima distinguible

La ROE mínima distinguible debe ser menor o igual a 1,5.

UNMDP-FI

Carrera: Ingeniería Electrónica

Proyecto:
Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia

Especificación Técnica



Versión 2.0

Fecha	Versión	Descripción	Autor/a
27/08/2026	1.0	Versión inicial	Victoria Carla Torres
21/09/2026	2.0	Corrección	Victoria Carla Torres

Índice

1. Introducción 4
1.1 Propósito del documento 4
1.2 Alcance del proyecto 4
1.3 Definiciones, acrónimos y abreviaturas 4
2. Hardware 5
2.1. ε-μVNA 5
2.1.1 CLK 5
2.1.2 Generadores de RF 5
2.1.3 Mezclador 6
2.1.4 Acoplador direccional 7
2.1.5 Amplificadores diferenciales 7
2.1.6 Alimentación 7
2.1.7 Conectores 7
2.2 Diseño PCB 7
3. Firmware 9
4. Software 10
5. Pruebas técnicas 11
6. Referencias 11

1. Introducción

Este documento corresponde a la Especificación Técnica para la solución “Analizador Vectorial de Redes para medición no invasiva de glucemia”. Esta especificación se ha estructurado basándose en la información mencionada en el documento de Especificación Funcional (EF) “EF_Torres-v1” .

1.1 Propósito del documento

El presente documento provee información detallada de cómo debe funcionar el sistema, cuáles son sus comportamientos deseados y cómo se debe construir, con base en los requerimientos anteriormente definidos en la EF.

Se dirige al desarrollador del proyecto (quien lo va a construir) y a los solicitantes del proyecto, directores y evaluadores.

1.2 Alcance del proyecto

El proyecto propone el diseño, desarrollo, calibración y ensayo de un Analizador Vectorial de Redes (VNA) que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 GHz hasta los 2,2 GHz, adaptado para medición no invasiva de glucemia. Se utilizan circuitos integrados comerciales, dispositivos de montaje superficial (*Surface-Mount Device* - SMD), y conectores. Se aplican técnicas de reflectometría y de diseño en radiofrecuencia. Para realizar las mediciones se utiliza instrumental de laboratorio provisto por el Laboratorio de Comunicaciones (LAC) de la Facultad de Ingeniería de la Universidad Nacional de Mar del Plata (UNMDP).

1.3 Definiciones, acrónimos y abreviaturas

Nombre	Descripción
VNA	Analizador Vectorial de Redes
UHF	Ultra High Frequency
SMD	Surface - Mount Device
LAC	Laboratorio de Comunicaciones
UNMDP	Universidad Nacional de Mar del Plata
PCB	Printed Circuit Board
IC	Integrated Circuit
DUT	Device Under Test
SMA	SubMiniature version A
FI	Frecuencia Intermedia
ROE	Razón de Onda Estacionaria
PC	Personal Computer

2. Hardware

El sistema general está compuesto por tres módulos principales, que se pueden observar en la figura 1. Se encargan, en términos generales: ε-µVNA, de la generación de radiofrecuencias (RF), mediciones y de trasladarlas a una Frecuencia Intermedia (FI) para poder luego adquirirlas; PC, realiza la adquisición, procesamiento de señales y muestra de resultados; y microcontrolador que realiza los cálculos para luego programar los registros de los generadores de RF a partir de las frecuencias elegidas por el usuario.

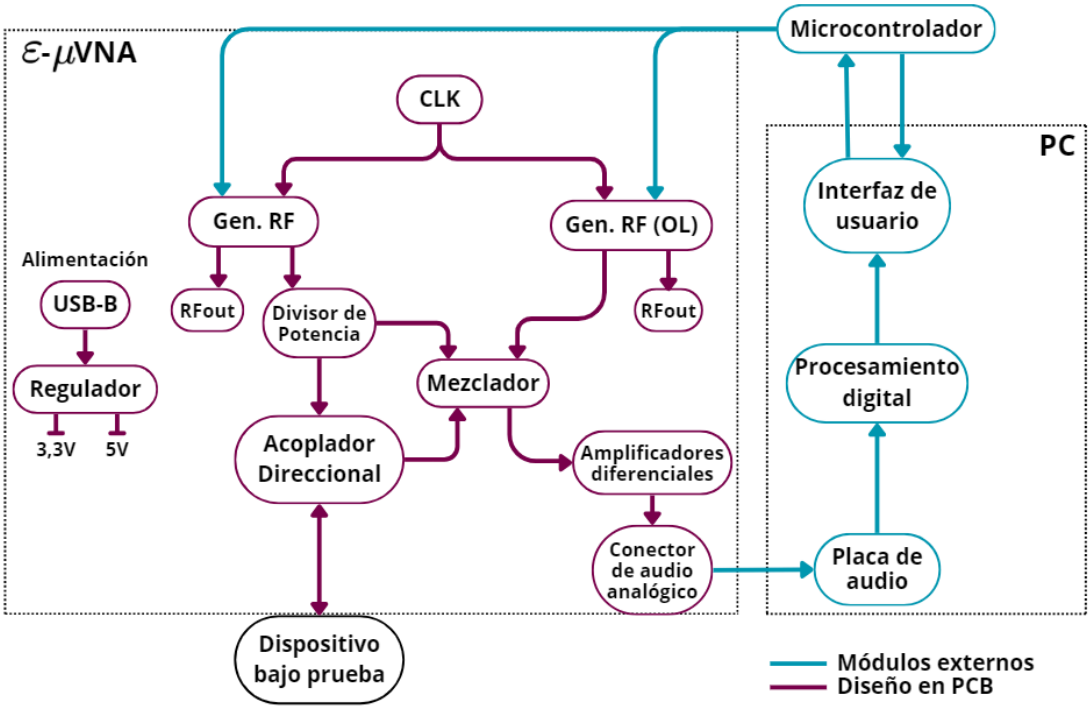


Figura 1: diagrama en bloques del sistema

2.1. ε-µVNA

La construcción del dispositivo ε-µVNA consiste en una placa de circuito impreso (PCB - Printed Circuit Board). Los integrados a utilizar y sus funciones, según los bloques presentados en la figura 1, son:

2.1.1 CLK

Oscilador a cristal ECS-2200BX-100 [1]. Se utiliza como referencia para los generadores de RF, junto con un buffer 74LVC2G07W6-7 [2] que permite bifurcar la señal para que ingrese a los generadores en forma independiente.

2.1.2 Generadores de RF

Basados en el IC ADF4351[3], sintetizador de banda ancha con VCO integrado, generan señales de frecuencia programable por interfaz serie desde 35 a 4400 MHz con potencia de salida programable, y tensión de alimentación de 3 a 3,6 V. Uno se utiliza como señal incidente en el acoplador direccional, mientras que el otro hace las veces de oscilador local para el proceso de heterodinaje junto con el mezclador. El esquemático general es el que se ve en la figura 2. Allí se observa una salida auxiliar para medición de las frecuencias generadas, un led con postes para verificar el enganche, capacitores de desacople en la alimentación y para estabilidad en pines requeridos por la hoja de datos (como TEMP, Vcom, y Vref). Entre los pines de Vtune y CPout

está el filtro externo del lazo de control. Fue diseñado en la aplicación de Analog Devices ADISIMPLL 5.9 [4], para cumplir con un margen de fase de 49° y ancho de banda de 9,9 kHz.

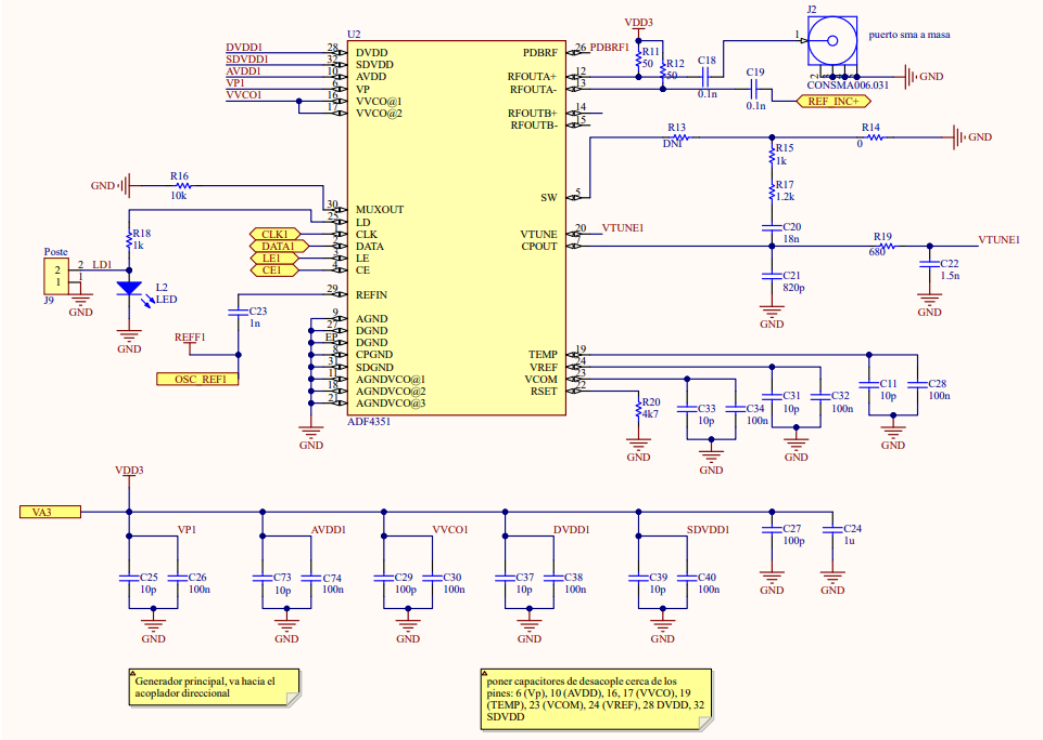


Figura 2: esquemático ADF4351.

2.1.3 Mezclador

Consiste en el IC ADL5802 [5], mezclador activo de dos canales que funciona de 100 MHz a 6 GHz con una ganancia de conversión de potencia de 1,6dB, y se alimenta con 5V. Tiene salidas diferenciales de 200Ω de colector abierto, por lo que se colocan resistencias de pull-up. Se agregan filtros RC simples de frecuencia de corte para atenuar la frecuencia suma. Se puede observar el esquemático en la figura 3.

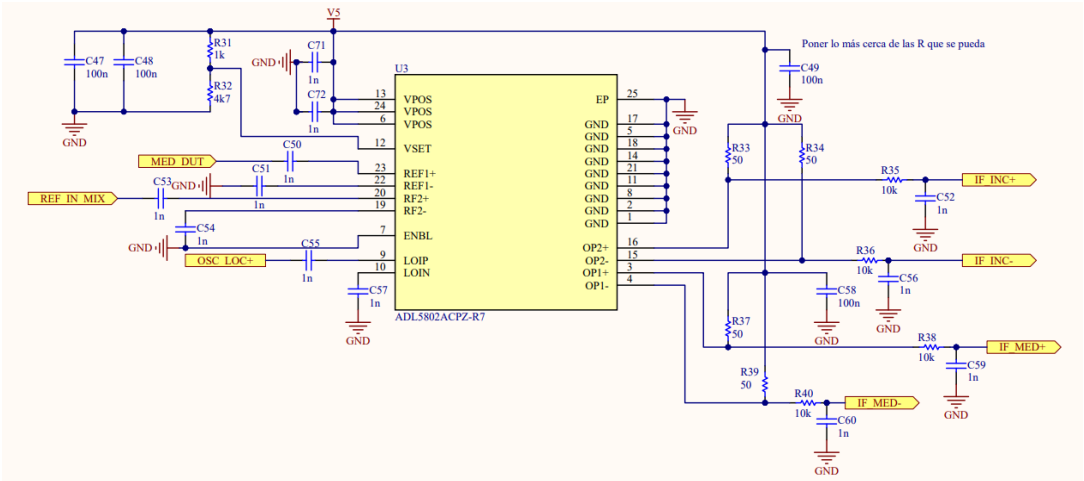


Figura 3: esquemático ADL5802.

2.1.4 Acoplador direccional

Basado en un puente de Wheatstone, red de resistencias que permite la comparación de impedancias con un ancho de banda teórico infinito. Como se observa en la figura 4, una parte de la señal que proviene del generador de RF se desvía hacia el mezclador, mientras que otra excita el puente resistivo. Este posee dos transformadores de RF TC1-1-13M+ [6] en cascada, que permiten medir la señal reflejada de manera desbalanceada y filtrar el ruido de modo común que provocaría utilizar solo uno. Las resistencias de 68Ω logran que la impedancia vista desde el puerto de salida (J1) sea 50Ω.

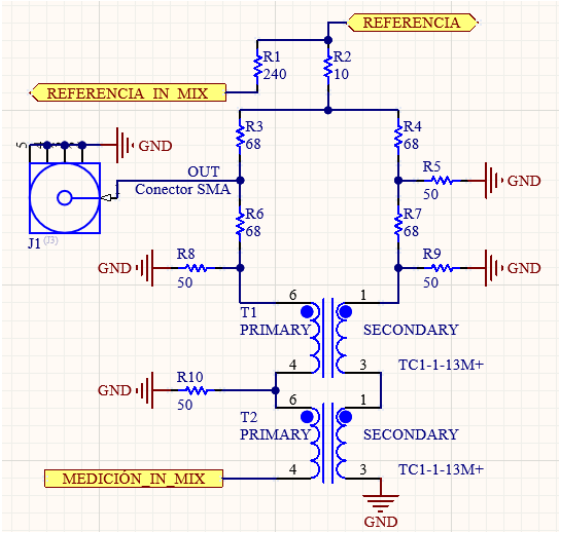


Figura 4: puente resistivo

2.1.5 Amplificadores diferenciales

Amplificadores operacionales MCP6023 [7]. Son restadores que toman la señal en FI proveniente del mezclador y la acondicionan para la entrada de línea de audio analógica de la PC, permiten obtener una señal desbalanceada y amplificada.

2.1.6 Alimentación

La alimentación general de 5V puede provenir de un USB-B o de una fuente de laboratorio. Un regulador de tensión UA78M33CDCYR [8] provee los 3,3V necesarios para alimentar los generadores de RF y el oscilador.

2.1.7 Conectores

Los conectores a utilizar son: SMA para mediciones de RF y entrada de referencia auxiliar; conector de audio analógico para entrada a la PC; USB-B para alimentación y tiras de pines junto con cables puente para escribir los registros de los ADF4351, entrada de alimentación auxiliar y medición de enganche.

2.2 Diseño PCB

La implementación en PCB es diseñada en el software Altium Designer, del que se dispone una licencia estudiantil gracias al programa Altium Student Lab. El diseño posee cuatro capas, debido a la dificultad para rutear pistas en dos capas. Un ejemplo es el caso de los pines Vtune y CPout de los ADF4351, que requieren atravesar caminos de RF por lo que se interrumpiría el camino de la corriente de retorno. Además, las pistas son susceptibles a interferencias electromagnéticas. Otro ejemplo es poder llevar alimentación a regiones rodeadas

de pistas de RF, por los mismos motivos.

Los componentes se sueldan en la capa 1. La capa 2 corresponde a un plano de masa continuo, la capa 3 se destina a alimentación. La última capa se usa para ruteo.

Para las trazas de RF se utilizaron líneas de transmisión coplanares de impedancia controlada de 50Ω, y vía stitching para evitar resonancias dentro del dieléctrico y confinar los campos electromagnéticos a las líneas de transmisión. También se evitaron, dentro de lo posible, ángulos con transiciones abruptas, se usó ruteo curvo para mantener la integridad de la señal.

Los componentes dispuestos sobre el PCB se modelizan en la figura 5.

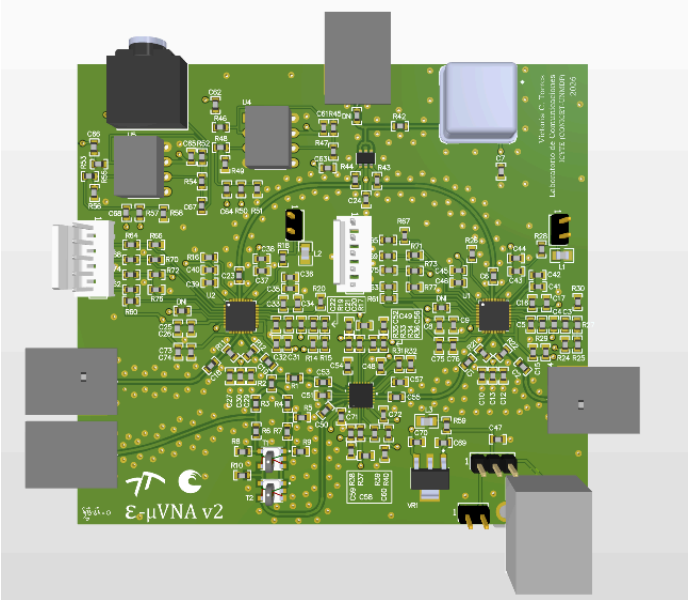


Figura 5: modelo 3D de la implementación en PCB.

2.2. PC

En la figura 6 se puede observar un esquema del hardware que se utiliza para comunicación con el microcontrolador y el ε-µVNA. Se utilizan puertos USB para alimentación de ambas placas. El software se comunica con el microcontrolador por medio de un puerto serie, y recibe las señales del ε-µVNA por medio de la entrada de línea, donde el canal derecho corresponde a la señal medida (proveniente del dispositivo bajo prueba - DUT) y el canal izquierdo tiene la referencia. En general, las entradas de línea poseen una impedancia de entrada de entre 10 y 47kΩ [9] y según mediciones realizadas en el laboratorio, saturan en alrededor de 2Vpp.

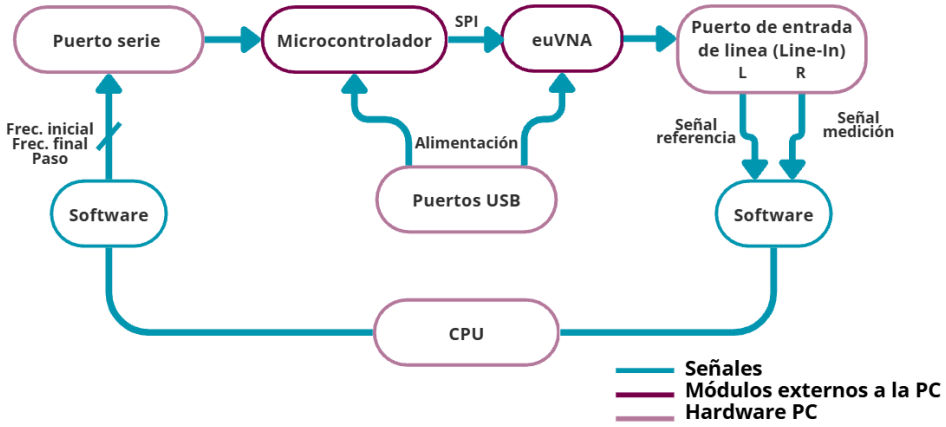


Figura 6: esquema de hardware de PC que se utiliza en el proyecto.

2.3 Microcontrolador

El microcontrolador se implementa utilizando una placa Arduino Uno, que se conecta con el ε-μVNA por medio de cables puente. Se encarga de recibir los datos del software vía puerto serie (frecuencia inicial, frecuencia final y paso) y calcula los registros que luego escribe en los ADF4351 con un protocolo SPI. La transferencia de datos a los registros internos de los ADF es controlada por los pines CLK, DATA, LE y CE.

3. Firmware

El firmware se programa en la plataforma Arduino IDE, en lenguaje C++. El diagrama de flujo del bucle principal es el que muestra la figura 7.

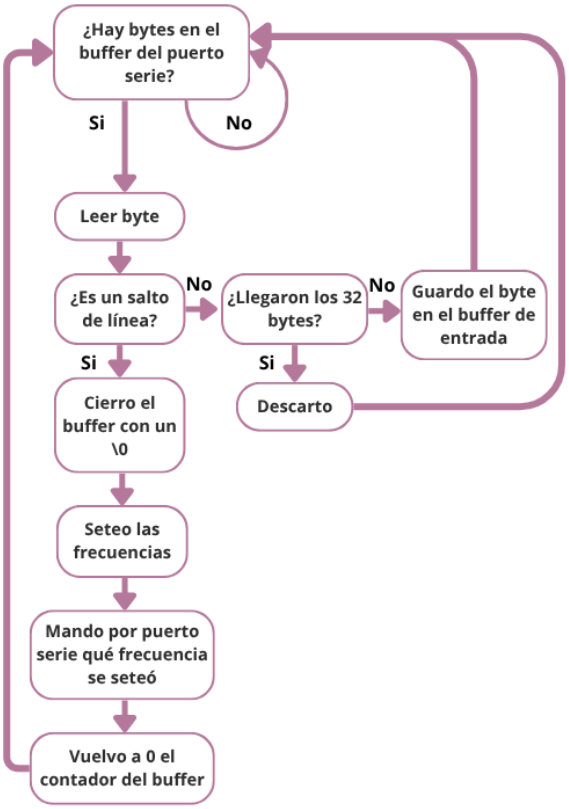


Figura 7: diagrama de flujo del bucle principal del microcontrolador.

En el setup se configuran los pines de salida y la comunicación SPI a 115200 baudios. Se esperan 100 ms y se imprime por puerto serie “Listo”.

Dentro del procesamiento del comando, verifica que la frecuencia inicial sea positiva y ajusta para el ADF2 un offset equivalente a la frecuencia de FI. Luego setea las frecuencias, lo que equivale a calcular los registros como se observa en la figura 8. Por último actualiza los ADF enviando los registros por el pin MOSI del SPI.

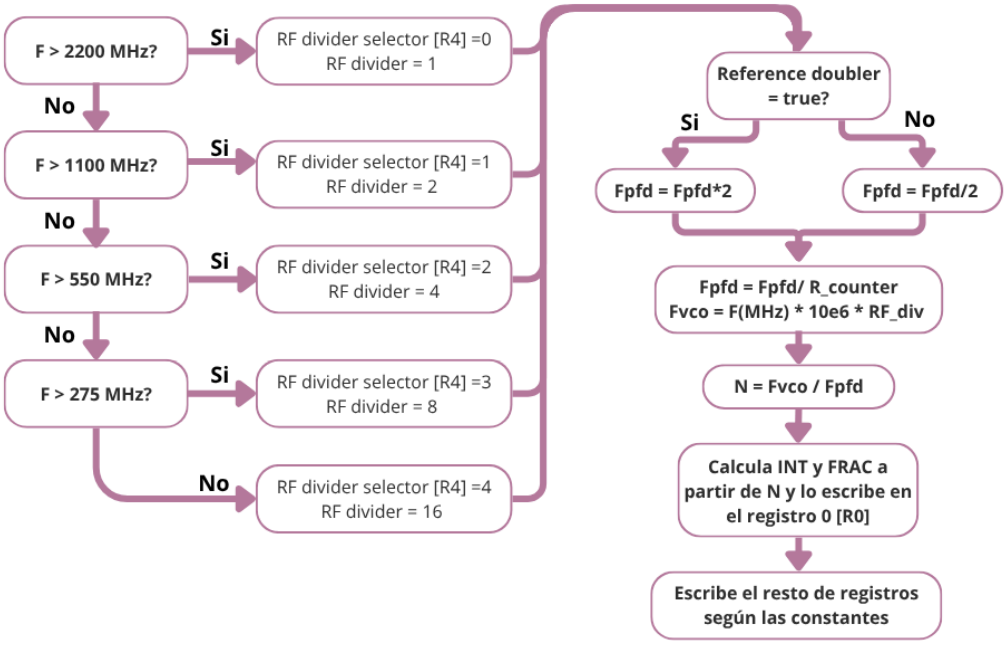


Figura 8: diagrama de flujo del cálculo de registros

4. Software

El software se programa en lenguaje Python, dentro de la aplicación Visual Studio Code. Se puede observar un diagrama de flujos general en la figura 9.

Las bibliotecas a utilizar son, como mínimo: numpy, matplotlib, serial, sounddevice y scipy.signal.

Entre las configuraciones iniciales se encuentran el puerto COM, baud rate, frecuencias de inicio, fin y paso, frecuencia de muestreo de audio, duración de la captura de audio, frecuencia central, ancho de banda y orden del filtro FIR y nombre de los archivos de salida.

El vector de frecuencias se genera con un ciclo for. Luego se diseña el filtro FIR pasabanda con la función “firwin” perteneciente a la biblioteca SciPy. También se establece la conexión serie con la función “serial” de la biblioteca Serial.

Para la adquisición se usa la función “rec” y se obtienen vectores de datos del tipo float64. Se recorta la primera parte para eliminar el transitorio, y se separan los canales en referencia y medición (canal izquierdo y derecho respectivamente).

La FFT compleja de ambos canales se calcula con la función “fft” de la librería NumPy, usando la ventana flattop que permite obtener mejor precisión en amplitud a comparación de otras. Con las funciones “searchsorted” y “argmax” de NumPy se busca el índice del bin de mayor magnitud dentro de la banda del filtro, y luego se calcula S11 en base a los valores obtenidos en la FFT, con el índice mencionado.

Para la calibración se implementa el método SOL de corrección de errores. que consiste en utilizar tres cargas de valores conocidos (Short-Open-Load) para calcular incógnitas asociadas a errores debidos a las adaptaciones de puerto, pérdidas por conectores y errores de directividad. Los valores obtenidos se guardan en un archivo CSV para poder reutilizarlos para mediciones posteriores. En ese caso, se arman nuevamente los vectores de valores complejos leyendo cada línea del archivo.

Por último, se grafican el módulo y fase del parámetro S11 en función de la frecuencia con funciones de la biblioteca MatplotLib.

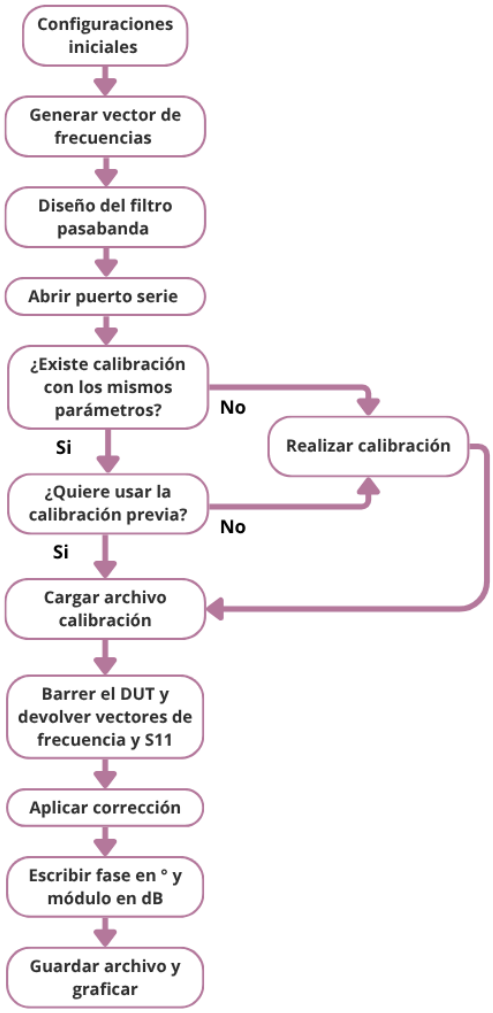


Figura 9: diagrama de flujo general del software

5. Pruebas técnicas

Las pruebas que verifican el correcto funcionamiento del dispositivo son, en principio:

- Medición de tensiones de alimentación.
- Medición de frecuencia del CLK.
- Medición de señal temporal de CLK.
- Prueba con entrada auxiliar de CLK.
- Medición de generación de frecuencia única por parte de ambos generadores ADF.
- Medición de generación de barridos de frecuencias, en ambos generadores ADF.
- Medición de salida de IF a la salida de los amplificadores operacionales.
- Adquisición de audio desde la PC: niveles de señal, FFT.
- Mediciones con cargas conocidas antes y después de calibrar.
- Medición con sensor de glucosa.

6. Referencias

[1] Cristal oscilador 10MHz, Ecs Inc. Datasheet: https://ecsxtal.com/store/pdf/ecs-2200x.pdf

[2] Buffer, Diodes Incorporated. Datasheet: https://www.diodes.com/assets/Datasheets/74LVC2G07.pdf

[3] Sintetizador de RF ADF4351, Analog Devices. Datasheet: https://www.analog.com/media/en/technical-documentation/data-sheets/ADF4351.pdf

[4] Herramienta de diseño ADISIMPLL 5.9, Analog Devices. descargable en https://www.analog.com/en/lp/resources/adisimpll.html

[5] Mezclador ADL5802, Analog Devices. Datasheet: https://www.analog.com/media/en/technical-documentation/data-sheets/ADL5802.pdf

[6] Transformador de RF Mini-Circuits. Datasheet: https://www.minicircuits.com/pdfs/TC1-1-13M+.pdf

[7] Amplificadores operacionales MCP6023, Microchip Technology. Datasheet: https://ww1.microchip.com/downloads/en/DeviceDoc/20001685E.pdf

[8] Regulador de tensión lineal UA78M33CDCYR. Texas Instruments. Datasheet: https://www.ti.com/lit/ds/slvs059v/slvs059v.pdf?ts=1789190587458

[9] https://www.hispasonic.com/tutoriales/entradas-mic-inst-line-son-para-sirven/43068

UNMDP-FI

Carrera: Ingeniería Electrónica

Proyecto:
Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia

Plan de Pruebas



Versión 1.0

1. Alcance

El proyecto propone el diseño, desarrollo, calibración y ensayo de un Analizador Vectorial de Redes (VNA) que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 GHz hasta los 2,2 GHz, adaptado para medición no invasiva de glucemia. Se utilizan circuitos integrados comerciales, dispositivos de montaje superficial (*Surface-Mount Device* - SMD), y conectores. Se aplican técnicas de reflectometría y de diseño en radiofrecuencia.

A continuación se presenta el plan de pruebas para validar el correcto funcionamiento de los módulos que componen el proyecto. Se las puede clasificar en:

- Pruebas unitarias (PU): se concentran en cada componente o módulo del sistema, de forma aislada.
- Pruebas integrales (PI): se realizan a medida que se integran los bloques de componentes.
- Pruebas de homologación (PH): se realizan sobre el sistema completo, validando los requerimientos planteados al inicio del proyecto.
- Pruebas de regresión (PR): se realizan cuando se modifica, ajusta o agrega un módulo o componente.
- Pruebas de aceptación (PA): son las que realiza el usuario final. Se enfocan en características generales y la funcionalidad del sistema.

2. Ambientes de prueba

Todas las pruebas sobre componentes de circuitos integrados se realizarán en un ambiente de laboratorio, con instrumental proporcionado por el Laboratorio de Comunicaciones (LAC) de la Facultad de Ingeniería de la Universidad Nacional de Mar del Plata (UNMDP).

Las pruebas de adquisición se realizan en la aplicación para PC Audacity, las de software en Visual Studio Code.

3. Recursos, herramientas e instrumentos

El instrumental a utilizar comprende los elementos:

- Computadora personal (PC);
- Multímetro Uni-T UT55 [1];
- Osciloscopio Tektronix TDS210 [2];
- Frecuencímetro Optoelectronics Handi-Counter 2810 [3];
- Analizador de espectros Rigol RSA3030N [4];
- Fuente de alimentación regulable Kenwood PA36-3A [5];
- Generador de señales HP 33120A[6];
- Atenuador 110dB HP 8496B [7];
- Cargas SMA estándar;
- Cables y conectores.

4. Políticas de trabajo

Los resultados de las pruebas se reportan a los directores del proyecto: Dr. Ing. Ramiro Avalos Ribas e Ing. Brian Gluzman, quienes participan y supervisan el proceso.

5. Pruebas

5.1 Medición de tensiones de alimentación.

Prueba las conexiones de conectores y cables, planos de alimentación y regulador de tensión.
Tipo de prueba: unitaria.
Resultado esperado: obtener 5±0,2 V en la entrada del regulador, las alimentaciones de los operacionales y mezclador, y 3,3±0,1V a la salida del regulador, en los generadores de RF y oscilador.
Instrumental: Fuente de alimentación, multímetro.
Procedimiento:

1) Setear la fuente de alimentación en 5V con una corriente máxima de 0,5A.
2) Conectar la fuente de alimentación a los postes de la placa, con el jumper en posición alimentación externa.
3) Medir la entrada y salida del regulador de tensión con el multímetro.
4) Medir el pin 24, 6 o 13 del mixer.
5) Medir el pin 10, 28 o 32 de los generadores de RF.
6) Medir el pin 8 (según la hoja de datos) del oscilador.

5.2 Medición de frecuencia y señal temporal del CLK.

Prueba el oscilador a cristal y su integración con el buffer, respectivamente.
Tipo de prueba: unitaria.
Resultado esperado: tener una señal de frecuencia 10MHz, con cruces por cero sin overshoot o ruido apreciable en el osciloscopio.
Instrumental: frecuencímetro y osciloscopio.
Procedimiento:

1) Para medir frecuencia, usar el frecuencímetro. Alimentar el oscilador conectando los pines 8 y 1 a 3,3V y 4 a GND. Medir entre output (pin 5) y GND.
2) Para medir forma de onda, alimentar el ε-μVNA y medir la salida del buffer (pin 4 o 6).

5.3 Prueba con entrada auxiliar de CLK.

Prueba cómo se comportan los ADF4351 con un cambio en la frecuencia de CLK.
Tipo de prueba: de regresión.
Resultado esperado: con los ajustes pertinentes en el firmware, debería generar correctamente la señal de RF.
Instrumental: PC, generador de señales, analizador de espectros, atenuador.
Procedimiento:

1) Soldar una resistencia de 0Ω en “DNI” dejando R42 sin soldar, para alimentar con fuente externa.
2) Setear el generador de señales en una señal senoidal o cuadrada de 3,3Vpp, de frecuencia entre 10 y 250 MHz. Tener en cuenta las consideraciones de la hoja de datos del ADF4351 para la frecuencia elegida (por ejemplo una fpfd menor a 125 kHz).
3) Escribir los registros para generar una frecuencia arbitraria.
4) Conectar el analizador de espectros protegiendo su entrada con el atenuador.
5) Medir las salidas auxiliares de los ADF4351. Verificar que genera la frecuencia pedida con la potencia programada.
6) Volver a soldar la resistencia R42 dejando DNI sin insertar para usar el oscilador a cristal como entrada de referencia default.

5.4 Medición de generación de frecuencia única por parte de ambos generadores ADF.

Prueba el firmware y la correcta generación de frecuencia por parte de los ADF4351.
Tipo de prueba: unitaria.
Resultado esperado: una señal de la potencia elegida, en la frecuencia elegida.
Instrumental: PC, analizador de espectros, atenuador.
Procedimiento:

1) Alimentar el ε-μVNA.
2) Programar una frecuencia en el Arduino para que escriba los registros de los ADF.
3) Conectar el analizador de espectros protegiendo su entrada con el atenuador, y medir las salidas auxiliares de los ADF. Verificar que genera la frecuencia pedida con la potencia programada, y que los LEDs de enganche se encienden.

5.5 Medición de generación de barridos de frecuencias, en ambos generadores ADF.

Prueba la sección de barrido del software, y la generación de los ADF4351.
Tipo de prueba: unitaria.
Resultado esperado: generación de las frecuencias con la potencia programada en un rango de 1800 a 2200 MHz.
Instrumental: PC, analizador de espectros.
Procedimiento:

1) Alimentar el ε-μVNA.
2) Conectar el analizador de espectros protegiendo su entrada con el atenuador, y medir las salidas auxiliares de los ADF4351.
3) Programar el barrido desde el software, con un ciclo que mande cada frecuencia por puerto serie al Arduino.
4) Verificar que genera cada frecuencia pedida con la potencia programada, que los LEDs detectores de enganche se encienden y que el enganche se produce dentro del tiempo programado.

5.7 Medición de salida de IF a la salida de los amplificadores operacionales.

Prueba el módulo FI: mezclador ADL5802 y amplificadores operacionales MCP6023.
Tipo de prueba: unitaria.
Resultado esperado: observar señales de referencia y medición de frecuencia FI.
Instrumental: PC, osciloscopio (opcional).
Procedimiento:

1) Alimentar el ε-μVNA.
2) Generar una frecuencia única en el ADF principal y la misma con un offset de FI en el ADF que se usa como oscilador local.
3) Poner una carga conocida como dispositivo bajo prueba.
4) Adquirir la señal con el programa de PC Audacity o medir la salida de los amplificadores operacionales con un osciloscopio. Tener en cuenta que pueden ser señales de muy baja amplitud.
5) Observar que se generen las señales incidente/referencia y reflejada/medición, donde la incidente debe tener mayor (o igual) amplitud y se debe tomar de referencia para medir la fase de la reflejada. Si, por ejemplo, el dispositivo bajo prueba es un cortocircuito, la señal reflejada debería tener la fase rotada 180°. Tener en cuenta que puede haber diferencias debido a que no se ha implementado la calibración.

5.8 Adquisición de audio desde la PC: niveles de señal, FFT.

Prueba la adquisición y el cálculo de la FFT a partir de una única frecuencia.
Tipo de prueba: de integración.
Resultado esperado: observar señales de audio de la frecuencia de FI con amplitudes similares a las medidas en la prueba 5.7. Obtener la FFT con un pico máximo en la frecuencia de FI, mínimo 10dB por encima de la tercera armónica.
Instrumental: PC.
Procedimiento:

1) Configurar los generadores de RF para obtener una salida en FI.
2) Programar la adquisición de audio y el cálculo de la FFT.
3) Graficar la onda adquirida en función del tiempo y la FFT.

5.9 Mediciones con cargas conocidas antes y después de calibrar.

Prueba el software integrado con el ε-μVNA.
Tipo de prueba: de homologación.
Resultado esperado: antes de calibrar deberían verse variaciones con respecto a lo esperado para las cargas estándar. Luego de calibrar se debería observar correctamente o con mínimas variaciones el coeficiente de reflexión esperado
Instrumental: PC.
Procedimiento antes de calibrar:

1) Programar el Arduino para que espere las frecuencias por puerto serie y calcule y escriba los registros.
2) Colocar como dispositivo bajo prueba una carga de valor conocido (cortocircuito, circuito abierto, 50Ω)
3) Ejecutar el barrido y calcular el S11 para todas las frecuencias.
4) Mostrar en un gráfico de módulo [dB] y fase [°] el S11 calculado.

Procedimiento luego de calibrar:

1) Elegir un barrido y realizar la calibración para ese barrido.
2) Colocar una carga de coeficiente de reflexión conocido
3) Correr el programa y evaluar resultados.

5.10 Medición con sensor de glucosa.

Prueba el sistema completo en su capacidad para detectar una resonancia entre 1800 y 2200 MHz.
Tipo de prueba: de aceptación.
Resultado esperado: debería observarse una curva de módulo cercano a 0 y luego un pico de resonancia alrededor de los 2000 MHz.
Instrumental: PC.
Procedimiento:

1) Realizar la calibración para un barrido de entre 1800 y 2200MHz, con un paso de 1MHz o menor.
2) Conectar el sensor de glucemia como dispositivo bajo prueba.
3) Correr el programa y evaluar resultados.

6. Referencias

[1] Manual Tester UT55. Disponible en:
https://www.tme.com/Document/9d0acda5b1c310707bd15e5f784f9bae/ut55_en.pdf
[2] Manual osciloscopio Series TDS200. Disponible en:
https://www.tek.com/en/oscilloscope/tds210-manual/tds200-series-user-manual
[3] Manual frecuencímetro. Disponible en:
https://www.optoelectronics.com/_files/ugd/c0ec7f_ab98242f8c3842b889cf7a8ff2227f43.pdf
[4] Manual analizador de espectros Rigol Series RSA3000. Disponible en:
https://telonic.co.uk/jg/wp-content/uploads/2021/05/Rigol-RSA3000-UserGuide.pdf
[5] Datasheet Kenwood PA36-3A. Disponble en: https://www.ntecusa.com/docs/pa-a.pdf
[6] Datasheet HP 33120A. Disponible en:
https://www.keysight.com/us/en/assets/7018-06809/data-sheets/5968-0125.pdf
[7] Datasheet atenuador HP8496B. Disponible en:
https://www.keysight.com/us/en/assets/9018-01300/user-manuals/9018-01300.pdf?success=true

Bibliografía

[1] E. Topsakal, T. Karacolak, and E. C. Moreland. "Glucose-dependent dielectric properties of blood plasma." In: *Proceedings of the XXXth URSI General Assembly and Scientific Symposium* (2011). DOI: 10.1109/URSIGASS.2011.6051324.

[2] World Health Organization. *Global report on diabetes.* Sitio Web Oficial. 2016. URL: https://www.who.int/publications/i/item/9789241565257.

[3] World Health Organization. *Diabetes.* Sitio Web Oficial. Nov. 2024. URL: https://www.who.int/news-room/fact-sheets/detail/diabetes.

[4] Sitio Web Oficial. URL: https://www.freestylelibre.com.ar/.

[5] Tektronix Inc. *Vector Network Analysis: A Practical Introduction.* 2023. URL: https://download.tek.com/document/70W_60918_0_Tek_VNA_PR1.pdf?_gl=1*wh717q*_gcl_au*MjQyMjEyMTE5LjE3ODk0ODkzMjU.*_ga*NjkwMjAxNzcxLjE3ODk0ODkzMjU.*_ga_1HMYS1JH9M*czE3OTAxODY4MjkkbzIkZzAkdDE3OTAxODY4MzIkajU3JGwwJGgwJGRFXzVmVFRzOGMwbmFqaG8zXzY5VjdXYVRUVWRWTDAybkh3.

[6] Hewlett Packard Inc. Rytting D. "Network analyzer error models and calibration methods". In: *HP* (1998).

[7] Urvashi Sengal. *Directional Couplers: Their Operation and Application.* Mini-Circuits. Oct. 2020. URL: https://blog.minicircuits.com/directional-couplers-their-operation-and-application/.

[8] Keysight Technologies. *778D Coaxial Dual-Directional Coupler, 100 MHz to 2 GHz.* URL: https://www.keysight.com/us/en/product/778D/coaxial-dualdirectional-coupler-100-mhz-to-2-ghz.html.

[9] Mini-Circuits. *ZABDC20-322H-S+ Coaxial Directional Coupler.* URL: https://www.minicircuits.com/WebStore/dashboard.html?model=ZABDC20-322H-S%2B.

[10] Altium. *Via Stitching and Via Shielding in PCB Design.* s. f. URL: https://www.altium.com/documentation/altium-designer/via-stitching-via-shielding-pcb.

[11] Altium. *Consideraciones para el diseño de PCB de alta velocidad: Prácticas recomendadas para la colocación y el enrutamiento de condensadores de desacoplo.* Dec. 2020. URL: https://resources.altium.com/es/p/high-speed-pcb-design-considerations-bypass-capacitor-place-and-route-tips.

[12] Analog Devices. *HMC601 75 dB logarithmic detector/controller.* https://www.analog.com/en/products/hmc601.html.

[13] Dimitris G. Manolakis and Vinay K. Ingle. *Applied Digital Signal Processing: Theory and Practice.* Cambridge, UK: Cambridge University Press, 2011. ISBN: 978-0-521-11002-0.

[14] Alan V. Oppenheim and Ronald W. Schafer. *Discrete-time signal processing, second edition.* Upper Saddle River, NJ: Prentice Hall, 1999. ISBN: 0-13-754920-2.

[15] Altium Limited. *Altium Designer.* Version 26.1.1. Licencia educativa. 2026. URL: https://www.altium.com.

[16] Analog Devices. *Sintetizador de RF ADF4351*. Datasheet. URL: https://www.analog.com/media/en/technical-documentation/data-sheets/ADF4351.pdf.

[17] Analog Devices. *ADIsimPLL*. Herramienta de diseño. Version 5.9. URL: https://www.analog.com/en/lp/resources/adisimpll.html.

[18] *ECS-2200X Series SMD Clock Oscillator Datasheet*. Cristal oscilador 10MHz. ECS Inc. International. URL: https://ecsxtal.com/store/pdf/ecs-2200x.pdf.

[19] *74LVC2G07: Dual Buffers with Open Drain Outputs*. DS35162. Datasheet. Diodes Incorporated. URL: https://www.diodes.com/assets/Datasheets/74LVC2G07.pdf.

[20] Mini-Circuits. *TC1-1-13M+ Surface Mount RF Transformer, 50Ω, 4.5 to 3000 MHz*. Rev. G. Mini-Circuits. 2023. URL: https://www.minicircuits.com/pdfs/TC1-1-13M+.pdf.

[21] Michael Margraf. *uSimmics*. Version 5.9. Anteriormente conocido como QucsStudio. Jan. 10, 2026. URL: https://dd6um.darc.de/QucsStudio/.

[22] *MCP6021/1R/2/3/4: Rail-to-Rail Input/Output, 10 MHz Op Amps*. DS20001685E. Microchip Technology Inc. 2017. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/20001685E.pdf (visited on 10/28/2026).

[23] Texas Instruments. *UA78M 500-mA, Positive-Voltage Linear Regulator Datasheet*. Rev. V, Document SLVS059V. Texas Instruments Incorporated. Feb. 2025. URL: https://www.ti.com/lit/ds/slvs059v/slvs059v.pdf.

[24] PCBWay. *Printed Circuit Board Fabrication and Assembly Services*. Shenzhen PCBWay Technology Co., Ltd. URL: https://www.pcbway.com.

[25] Optoelectronics, Inc. *Model 2810 Handi-Counter Frequency Counter Operation Manual*. Frecuencímetro portátil de mano (10 Hz–3 GHz). Optoelectronics, Inc. 1998.

[26] Kenwood Corporation. *PA Series Regulated DC Power Supply Instruction Manual*. Modelo PA36-3A (0–36 V / 0–3 A). Kenwood Corporation. 1995.

[27] RIGOL Technologies, Inc. *RSA3030N 3 GHz Real-Time Spectrum Analyzer*. 2024. URL: https://www.rigolna.com/products/spectrum-analyzers/rsa3000/.

[28] Hewlett-Packard Company. *HP 8496B Manual Step Attenuator Operating and Service Manual*. Atenuador de paso manual 0–110 dB, DC–18 GHz. Hewlett-Packard Company. 1998. URL: https://www.keysight.com/us/en/assets/9018-01300/user-manuals/9018-01300.pdf.

[29] Audacity Team. *Audacity*. Version 3.7.6. Muse Group. URL: https://www.audacityteam.org/.